Computational Optimization

with Applications to Machine Learning

A Series of Lectures at Missouri S&T

Wenqing Hu [1]

[1]Department of Mathematics and Statistics, Missouri University of Science and Technology (formerly University of Missouri, Rolla), Rolla, Missouri, 65409, USA. Web: https://huwenqing0606.github.io/
Email: huwen@mst.edu

Preface

These lecture notes grew out of a series of lectures for the course MATH 5001: Computational Optimization taught at Missouri University of Science and Technology (formerly University of Missouri, Rolla) during Spring 2026.

These notes present a mixture of several topics, including optimization, machine learning, and computation, with a primary focus on optimization. We also provide a set of codes for students to experiment some of the algorithms discussed in the lectures. These codes can be found at https://github.com/huwenqing0606/Computational_Optimization.

We thank the participating students: Stanislav Butkovich, Erich Gozebina, Qitong Huang, Nikunj Pradhan, and Mohammed Sleiman.

We also thank our Interim Chair Professor John Singler of the Mathematics & Statistics Department for his support of the course.

Wenqing Hu
Rolla, Missouri
Spring 2026

Contents

1 Motivating Examples **5**

1.1 Supervised Learning . . . 5

1.2 Matrix Optimizations . . . 9

1.3 Experiment: Activation Functions . . . 10

2 Convex Functions **11**

2.1 Optimization problem: set-up and its solutions . . . 11

2.2 Convexity . . . 11

2.3 Taylor's theorem and convexity in Taylor's expansion . . . 13

2.4 Optimality Conditions . . . 19

2.5 Experiment: Nonconvexity of the Loss Landscape of Neural Networks . . . 20

3 Gradient Descent and Line Search Methods **23**

3.1 Gradient Descent . . . 23

3.2 Back Propagation and GD on Neural Networks . . . 28

3.3 Online Principle Component Analysis (PCA) . . . 31

3.4 Line Search Methods . . . 32

3.5 Convergence to Approximate Second Order Necessary Points . . . 33

3.6 Experiment: Learning a Neural Network via Gradient Descent and Backpropagation 35

4 Accelerated Gradient Methods **36**

4.1 Polyak's heavy ball method . . . 36

4.2 Nesterov's Accelerated Gradient Descent (AGD) . . . 37

4.2.1 Convergence Proof of Nesterov's method: Quadratic Case. . . . 38

4.2.2 Convergence Proof of Nesterov's method: Strongly Convex Case. . . . 40

4.2.3 Convergence Proof of Nesterov's method: Convex Case. . . . 45

4.2.4 Nesterov's method reaches first-order optimality . . . 48

4.3 Conjugate gradient method . . . 50

4.4 Experiment: GD vs Accelerated GD . . . 51

5 Stochastic Gradient Methods **53**
5.1 SGD . . . 53
5.2 Stochastic Variance-Reduced Gradient Descent (SVRG) and its variants . . . 57
5.2.1 SVRG and its Convergence Proof . . . 57
5.2.2 SAG, SAGA, SARAH, SPIDER, STORM . . . 58
5.3 Escape from saddle points and nonconvex problems, SGLD, Perturbed GD . . . 63
5.4 Coordinate Descent and Stochastic Coordinate Descent . . . 64

6 Second Order Methods **66**
6.1 Newton's method . . . 66
6.2 Quasi–Newton: BFGS . . . 67
6.3 Stochastic Quasi–Newton . . . 69

7 Optimization in Modern Machine Learning **71**
7.1 Adaptive Methods . . . 71
7.1.1 SGD with momentum . . . 71
7.1.2 AdaGrad, RMSProp, AdaDelta . . . 71
7.1.3 Adam . . . 73
7.2 Training a Transformer: AdamW and Muon . . . 73
7.2.1 Transformer Neural Network . . . 73
7.2.2 AdamW . . . 76
7.2.3 Muon . . . 78
7.3 Dual Methods . . . 81
7.3.1 Dual Coordinate Descent/Ascent . . . 81
7.3.2 SDCA . . . 82
7.4 Optimization with Constraints and Discontinuous Objective Function . . . 84
7.4.1 Subgradient Projection Method, Frank–Wolfe Method . . . 84
7.4.2 ADMM . . . 85
7.4.3 Proximal operators and proximal algorithms . . . 86
7.5 Optimization and Generalization . . . 87
7.5.1 Generalization Bounds in Statistical Machine Learning . . . 87
7.5.2 Training Matters: Connecting Optimization with Generalization . . . 89
7.5.3 Experiments: Real Data Sets . . . 89

Chapter 1

Motivating Examples

1.1 Supervised Learning

Empirical Risk Minimization

Supervised Learning: Given training data points $(x_1, y_1), ..., (x_n, y_n)$, construct a learning model $y = g(x, \omega)$ that best fits the training data. Here ω stands for the parameters of the learning model, say $\omega = (\omega_1, ..., \omega_d)$.

Here (x_i, y_i) comes from an independent identically distributed family $(x_i, y_i) \sim p(x, y)$, where $p(x, y)$ is the joint density. The model $x \to y$ is a black-box $p(y|x)$, which is to be fit by $g(x, \omega)$.

"Loss function" $L(g(x, \omega), y)$, for example, can be $L(g(x, \omega), y) = (g(x, \omega) - y)^2$.

Empirical Risk Minimization (ERM):

$$\omega_n^* = \arg\min_{\omega} \frac{1}{n} \sum_{i=1}^{n} L(y_i, g(x_i, \omega)) \ . \tag{1.1}$$

Regularized Empirical Risk Minimization (R-ERM):

$$\omega_n^* = \arg\min_{\omega} \frac{1}{n} \sum_{i=1}^{n} L(y_i, g(x_i, \omega)) + \lambda R(\omega) \ . \tag{1.2}$$

For example, we can take $R(\omega) = \|\omega\|^2 = \omega_1^2 + ... + \omega_d^2$. This regularization helps to control very irregular minimizers (unwanted ω).

In general let $f_i(\omega) = L(y_i, g(x_i, \omega))$ or $f_i(\omega) = L(y_i, g(x_i, \omega)) + \lambda R(\omega)$, then the optimization problem is

$$\omega_n^* = \arg\min_{\omega} \frac{1}{n} \sum_{i=1}^{n} f_i(\omega) \ . \tag{1.3}$$

Key features of nonlinear optimization problem in machine learning: large–scale, nonconvex, ... etc.

Key problems in machine learning: optimization combined with generalization. "Population Loss": $\mathbf{E}L(g(x,\omega),y)$, minimizer

$$\omega^* = \arg\min_{\omega} \mathbf{E}L(g(x,\omega),y) \ .$$

Generalization Error: $\mathbf{E}L(y,g(x,\omega_n^*))$. Consistency: Do we have $\omega_n^* \to \omega^*$? At what speed?

Key problems in optimization: convergence, acceleration, variance reduction.

How can optimization be related to generalization? There are quite abstract notions related to this topic, such as Vapnik–Chervonenkis dimension (VC dimension) and Radmacher complexity, which we might touch later. Also, we want to look at the geometry of the loss landscape, which is closely related to neural network structure.

Loss Functions

Classification Problems: label $y = 1$ or -1. Choice of Loss function $L(y,g)$, $y = 1, -1$. 0/1 Loss: $\ell_{0/1}(y,g) = 1$ if $yg < 0$ and $\ell_{0/1}(y,g) = 0$ otherwise.

(1) Hinge Loss.

$$L(y,g) = \max(0, 1 - yg) \ ; \tag{1.4}$$

(2) Exponential Loss.

$$L(y,g) = \exp(-yg) \ ; \tag{1.5}$$

(3) Cross Entropy Loss.

$$L(y,g) = -\left(I_{\{y=1\}} \ln \frac{e^g}{e^g + e^{-g}} + I_{\{y=-1\}} \ln \frac{e^{-g}}{e^g + e^{-g}} \right) \ . \tag{1.6}$$

This is to use $p(y) = \dfrac{e^{yg}}{e^{yg} + e^{-yg}}$, $y = \pm 1$ and the binary cross entropy as

$$-(I_{\{y=1\}} \ln p(1) + I_{\{y=-1\}} \ln p(-1)) \ .$$

Regression Problems: Choice of Loss function $L(y,g)$.

(1) L^2–Loss.

$$L(y,g) = |y-g|_2^2 \ . \tag{1.7}$$

L^2–norm: $|x|_2^2 = x_1^2 + ... + x_d^2$

(2) L^1–Loss.

$$L(y,g) = |y-g|_1 \ . \tag{1.8}$$

L^1–norm: $|x|_1 = |x_1| + ... + |x_d|$.

(3) L^0–Loss.

$$L(y,g) = |y-g|_0 \ . \tag{1.9}$$

L^0–norm: $|x|_0 = \sharp\{i : x_i \neq 0, 1 \leq i \leq d\}$.

Regularized (penalize) term $R(\omega)$:

L^1–regularized $R(\omega) = |\omega|_1$;

L^0–regularized $R(\omega) = |\omega|_0$.

Learning Models

(1) Linear regression: $g(x,\omega) = \omega^T x$. $g(x,\omega) = \dfrac{1}{1+\exp(-\omega^T x)}$.

Least squares problem:

$$\min_{\omega\in\mathbb{R}^d} \frac{1}{2m}\sum_{j=1}^{m}(x_j^T\omega - y_j)^2 = \frac{1}{2m}|A\omega - y|_2^2 \tag{1.10}$$

Here training data $(x_1,y_1),...,(x_m,y_m)$ where $x_i \in \mathbb{R}^d$, $y \in \mathbb{R}$, and $A = \begin{pmatrix} x_1^T \\ ... \\ x_m^T \end{pmatrix}$.

Tikhonov regularization:

$$\min_{\omega} \frac{1}{2m}|A\omega - y|_2^2 + \lambda|\omega|_2^2 \ . \tag{1.11}$$

LASSO (Least Absolute Shrinkage and Selection Operator):

$$\min_{\omega} \frac{1}{2m}|A\omega - y|_2^2 + \lambda|\omega|_1 \ . \tag{1.12}$$

See [11].

(2) Support Vector Machines (SVM):

Set-up: $x_j \in \mathbb{R}^n$, $y_j \in \{1,-1\}$. Separating hyperplane $\omega^T x + \beta = 0$ where $\omega \in \mathbb{R}^n$ and $\beta \in \mathbb{R}$.

Classification Problem: Goal is to find a hyperplane $\omega^T x + \beta = 0$ such that it classifies the two kinds of data points most efficiently. The signed distance of any point $x \in \mathbb{R}^n$ to the hyperplane is given by $r = \dfrac{\omega^T x + \beta}{|\omega|}$. If the classification is good enough, we expect to have $\omega^T x_j + \beta > 0$ when $y = 1$ and $\omega^T x_j + \beta < 0$ when $y = -1$. After rescaling ω and β, we can then formulate the problem as looking for optimal ω and β such that $\omega^T x_j + \beta \geq 1$ when $y_j = 1$ and $\omega^T x_j + \beta \leq -1$ when $y_i = -1$. The closest few data points that match these two inequalities are called "support vectors". The distance to the separating hyperplane created by two support vectors of opposite type is

$$\left|\frac{1}{|\omega|}\right| + \left|\frac{-1}{|\omega|}\right| = \frac{2}{|\omega|} \ .$$

So we can formulate the following optimization problem

$$\max_{\omega\in\mathbb{R}^n,\beta\in\mathbb{R}} \frac{2}{|\omega|} \text{ such that } y_j(\omega^T x_j + \beta) \geq 1 \text{ for } j = 1,2,...,m \ .$$

Or in other words we have the *constrained* optimization problem

$$\min_{\omega\in\mathbb{R}^n,\beta\in\mathbb{R}} \frac{1}{2}|\omega|^2 \text{ such that } y_j(\omega^T x_j + \beta) \geq 1 \text{ for } j = 1,2,...,m \ . \tag{1.13}$$

"Soft margin": We allow the SVM to make errors on some training data points but we want to minimize the error. In fact, we allow some training data to violate $y_j(\omega^T x_j + \beta) \geq 1$, so that ideally we minimize

$$\min_{\omega\in\mathbb{R}^n,\beta\in\mathbb{R}} \frac{1}{2}|\omega|^2 + C\sum_{j=1}^{m}\ell_{0/1}(y_j(\omega^T x_j + \beta) - 1) \ .$$

Here the 0/1 loss is $\ell_{0/1}(z) = 1$ if $z < 0$ and $\ell_{0/1}(z) = 0$ otherwise, and $C > 0$ is a penalization parameter. We can then turn the 0/1 loss to Hinge Loss, that is why Hinge Loss comes in:

$$\min_{\omega \in \mathbb{R}^n, \beta \in \mathbb{R}} \frac{1}{2}|\omega|^2 + C \sum_{j=1}^{m} \max(0, 1 - y_j(\omega^T x_j + \beta)) \ . \tag{1.14}$$

We can introduce "slack variables" $\xi_i \geq 0$ to introduce weights to classification errors in the above problem. This leads to "Soft margin SVM with slack variables":

$$\min_{\omega \in \mathbb{R}^n, \beta \in \mathbb{R}} \frac{1}{2}|\omega|^2 + C \sum_{j=1}^{m} \xi_j \text{ s.t. } y_j(\omega^T x_j + b) \geq 1 - \xi_j \ , \xi_j \geq 0 \ , j = 1, 2, ..., m \ . \tag{1.15}$$

See [27].

(3) Neural Network: "activation function" σ.

Sigmoid:

$$\sigma(z) = \frac{1}{1 + \exp(-z)} \ ; \tag{1.16}$$

tanh:

$$\sigma(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)} \ ; \tag{1.17}$$

ReLU (Rectified Linear Unit):

$$\sigma(z) = \max(0, z) \ . \tag{1.18}$$

Vector–valued activation: if $z \in \mathbb{R}^n$ then $\sigma : \mathbb{R}^n \to \mathbb{R}^n$ is defined by $(\sigma(z))_i = \sigma(z_i)$ where each $\sigma(z_i)$ is the scalar activation function.

Fully connected neural network prediction function

$$g(x, \omega) = a^T \left(\sigma \left(W^{(H)} (\sigma(W^{(H-1)}(...(\sigma(W^{(1)}x + b_1))...) + b_{H-1}) + b_H \right) \right) \ . \tag{1.19}$$

Optimization

$$\min_{\omega} \frac{1}{2} \sum_{i=1}^{n} (g(x_i, \omega) - y_i)^2 \ .$$

Many other different network structures that we do not expand here: convolutional, recurrent (Gate: GRU, LSTM), ResNet, Transformer, ...

Two layer (one hidden layer) fully connected ReLU neural network has specific loss function structure: our $W^{(1)} = \begin{pmatrix} \omega_1^T \\ ... \\ \omega_m^T \end{pmatrix}$ and

$$g(x, \omega) = \sum_{r=1}^{m} a_r \max\left(\omega_r^T x, 0\right) \ . \tag{1.20}$$

Optimization problem is given by

$$\min_{\omega} \frac{1}{2} \sum_{i=1}^{n} \left(\sum_{r=1}^{m} a_r \max\left(\omega_r^T x_i, 0\right) - y_i \right)^2 \ .$$

Non–convexity issues: see [5].

1.2 Matrix Optimizations

Many machine learning/statistical learning problems are related to matrix optimizations.

(1) Matrix Completion: Each A_j is $n \times p$ matrix, and we seek for another $n \times p$ matrix $\widehat{X}$ such that

$$\widehat{X} = \arg\min_X \frac{1}{2m} \sum_{j=1}^{m} (\langle A_j, X \rangle - y_j)^2 \tag{1.21}$$

where $\langle A, B \rangle = \mathrm{tr}(A^T B)$. We can think of the A_j as "probing" the unknown matrix X. In other words, we want the best X such that $\langle A_j, X \rangle \approx y_j$ for all $1 \leq j \leq m$.

(2) Nonnegative Matrix Factorization: If the full matrix $Y \in \mathbb{R}^{n \times p}$ is observed, then we seek for $L \in \mathbb{R}^{n \times r}$ and $R \in \mathbb{R}^{p \times r}$ such that

$$\min_{L,R} \|LR^T - Y\|_F^2 \text{ subject to } L \geq 0 \text{ and } R \geq 0 \ . \tag{1.22}$$

Here $\|A\|_F = \left(\sum \sum |a_{ij}|^2\right)^{1/2}$ is the Frobenius norm of a matrix A. This is used very often in recommendation systems (see [1]).

See [7] for an overview.

(3) Principle Component Analysis (PCA): Let S be a positive-definite (non-negative definite) matrix of size $n \times n$. Then we can diagonalize it as $Se_i = \lambda_i e_i$, $1 \leq i \leq n$, $\lambda_1 \geq ... \geq \lambda_n > 0$ (or ≥ 0). Let $v \in \mathbb{R}^n$ be written as $v = v_1 e_1 + ... + v_n e_n$. Then

$$v^T S v = \lambda_1 v_1^2 + ... + \lambda_n v_n^2$$

is a quadratic form. If we restrict $v_1^2 + ... + v_n^2 = 1$, then the maximum of above quadratic form will give the direction of e_1 and value λ_1 (principle component).

PCA:

$$\max_{v \in \mathbb{R}^n} v^T S v \text{ such that } \|v\|_2 = 1 \ , \ \|v\|_0 \leq k \ . \tag{1.23}$$

Here S is a positive-definite (or non-negative definite) matrix. The objective function is convex, but if you take into account the constraint, then this problem becomes non–convex. A picture for dimension 1 example can be shown below.

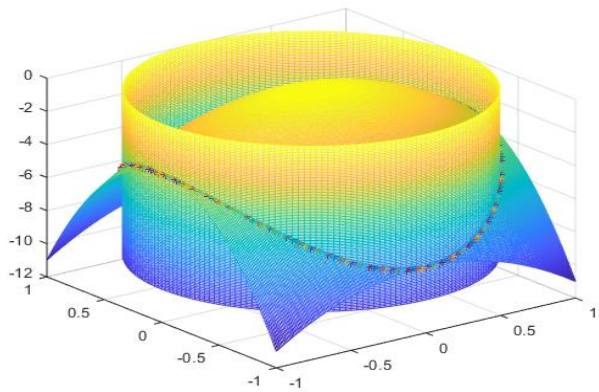

Figure 1.1: Loss Landscape of 1-dimensional PCA.

Online PCA: see [16].

(4) Sparse inverse covariance matrix estimation: Sample covariance matrix $S = \frac{1}{m-1}\sum_{j=1}^{m} a_j a_j^T$. $S^{-1} = X$. "Graphical LASSO":

$$\min_{X \in \text{ Symmetric } \mathbb{R}^{n\times n}, X \succeq 0} \langle S, X\rangle - \ln\det X + \lambda \|X\|_1 \tag{1.24}$$

where $\|X\|_1 = \sum |X_{ij}|$.

1.3 Experiment: Activation Functions

In this experiment we plot the following activation functions as well as their first and second derivatives. (1) Sigmoid; (2) ReLU; (3) Tanh; (4) Exponential. See [2], `src/activations`.

Chapter 2

Convex Functions

2.1 Optimization problem: set-up and its solutions

For the objective function $f : \mathbb{R}^n \supset \mathcal{D} \to \mathbb{R}$, we have various notions of minimizers:

- Local minimizer: $x^* \in \mathcal{D}$ is a local minimizer if there exist a neighborhood $\mathcal{N}$ containing x^*, such that $f(x) \geq f(x^*)$ for all $x \in \mathcal{N} \cap \mathcal{D}$;
- Global minimizer: $x^* \in \mathcal{D}$ is a global minimizer if $f(x) \geq f(x^*)$ for all $x \in \mathcal{D}$;
- Strict Local Minimizer: $x^* \in \mathcal{D}$ is a strict local minimizer iff x^* is a local minimizer and $f(x) > f(x^*)$ if $x \neq x^*$, $x \in \mathcal{N}$;
- Isolated Local Minimizer: $x^* \in \mathcal{D}$ is an isolated local minimizer iff there exists a neighborhood $\mathcal{N}$ containing x^* such that $f(x) \geq f(x^*)$ for all $x \in \mathcal{N} \cap \mathcal{D}$ and $\mathcal{N}$ does not contain any other local minimizers.

Constrained optimization problem

$$\min_{x \in \Omega} f(x) \ , \tag{2.1}$$

where $\Omega \subset \mathcal{D} \subset \mathbb{R}^n$ is a closed set.

x^* is a *local solution*: there exist a neighborhood $\mathcal{N}$ containing x^* such that $f(x) \geq f(x^*)$ for any $x \in \mathcal{N} \cap \Omega$.

x^* is a *global solution*: $f(x) \geq f(x^*)$ for any $x \in \Omega$.

2.2 Convexity

Definition 2.1 (Convex Set)**.** *A set Ω is called a convex set if for any $x, y \in \Omega$ we have*

$$(1 - \alpha)x + \alpha y \in \Omega$$

for all $\alpha \in [0, 1]$.

Given convex set $\Omega \subset \mathbb{R}^n$, the *projection operator* $P : \mathbb{R}^n \to \Omega$ is given by

$$P(y) = \arg\min_{z \in \Omega} \|z - y\|_2^2 \ . \tag{2.2}$$

$P(y)$ is the point in Ω that is closest to y in the sense of Euclidean norm. If $x \in \Omega$, then $P(x) = x$.

Definition 2.2 (Convex Function)**.** *A function $\phi : \mathbb{R}^n \to \mathbb{R} \cup \{\pm\infty\}$ is convex if for all $x, y \in \mathbb{R}^n$ we have*

$$\phi((1-\alpha)x + \alpha y) \leq (1-\alpha)\phi(x) + \alpha\phi(y) \tag{2.3}$$

for all $\alpha \in [0, 1]$.

Definition 2.3 (Strongly Convex Function)**.** *A convex function ϕ is called strongly convex with modulus of convexity $m > 0$ if*

$$\phi((1-\alpha)x + \alpha y) \leq (1-\alpha)\phi(x) + \alpha\phi(y) - \frac{1}{2}m\alpha(1-\alpha)\|x - y\|_2^2 \tag{2.4}$$

for all x, y in the domain of ϕ.

Let $\Omega \subset \mathbb{R}^n$ be convex. We define the indicator function

$$I_\Omega(x) = \begin{cases} 0 & \text{if } x \in \Omega \ , \\ +\infty & \text{otherwise} \ . \end{cases} \tag{2.5}$$

Constrained optimization problem $\min_{x \in \Omega} f(x)$ is the same thing as the unconstrained optimization problem $\min[f(x) + I_\Omega(x)]$. We can set a sequence of functions $F_{\lambda\Omega} \uparrow I_\Omega(x)$ as $\lambda \uparrow \infty$, and then we solve the relaxed problem $\min_{x \in \mathbb{R}^d}[f(x) + F_{\lambda\Omega}(x)]$.

Theorem 2.1 (Minimizers for convex functions)**.** *If the function f is convex and the set Ω is closed and convex, then*

(a) Any local solution of (2.1) is also global;

(b) The set of global solutions of (2.1) is a convex set.

Proof. (a) Suppose x_1^* is a local optimizer that is not global. This means there exists some $x_2^* \neq x_1^*$ such that $f(x_2^*) < f(x_1^*)$. Then by convexity for any $\alpha \in [0, 1]$ we have

$$f((1-\alpha)x_1^* + \alpha x_2^*) \leq (1-\alpha)f(x_1^*) + \alpha f(x_2^*) < f(x_1^*) \ .$$

If α is close to 0 and the above inequality will violate the fact that x_1^* is a local minimizer.

(b) Let S be the set of global minimizers. Then for any $x_1, x_2 \in S$ we have $f(x_1) = f(x_2) = \min_{x \in \Omega} f(x)$. By convexity we have

$$f(\alpha x_1 + (1-\alpha)x_2) \leq \alpha f(x_1) + (1-\alpha)f(x_2) = \min_{x \in \Omega} f(x) \ ,$$

which means that S is convex. □

A few more auxiliary notions:

- "Effective domain" of ϕ: $\{x \in \Omega : \phi(x) < \infty\}$;
- "Epigraph" of ϕ:
$$\mathrm{epi}(\phi) = \{(x,t) \in \Omega \times \mathbb{R} : t \geq \phi(x)\} \ .$$
- ϕ is a "proper convex function" if $\phi(x) < \infty$ for some $x \in \Omega$ and $\phi(x) > -\infty$ for all $x \in \Omega$;
- ϕ is a "closed proper convex function" if ϕ is a proper convex function and $\{x \in \Omega : \phi(x) \leq t\}$ is a closed set for all $t \in \mathbb{R}$.

2.3 Taylor's theorem and convexity in Taylor's expansion

Theorem 2.2 (Taylor's theorem)**.** *Given a continuously differentiable function $f : \mathbb{R}^n \to \mathbb{R}$ and given $x, p \in \mathbb{R}^n$ we have*

$$f(x+p) = f(x) + \int_0^1 \nabla f(x+\gamma p)^T p d\gamma \ , \tag{2.6}$$

$$f(x+p) = f(x) + \nabla f(x+\gamma p)^T p \ , \quad \textit{for some } \gamma \in (0,1) \ . \tag{2.7}$$

If f is twice continuously differentiable, then

$$\nabla f(x+p) = \nabla f(x) + \int_0^1 \nabla^2 f(x+\gamma p)^T p d\gamma \ , \tag{2.8}$$

$$f(x+p) = f(x) + \nabla f(x)^T p + \frac{1}{2} p^T \nabla^2 f(x+\gamma p) p \ , \quad \textit{for some } \gamma \in (0,1) \ . \tag{2.9}$$

Proof. (1) Let $g(\gamma) = f(x+\gamma p)$, then $g'(\gamma) = \nabla f(x+\gamma p) \cdot p = (\nabla f(x+\gamma p))^T p$. Then by Newton-Leibniz we have

$$g(1) - g(0) = \int_0^1 g'(\gamma) d\gamma = \int_0^1 (\nabla f(x+\gamma p))^T p d\gamma \ ,$$

which gives (2.6).

(2) Using mean-value theorem $\int_a^b h(t)dt = (b-a)h(\xi)$ for some $\xi \in (a,b)$, we get

$$\int_0^1 (\nabla f(x+\gamma p))^T p d\gamma = (\nabla f(x+\gamma_0 p))^T p$$

for some $\gamma_0 \in (0,1)$, which is (2.7).

Another way is to use the fact that

$$f(y) = f(x) + (\nabla f(x + \gamma_0(y-x)))^T (y-x) \ .$$

Set $p = y - x$, this gives

$$f(x+p) = f(x) + (\nabla f(x+\gamma_0 p))^T p \ .$$

(3) We apply (2.8) to each of $\dfrac{\partial f}{\partial x_i}$, and we get

$$\frac{\partial f}{\partial x_i}(x+p) = \frac{\partial f}{\partial x_i}(x) + \int_0^1 \left[\nabla\left(\frac{\partial f}{\partial x_i}\right)(x+\gamma p)\right]^T p d\gamma \ .$$

Introduce the Hessian matrix $\nabla^2 f(x) = \left(\dfrac{\partial^2 f}{\partial x_i \partial x_j}\right)_{1\le i,j\le n}$. Then when we put the above equation for all $1 \le i \le n$, we get (2.8).

(4) Define an auxiliary function of a single variable $\phi(t) = f(x+tp)$ for $t \in [0,1]$. By applying the univariate Taylor's Theorem with the Lagrange form of the remainder to $\phi(t)$ at $t = 0$, we have:

$$\phi(1) = \phi(0) + \phi'(0)(1-0) + \frac{1}{2}\phi''(\gamma)(1-0)^2$$

for some $\gamma \in (0,1)$.

Now, we compute the derivatives of $\phi(t)$ using the multi-variable chain rule:

$$\phi'(t) = \frac{d}{dt} f(x+tp) = \nabla f(x+tp)^T p$$

At $t = 0$, we obtain $\phi'(0) = \nabla f(x)^T p$.

$$\phi''(t) = \frac{d}{dt}\left(\nabla f(x+tp)^T p\right) = p^T \nabla^2 f(x+tp) p$$

where $\nabla^2 f$ is the Hessian matrix of f.

Substituting these expressions back into the Taylor expansion of $\phi(1)$: Since $\phi(1) = f(x+p)$ and $\phi(0) = f(x)$, we get (2.9). □

Definition 2.4 (L–Lipschitz)**.** *If $f : \mathbb{R}^n \to \mathbb{R}$ is such that*

$$|f(x) - f(y)| \le L\|x-y\| \ , \tag{2.10}$$

for $L > 0$ and any $x, y \in \mathbb{R}^n$, then $f(x)$ is L–Lipschitz.

Optimization literatures often assume that ∇f is L–Lipschitz

$$\|\nabla f(x) - \nabla f(y)\| \le L\|x-y\| \ . \tag{2.11}$$

Theorem 2.3. *(1) If f is continuously differentiable and convex then*

$$f(y) \ge f(x) + (\nabla f(x))^T (y-x) \tag{2.12}$$

for any $x, y \in dom(f)$.

(2) If f is differentiable and m–strongly convex then

$$f(y) \ge f(x) + (\nabla f(x))^T (y-x) + \frac{m}{2}\|y-x\|^2 \tag{2.13}$$

for any $x, y \in dom(f)$.

(3) If ∇f is uniformly Lipschitz continuous with Lipschitz constant $L > 0$ and f is convex then

$$f(y) \leq f(x) + (\nabla f(x))^T(y-x) + \frac{L}{2}\|y-x\|^2 \tag{2.14}$$

for any $x, y \in dom(f)$.

Proof. (1) Let $z_\alpha = \alpha x + (1-\alpha)y$. Then by convexity

$$f(z_\alpha) \leq \alpha f(x) + (1-\alpha)f(y) \ ,$$

which gives

$$f(z_\alpha) - f(x) \leq (1-\alpha)(f(y) - f(x)) \ .$$

We can use (2.7) to get

$$f(z_\alpha) - f(x) = (\nabla f(x + \gamma(z_\alpha - x)))^T(z_\alpha - x)$$

for some $\gamma \in [0,1]$. Since $z_\alpha - x = (1-\alpha)(y-x)$, this gives

$$f(y) - f(x) \geq (\nabla f(x + \gamma(z_\alpha - x)))^T(y-x) \ .$$

Letting $\alpha \to 1$ we have $z_\alpha \to x$, that will give (2.12).

(2) We extend the above argument using strong convexity (2.4). This means we have

$$f(z_\alpha) \leq \alpha f(x) + (1-\alpha)f(y) - \frac{1}{2}m\alpha(1-\alpha)\|x-y\|_2^2 \ ,$$

which gives

$$f(z_\alpha) - f(x) + \frac{m}{2}\alpha(1-\alpha)\|x-y\|^2 \leq (1-\alpha)(f(y) - f(x)) \ .$$

By (2.7) again we have

$$(1-\alpha)(\nabla f(x + \gamma(z_\alpha - x)))^T(y-x) + \frac{m}{2}\alpha(1-\alpha)\|y-x\|^2 \leq (1-\alpha)(f(y) - f(x)) \ .$$

Dividing both sides by $1-\alpha$ and setting $\alpha \to 1$ we get (2.13).

(3) We app;y (2.6) to get

$$f(y) = f(x) + \int_0^1 (\nabla f(x + \gamma(y-x)))^T(y-x)d\gamma \ .$$

This gives us

$$\begin{aligned} & f(y) - f(x) - (\nabla f(x))^T(y-x) \\ = & \int_0^1 [(\nabla f(x + \gamma(y-x)))^T - (\nabla f(x))^T](y-x)d\gamma \\ \leq & \int_0^1 \|(\nabla f(x + \gamma(y-x)))^T - (\nabla f(x))^T\| \cdot \|y-x\|d\gamma \\ \leq & \int_0^1 L\gamma\|y-x\|^2 d\gamma = \frac{L}{2}\|y-x\|^2 \ , \end{aligned}$$

which is (2.14). □

We use the symbol $A \succeq B$ ($A \preceq B$) to mean $\lambda_A \geq \lambda_B$ ($\lambda_A \leq \lambda_B$) for every corresponding pair of eigenvalues of A, B.

Theorem 2.4. *Suppose that the function f is twice continuously differentiable on $\mathbb{R}^n$. Then*

(1) f is strongly convex with modulus of convexity m if and only if $\nabla^2 f(x) \succeq mI$ for all x;

(2) ∇f is Lipschitz continuous with Lipschitz constant L if and only if $\nabla^2 f(x) \preceq LI$ for all x.

Proof. (1) Suppose that the function f is strongly–m convex. Set $u \in \mathbb{R}^n$ and $\alpha > 0$, then we consider the Taylor's expansion

$$f(x + \alpha u) = f(x) + \alpha \nabla f(x)^T u + \frac{1}{2}\alpha^2 u^T \nabla^2 f(x + t\alpha u) u \tag{2.15}$$

for some $0 \leq t \leq 1$. We apply (2.13) with $y = x + \alpha u$ so that

$$f(x + \alpha u) \geq f(x) + \alpha (\nabla f(x))^T u + \frac{m}{2}\alpha^2 \|u\|^2 \ . \tag{2.16}$$

Comparing (2.15) and (2.16) we see that for arbitrary choice of $u \in \mathbb{R}^n$ we have

$$u^T \nabla^2 f(x + t\alpha u) u \geq m\|u\|^2 \ .$$

This implies that $\nabla^2 f(x) \succeq mI$ as claimed. This shows the "only if" part.

Indeed one can also show the "if" part. To this end assume that $\nabla^2 f(x) \succeq mI$. Then for any $z \in \mathbb{R}^n$ we have that $(x - z)^T \nabla^2 f(z + t(x - z))(x - z) \geq m\|x - z\|^2$. Thus

$$\begin{aligned} f(x) &= f(z) + (\nabla f(z))^T (x - z) + \frac{1}{2}(x - z)^T \nabla^2 f(z + t(x - z))(x - z) \\ &\geq f(z) + (\nabla f(z))^T (x - z) + \frac{m}{2}\|x - z\|^2 \ . \end{aligned} \tag{2.17}$$

Similarly

$$\begin{aligned} f(y) &= f(z) + (\nabla f(z))^T (y - z) + \frac{1}{2}(y - z)^T \nabla^2 f(z + t(y - z))(y - z) \\ &\geq f(z) + (\nabla f(z))^T (y - z) + \frac{m}{2}\|y - z\|^2 \ . \end{aligned} \tag{2.18}$$

We consider $(1 - \alpha)$(2.17)+α(2.18) and we set $z = (1 - \alpha)x + \alpha y$. This gives

$$\begin{aligned} &(1 - \alpha) f(x) + \alpha f(y) \\ &\geq (\alpha + (1 - \alpha)) f(z) + (\nabla f(z))^T ((1 - \alpha)(x - z) + \alpha(y - z)) + \frac{m}{2}\left((1 - \alpha)\|x - z\|^2 + \alpha\|y - z\|^2\right) \\ &= f(z) + (\nabla f(z))^T ((1 - \alpha)(x - z) + \alpha(y - z)) + \frac{m}{2}\left((1 - \alpha)\|x - z\|^2 + \alpha\|y - z\|^2\right) \ . \end{aligned} \tag{2.19}$$

Since $x - z = \alpha(x - y)$ and $y - z = (1 - \alpha)(y - x)$, we see that $((1 - \alpha)(x - z) + \alpha(y - z)) = 0$. Moreover, this means that

$$(1 - \alpha)\|x - z\|^2 + \alpha\|y - z\|^2 = \left[(1 - \alpha)\alpha^2 + \alpha(1 - \alpha)^2\right]\|x - y\|^2 = \alpha(1 - \alpha)\|x - y\|^2 \ .$$

From these we see that (2.19) is the same as saying

$$(1 - \alpha) f(x) + \alpha f(y) \geq f((1 - \alpha)x + \alpha y) + \frac{m}{2}\alpha(1 - \alpha)\|x - y\|^2 \ ,$$

which is (2.4).

(2) We want to show $\|\nabla f(y) - \nabla f(x)\| \leq L\|y - x\|$ is equivalent to $\nabla^2 f(x) \preceq LI$. Assume $\nabla^2 f(x) \preceq LI$, which means the spectral norm $\|\nabla^2 f(x)\|_2 \leq L$. Using (2.8) we get

$$\begin{aligned}\|\nabla f(y) - \nabla f(x)\| &= \left\| \int_0^1 \nabla^2 f(x + t(y - x))(y - x)dt \right\| \\ &\leq \int_0^1 \left\|\nabla^2 f(x + t(y - x))\right\| \cdot \|y - x\| \, dt \\ &\leq \int_0^1 L\|y - x\|dt = L\|y - x\| \ ,\end{aligned}$$

which proves ∇f is L-Lipschitz continuous. Assume ∇f is L-Lipschitz. For any vector v, consider the limit:

$$\nabla^2 f(x)v = \lim_{t\to 0} \frac{\nabla f(x + tv) - \nabla f(x)}{t}$$

Taking the norm:

$$\|\nabla^2 f(x)v\| = \lim_{t\to 0} \frac{\|\nabla f(x + tv) - \nabla f(x)\|}{t} \leq \lim_{t\to 0} \frac{L\|tv\|}{t} = L\|v\|$$

This implies the operator norm $\|\nabla^2 f(x)\|_2 \leq L$, which for a symmetric Hessian matrix is equivalent to $-LI \preceq \nabla^2 f(x) \preceq LI$. Since we usually deal with convex/semi-convex functions in this context, it implies $\nabla^2 f(x) \preceq LI$. □

Theorem 2.5 (Strongly convex functions have unique minimizers)**.** *Let f be differentiable and strongly convex with modulus $m > 0$. Then the minimizer x^* of f exists and is unique.*

Proof. Uniqueness. Suppose there exist two distinct minimizers x^* and z^*. Since f is differentiable, we must have $\nabla f(x^*) = 0$ and $\nabla f(z^*) = 0$. Using the strong convexity property (2.13) at x^*:

$$f(z^*) \geq f(x^*) + \nabla f(x^*)^T(z^* - x^*) + \frac{m}{2}\|z^* - x^*\|^2 = f(x^*) + \frac{m}{2}\|z^* - x^*\|^2 \ .$$

Similarly, using strong convexity at z^*:

$$f(x^*) \geq f(z^*) + \frac{m}{2}\|x^* - z^*\|^2 \ .$$

Adding these two inequalities yields $f(x^*) + f(z^*) \geq f(x^*) + f(z^*) + m\|x^* - z^*\|^2$, which simplifies to $0 \geq m\|x^* - z^*\|^2$. Since $m > 0$, it must be that $\|x^* - z^*\| = 0$, hence $x^* = z^*$.

Existence. From the strong convexity definition, $f(y) \to \infty$ as $\|y\| \to \infty$ (coercivity). Since f is continuous and coercive, it must attain a minimum on any closed set in $\mathbb{R}^n$. □

Below are a few more technical issues regarding convex and strongly convex functions.

Theorem 2.6. *Let f be convex and continuously differentiable and ∇f be L–Lipschitz. Then for any $x, y \in dom(f)$ the following bounds hold:*

$$f(x) + \nabla f(x)^T(y - x) + \frac{1}{2L}\|\nabla f(x) - \nabla f(y)\|^2 \leq f(y) \ , \tag{2.20}$$

$$\frac{1}{L}\|\nabla f(x) - \nabla f(y)\|^2 \leq (\nabla f(x) - \nabla f(y))^T(x-y) \leq L\|x-y\|^2 \ . \tag{2.21}$$

In addition, let f be strongly convex with modulus m and unique minimizer x^*. Then for any $x, y \in dom(f)$ we have that

$$f(y) - f(x) \geq -\frac{1}{2m}\|\nabla f(x)\|^2 \ . \tag{2.22}$$

Proof. Define an auxiliary function $g(z) = f(z) - \nabla f(x)^T z$. Since f is convex, g is also convex. Furthermore, $\nabla g(z) = \nabla f(z) - \nabla f(x)$, which is also L-Lipschitz continuous. Note that $\nabla g(x) = 0$, implying x is a global minimizer of $g(z)$ via Theorem 2.1. Using (2.14) we get

$$g(z) \leq g(y) + \nabla g(y)^T(z-y) + \frac{L}{2}\|z-y\|^2 \ .$$

Setting $z = y - \frac{1}{L}\nabla g(y)$, and since x minimizes g, we obtain:

$$g(x) \leq g(y) - \frac{1}{2L}\|\nabla g(y)\|^2 \ .$$

Substituting $g(z) = f(z) - \nabla f(x)^T z$ back into the inequality:

$$f(x) - \nabla f(x)^T x \leq f(y) - \nabla f(x)^T y - \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \ ,$$

which is (2.20).

We swap x and y in (2.20) to get

$$f(y) + \nabla f(y)^T(x-y) + \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \leq f(x) \ .$$

Adding this to (2.20) and canceling $f(x) + f(y)$ from both sides we get

$$\nabla f(x)^T(y-x) + \nabla f(y)^T(x-y) + \frac{1}{L}\|\nabla f(x) - \nabla f(y)\|^2 \leq 0 \ ,$$

which gives the first inequality in (2.21) by rearranging.

The second inequality in (2.21) follows directly from Cauchy-Schwarz and the L-Lipschitz property:

$$(\nabla f(x) - \nabla f(y))^T(x-y) \leq \|\nabla f(x) - \nabla f(y)\| \cdot \|x-y\| \leq L\|x-y\|^2 \ .$$

Assume f is strongly convex with modulus m. Then we have

$$\begin{aligned}
f(y) &\geq f(x) + \nabla f(x)^T(y-x) + \frac{m}{2}\|y-x\|^2 \\
&= f(x) + \frac{m}{2}\left[\|y-x\|^2 + \frac{2}{m}\nabla f(x)^T(y-x)\right] \\
&= f(x) + \frac{m}{2}\left[\|y-x\|^2 + 2\left(\frac{1}{m}\nabla f(x)\right)^T(y-x) + \frac{1}{m^2}\|\nabla f(x)\|^2\right] - \frac{1}{2m}\|\nabla f(x)\|^2 \\
&= f(x) + \frac{m}{2}\left\|y - x + \frac{1}{m}\nabla f(x)\right\|^2 - \frac{1}{2m}\|\nabla f(x)\|^2 \ ,
\end{aligned}$$

which gives (2.22). □

Definition 2.5 (Generalized Strong Convexity)**.** *We say the convex function $f(x)$ has* Generalized Strong Convexity *if*

$$\|\nabla f(x)\|^2 \geq 2m[f(x) - f^*] \text{ for some } m > 0 \ , \tag{2.23}$$

where $f^* = f(x^*)$ is the minimum of f.

The Generalized Strong Convexity holds in situations other than when f is strongly convex.

2.4 Optimality Conditions

We derive optimality conditions for the smooth unconstrained problem

$$\min_{x \in \mathbb{R}^n} f(x) \ , \tag{2.24}$$

where $f(x)$ is a smooth function.

Theorem 2.7 (Necessary conditions for smooth unconstrained optimization)**.** *We have*

(a) (first–order necessary condition) Suppose that f is continuously differentiable. Then if x^* is a local minimizer of (2.24), then $\nabla f(x^*) = 0$;

(b) (second–order necessary condition) Suppose that f is twice continuously differentiable. Then if x^* is a local minimizer of (2.24), then $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive semi-definite.

Proof. Suppose x^* is a local minimizer of f. Then for any direction $d \in \mathbb{R}^n$ and sufficiently small step size $\alpha > 0$, we have $f(x^* + \alpha d) \geq f(x^*)$.

(a) By (2.9) we have

$$f(x^* + \alpha d) = f(x^*) + \alpha \nabla f(x^*)^T d + O(\alpha^2)$$

Since $f(x^* + \alpha d) \geq f(x^*)$, it follows that $\alpha \nabla f(x^*)^T d + O(\alpha^2) \geq 0$. Dividing by α and letting $\alpha \to 0$, we obtain $\nabla f(x^*)^T d \geq 0$. Since this holds for any d, we choose $d = -\nabla f(x^*)$, which gives $-\|\nabla f(x^*)\|^2 \geq 0$. Thus, $\nabla f(x^*) = 0$.

(b) Given $\nabla f(x^*) = 0$, by (2.9) we get

$$f(x^* + \alpha d) = f(x^*) + \frac{\alpha^2}{2} d^T \nabla^2 f(x^*) d + o(\alpha^2)$$

For $f(x^* + \alpha d) \geq f(x^*)$ to hold for small α, we must have $d^T \nabla^2 f(x^*) d \geq 0$ for all d. This implies that the Hessian matrix $\nabla^2 f(x^*)$ is positive semi-definite. □

Theorem 2.8 (Sufficient conditions for convex functions in smooth unconstrained optimization)**.** *If f is continuously differentiable and convex, then if $\nabla f(x^*) = 0$, then x^* is a global minimizer of (2.24). In addition, if f is strongly convex, then x^* is the unique global minimizer.*

Proof. Assume f is convex and $\nabla f(x^*) = 0$. By (2.12), for any $x \in \mathbb{R}^n$ we have

$$f(x) \geq f(x^*) + \nabla f(x^*)^T (x - x^*) \ .$$

Since $\nabla f(x^*) = 0$, this gives $f(x) \geq f(x^*)$. This proves that x^* is a global minimizer. If f is further assumed to be strongly convex, then x^* is the unique global minimizer due to Theorem 2.5. □

Theorem 2.9 (Sufficient conditions for nonconvex functions in smooth unconstrained optimization)**.** *Suppose that f is twice continuously differentiable and that for some x^* we have $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite, then x^* is a strict local minimizer of* (2.24)*.*

Proof. Suppose $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite. Let $\lambda_{\min} > 0$ be the smallest eigenvalue of $\nabla^2 f(x^*)$. Then for any $d \neq 0$:

$$d^T \nabla^2 f(x^*) d \geq \lambda_{\min} \|d\|^2 \ .$$

Using (2.9), the second-order Taylor expansion around x^*, we get

$$f(x^* + d) = f(x^*) + \nabla f(x^*)^T d + \frac{1}{2} d^T \nabla^2 f(x^*) d + o(\|d\|^2) \ .$$

Since $\nabla f(x^*) = 0$, we have:

$$f(x^* + d) \geq f(x^*) + \frac{\lambda_{\min}}{2} \|d\|^2 + o(\|d\|^2)$$

For sufficiently small $\|d\|$, the term $\frac{\lambda_{\min}}{2} \|d\|^2$ dominates the higher-order terms $o(\|d\|^2)$, ensuring $f(x^* + d) > f(x^*)$. Thus, x^* is a strict local minimizer. □

2.5 Experiment: Nonconvexity of the Loss Landscape of Neural Networks

Consider a neural network with one hidden layer that consists of p neurons and input $x \in \mathbb{R}^1$, output $y \in \mathbb{R}^1$. The neural network function has the form $y(x) = \sum_{j=1}^{p} c_j \sigma(a_j x - b_j)$, where $\mathbf{a} = a_j, \mathbf{b} = b_j$ and $\mathbf{c} = c_j$ are the neural network weights. Assume $(a_j, b_j, c_j) \sim \mathcal{N}(0, I_3)$, $j = 1, 2, \ldots, p$ is a family of i.i.d multivariate normal distributions. For different realizations of (a_j, b_j, c_j), we plot the function $y(x)$ on x-y graph. We have experimented different hidden layer sizes.

With the above done, assume that the training data (x, y) follows a bivariate normal distribution $\mathcal{N}(0, I_2)$. Let the variables a_1 and a_2 vary in a certain interval. Let all other neural network weights a_j, b_j, c_j follow the same assumption as above. For different (a_1, a_2), we plot the empirical loss function of this network as a function of (a_1, a_2). We can see that the loss function may not be convex with respect to (a_1, a_2). We show experiment results for different hidden layer sizes.

We refer to [2], `src/one_hidden_layer_nn`.

A typical non-convex loss landscape looks as follows:

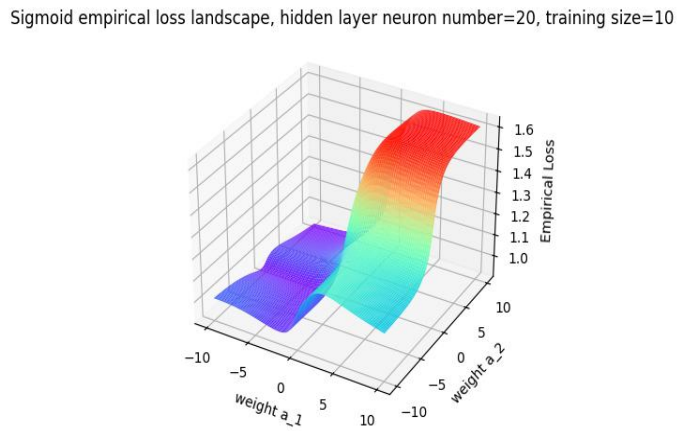

Figure 2.1: Loss Landscape of a neural network.

In the branch `support_gpu` (already merged to `main`), we supported GPU computations, with some optimizations made in the computing flow. This is especially useful when the hidden layer size is huge, where we expect to see the NTK regime (see [4]), and the loss landscape tends to be convex.

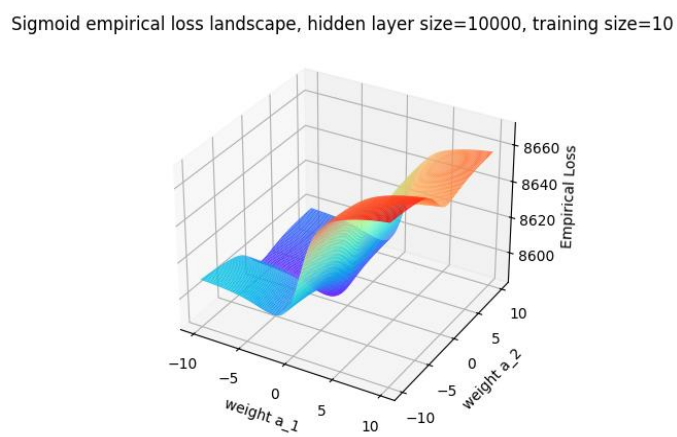

Figure 2.2: Loss Landscape of a neural network, 10000 neurons in the hidden layer.

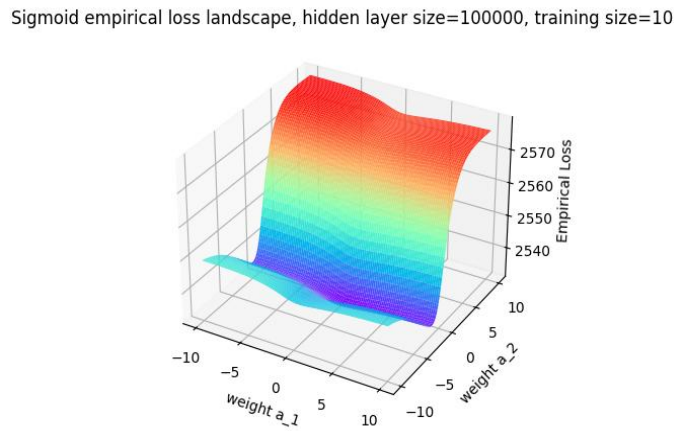

Figure 2.3: Loss Landscape of a neural network, 100000 neurons in the hidden layer.

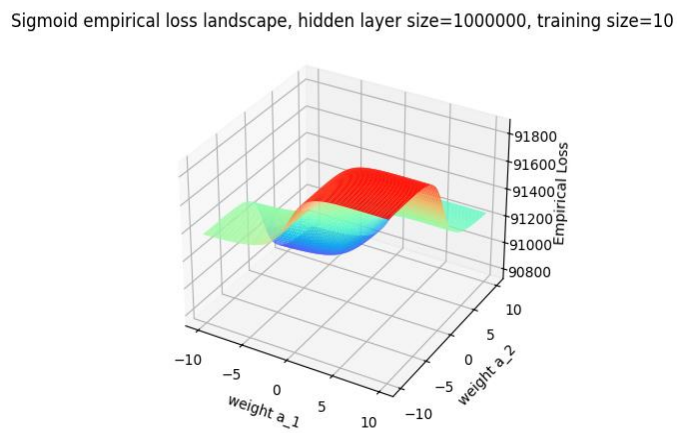

Figure 2.4: Loss Landscape of a neural network, 1000000 neurons in the hidden layer.

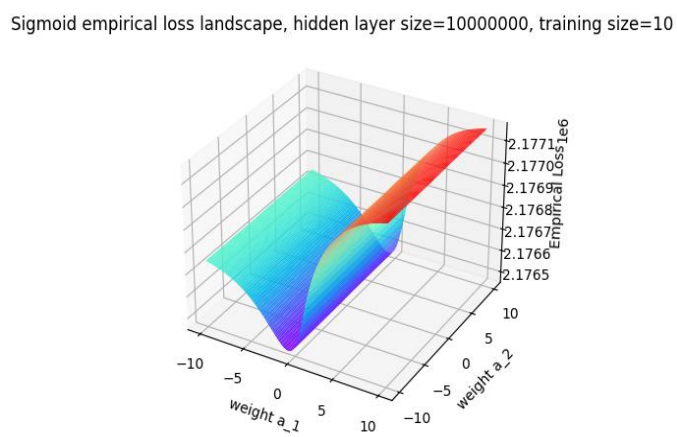

Figure 2.5: Loss Landscape of a neural network, 10000000 neurons in the hidden layer.

Chapter 3

Gradient Descent and Line Search Methods

3.1 Gradient Descent

We want to solve the optimization problem

$$\min_{x \in \mathbb{R}^n} f(x) \ .$$

The goal is to construct an approximating sequence $\{x^k\}$ such that $f(x^{k+1}) < f(x^k)$, $k = 0, 1, 2, \ldots$.

"descent direction" $d \in \mathbb{R}^n$ such that

$$f(x + td) < f(x) \text{ for all } t > 0 \text{ sufficiently small.}$$

Say f is continuously differentiable

$$f(x + td) = f(x) + t\nabla f(x + \gamma td)^T d \text{ for some } \gamma \in (0, 1) \ .$$

Definition 3.1. *The vector $d \in \mathbb{R}^n$ is a* descent direction *if*

$$d^T \nabla f(x) < 0 \ .$$

It is a steepest descent direction *if*

$$\inf_{\|d\|=1} d^T \nabla f(x) = -\|\nabla f(x)\| \ .$$

The inf is achieved when $d = -\dfrac{\nabla f(x)}{\|\nabla f(x)\|}$. This gives the Gradient Descent Method:

Algorithm 3.1 Gradient Descent

1: **Input**: Input initialization x^0, stepsize $\alpha_k > 0$, $k = 0, 1, 2, ...$
2: **for** $k = 0, 1, 2, ..., K-1$ **do**
3: Calculate $\nabla f(x^k)$
4: $x^{k+1} = x^k - \alpha_k \nabla f(x^k)$
5: **end for**
6: **Output**: Output x^K

Definition 3.2 (Convergence of optimization algorithms)**.** *We consider any of the following estimates as characterizing the convergence of optimization algorithms:*

$$\|x_T - x^*\| \leq \varepsilon(T) \tag{3.1}$$

or

$$\mathbf{E}\|x_T - x^*\|^2 \leq \varepsilon(T) \tag{3.2}$$

or

$$\mathbf{E}f(x_T) - f(x^*) \leq \varepsilon(T) \ . \tag{3.3}$$

Here $\varepsilon(T) \to 0$ as $T \to \infty$. The convergence rates *are measured by $\ln \varepsilon(T)$. We have the following cases:*

(1) *If $\ln \varepsilon(T) = O(-T)$, then we say the algorithm has* linear convergence rate*;*

(2) *If $\ln \varepsilon(T)$ decays slower than $O(-T)$, then we say the algorithm has* sub–linear convergence rate*;*

(3) *If $\ln \varepsilon(T)$ decays faster than $O(-T)$, then we say the algorithm has* super–linear convergence rate*. In particular, if $\ln \ln \varepsilon(T) = O(-T)$, then we say the algorithm has* second–order convergence rate*.*

Theorem 3.1. *Assume that f is twice continuously differentiable, m–strongly convex and ∇f is L–Lipschitz, so that $mI \preceq \nabla^2 f(x) \preceq LI$ for all $x \in \mathbb{R}^n$. Then the GD Algorithm 3.1 with constant stepsize $\alpha_k = \alpha$ and $\alpha \in \left[\frac{1-\beta}{m}, \frac{1+\beta}{L}\right]$ for some $0 \leq \beta < 1$ has linear convergence rate.*

Proof. Consider the optimization problem $\min_{x \in \mathbb{R}^n} f(x)$. We assume f is twice continuously differentiable and satisfy $mI \preceq \nabla^2 f(x) \preceq LI$ for all $x \in \mathbb{R}^n$.

The Gradient Descent (GD) update rule is given by $x^{k+1} = x^k - \alpha \nabla f(x^k)$. Let $\Phi(x) = x - \alpha \nabla f(x)$ be the iteration operator, so that $x^{k+1} = \Phi(x^k)$. Since x^* is a local minimizer, $\nabla f(x^*) = 0$, which implies x^* is a fixed point:

$$x^* = \Phi(x^*) \ .$$

By the Mean Value Theorem for vector-valued functions, for any $x, z \in \mathbb{R}^n$, there exists a point c on the line segment $[x, z]$ such that:

$$\begin{aligned}\Phi(x) - \Phi(z) &= \nabla\Phi(c)(x - z) \\ &= (I - \alpha\nabla^2 f(c))(x - z) \ .\end{aligned}$$

Taking the spectral norm on both sides:

$$\|\Phi(x) - \Phi(z)\| \leq \|I - \alpha\nabla^2 f(c)\|_2 \cdot \|x - z\| \ .$$

To ensure linear convergence, we require Φ to be a contraction mapping with factor $\beta < 1$, i.e., $\|I - \alpha\nabla^2 f(c)\|_2 \leq \beta$.

Given $mI \preceq \nabla^2 f(x) \preceq LI$, the eigenvalues of the matrix $I - \alpha\nabla^2 f(c)$ lie in the interval $[1 - \alpha L, 1 - \alpha m]$. To find the optimal α that minimizes the maximum absolute eigenvalue β, we set:

$$-\beta = 1 - \alpha L \quad \text{and} \quad \beta = 1 - \alpha m$$

Solving this system yields:

1. **Optimal Stepsize:** $\alpha = \frac{2}{L+m}$
2. **Contraction Factor:** $\beta = \frac{L-m}{L+m}$

Since $L \geq m > 0$, it follows that $0 \leq \beta < 1$.

Applying the contraction property iteratively:

$$\|x^T - x^*\| = \|\Phi(x^{T-1}) - \Phi(x^*)\| \leq \beta\|x^{T-1} - x^*\| \leq \cdots \leq \beta^T\|x^0 - x^*\| \ .$$

To achieve a precision $\|x^T - x^*\| \leq \epsilon$, we require:

$$\beta^T\|x^0 - x^*\| \leq \epsilon \implies T \ln\beta \leq \ln\left(\frac{\epsilon}{\|x^0 - x^*\|}\right) \ .$$

Since $\ln\beta < 0$, we have:

$$T \geq \frac{\ln(\|x^0 - x^*\|/\epsilon)}{|\ln\beta|} \ .$$

This confirms that the algorithm converges at a linear rate. □

Taylor's expansion can give us a standard scheme for obtaining the convergence of optimization algorithms. To this end we consider

Lemma 3.2 (Descent Lemma)**.** *We have*

$$f(x^{k+1}) \leq f(x^k) - \frac{1}{2L}\|\nabla f(x^k)\|^2 \ . \tag{3.4}$$

Proof. We apply Taylor's theorem to get

$$\begin{aligned} f(x+\alpha d) &= f(x) + \alpha \nabla f(x)^T d + \alpha \int_0^1 [\nabla f(x+\gamma\alpha d) - \nabla f(x)]^T d \, d\gamma \\ &\le f(x) + \alpha \nabla f(x)^T d + \alpha \int_0^1 \|\nabla f(x+\gamma\alpha d) - \nabla f(x)\| \|d\| d\gamma \\ &\le f(x) + \alpha \nabla f(x)^T d + \alpha^2 \frac{L}{2} \|d\|^2 \ . \end{aligned} \tag{3.5}$$

From (3.5) and the line search $d = -\nabla f(x^k)$, $x = x^k$ on RHS of (3.5), then the $\alpha = \dfrac{1}{L}$ minimizes RHS of (3.5), and thus for GD in particular, we have

$$f(x^{k+1}) \le f(x^k) + \left(-\alpha + \alpha^2 \frac{L}{2}\right) \|\nabla f(x^k)\|^2 = f(x^k) - \frac{1}{2L} \|\nabla f(x^k)\|^2 \ .$$

□

Estimates of type (3.4) is a standard procedure that leads to convergence analysis of iterative optimization algorithms. Using it, for GD Algorithm 3.1 with $\alpha = \dfrac{1}{L}$ we have

Theorem 3.3. *Assume ∇f is L-Lipschitz. The convergence rates of GD algorithm has the following cases:*

(a) *If f is general nonconvex, then*

$$\min_{0 \le k \le T-1} \|\nabla f(x^k)\| \le \sqrt{\frac{2L(f(x^0) - f(x^*))}{T}} \ .$$

Number of iterations $0 \le k \le T$ until $\|\nabla f(x^k)\| \le \varepsilon$ is $T \ge \dfrac{2L(f(x^0) - f(x^*))}{\varepsilon^2}$. This is sublinear *convergence;*

(b) *If f is convex, then when the stepsize is $\alpha = \dfrac{1}{L}$ we have*

$$f(x^k) - f(x^*) \le \frac{L}{2k} \|x^0 - x^*\|^2 \ .$$

Number of iterations $0 \le k \le T$ until $f(x^k) - f(x^*) \le \varepsilon$ is $T \ge \dfrac{f(x^0) - f(x^*)}{\varepsilon}$. This is sublinear *convergence;*

(c) *If f is strongly convex with convexity constant $m > 0$, then*

$$f(x^k) - f(x^*) \le \left(1 - \frac{m}{L}\right)^k (f(x^0) - f(x^*)) \ .$$

Number of iterations $0 \le k \le T$ until $f(x^k) - f(x^*) \le \varepsilon$ is $T \ge \dfrac{L}{m} \ln\left(\dfrac{f(x^0) - f(x^*)}{\varepsilon}\right)$. This is linear *convergence.*

Proof. (a) Summing the Descent Lemma from $k = 0$ to $T - 1$:

$$\sum_{k=0}^{T-1}(f(x^k) - f(x^{k+1})) \geq \sum_{k=0}^{T-1} \frac{1}{2L}\|\nabla f(x^k)\|^2 \ .$$

The left side telescopes to $f(x^0) - f(x^T)$. Since $f(x^T) \geq f^*$:

$$f(x^0) - f^* \geq \frac{1}{2L}\sum_{k=0}^{T-1}\|\nabla f(x^k)\|^2 \geq \frac{T}{2L}\min_{0\leq k\leq T-1}\|\nabla f(x^k)\|^2$$

Rearranging gives the sublinear rate $O(1/\sqrt{T})$ for the gradient norm:

$$\min_{0\leq k\leq T-1}\|\nabla f(x^k)\| \leq \sqrt{\frac{2L(f(x^0) - f^*)}{T}} \ .$$

(b) By the first-order condition for convex functions $f(x^*) \geq f(x^k) + \nabla f(x^k)^T(x^* - x^k)$, and using the Descent Lemma:

$$\begin{aligned} f(x^{k+1}) &\leq f(x^*) + \nabla f(x^k)^T(x^k - x^*) - \frac{1}{2L}\|\nabla f(x^k)\|^2 \\ &= f(x^*) + \frac{L}{2}\left[\frac{2}{L}\nabla f(x^k)^T(x^k - x^*) - \frac{1}{L^2}\|\nabla f(x^k)\|^2\right] \ . \end{aligned}$$

Using the identity $\|a - b\|^2 = \|a\|^2 - 2a^Tb + \|b\|^2$ and the update $x^{k+1} = x^k - \frac{1}{L}\nabla f(x^k)$:

$$f(x^{k+1}) \leq f(x^*) + \frac{L}{2}(\|x^k - x^*\|^2 - \|x^{k+1} - x^*\|^2) \ .$$

Summing from $k = 0$ to $T - 1$ and noting that $f(x^k)$ is non-increasing:

$$T(f(x^T) - f^*) \leq \sum_{k=0}^{T-1}(f(x^{k+1}) - f^*) \leq \frac{L}{2}\|x^0 - x^*\|^2 \ .$$

This yields the sublinear rate $O(1/T)$ in function value.

(c) For m-strongly convex functions, we have the bound $\|\nabla f(x)\|^2 \geq 2m(f(x) - f^*)$. Substituting this into the Descent Lemma:

$$\begin{aligned} f(x^{k+1}) &\leq f(x^k) - \frac{1}{2L}(2m(f(x^k) - f^*)) \ , \\ f(x^{k+1}) - f^* &\leq (1 - \frac{m}{L})(f(x^k) - f^*) \ . \end{aligned}$$

Applying this recursively T times gives the linear convergence rate:

$$f(x^T) - f^* \leq \left(1 - \frac{m}{L}\right)^T (f(x^0) - f^*) \ .$$

□

3.2 Back Propagation and GD on Neural Networks

Classical algorithm of training multi–layer neural networks: Back–propagation. This algorithm was proposed in [22]. For a modern approach with the flavor of applier mathematics, we refer to [12].

Back–propagation method goes as follows. Let us set

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]} \text{ for } l = 2, 3, ..., L \ ,$$

in which l stands for the number of layers in the neural network. The neural network iteration can be viewed as

$$a^{[l]} = \sigma(z^{[l]}) \text{ for } l = 2, 3, ..., L \ .$$

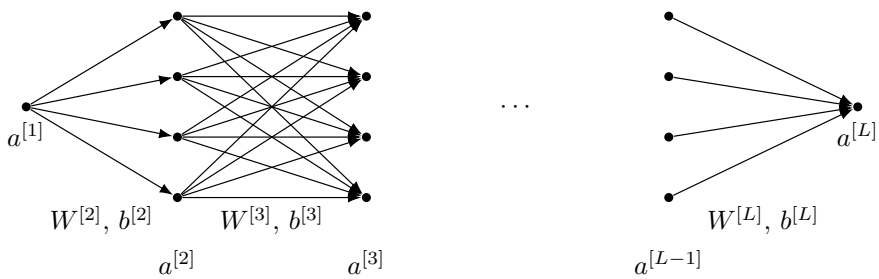

Figure 3.1: Feedforward fully-connected neural network (intermediate layers omitted).

We can view the neural network function as a chain with $a^{[1]} = x$ and $a^{[L]} = g(x; \omega)$. Let the training data be one point (x, y). Then the loss function is given by

$$C = C(\omega, b) = \frac{1}{2} \|y - a^{[L]}\|^2 \ .$$

To perform the GD algorithm as in Algorithm 3.1, we shall need the derivatives of the loss function C with respect to the neural–network parameters ω and b, that is $\dfrac{\partial C}{\partial w_{jk}^{[l]}}$ and $\dfrac{\partial C}{\partial b_j^{[l]}}$. We first have

Definition 3.3 (Error in Backpropagation)**.** *We say the* error *in the j–th neuron at layer l to be*

$$\delta^{[l]} = (\delta_j^{[l]})_j$$

(viewed as a column vector) with

$$\delta_j^{[l]} = \frac{\partial C}{\partial z_j^{[l]}} \ .$$

Definition 3.4 (Hadamad Product)**.** *If $x, y \in \mathbb{R}^n$, let $x \circ y \in \mathbb{R}^n$ to be the* Hadamard product *defined by*

$$(x \circ y)_i = x_i y_i \ .$$

Then one can calculate directly to obtain the following lemma

Lemma 3.4. *We have*

$$\delta^{[L]} = \sigma'(z^{[L]}) \circ (a^{[L]} - y) \ , \tag{3.6}$$

$$\delta^{[l]} = \sigma'(z^{[l]}) \circ (W^{[l+1]})^T \delta^{[l+1]} \ , \quad \textit{for } 2 \le l \le L-1 \ , \tag{3.7}$$

$$\frac{\partial C}{\partial b_j^{[l]}} = \delta_j^{[l]} \ , \ \textit{for } 2 \le l \le L \ , \tag{3.8}$$

$$\frac{\partial C}{\partial \omega_{jk}^{[l]}} = \delta_j^{[l]} a_k^{[l-1]} \ , \ \textit{for } 2 \le l \le L \ . \tag{3.9}$$

Proof. We prove each identity using the chain rule.
Proof of (3.6). Write the loss function componentwise as

$$C = \frac{1}{2} \sum_{j=1}^{n_L} (a_j^{[L]} - y_j)^2.$$

Then

$$\frac{\partial C}{\partial a_j^{[L]}} = a_j^{[L]} - y_j.$$

Since $a_j^{[L]} = \sigma(z_j^{[L]})$, the chain rule gives

$$\delta_j^{[L]} = \frac{\partial C}{\partial z_j^{[L]}} = \frac{\partial C}{\partial a_j^{[L]}} \frac{\partial a_j^{[L]}}{\partial z_j^{[L]}} = (a_j^{[L]} - y_j)\sigma'(z_j^{[L]}).$$

Writing this in vector form yields

$$\delta^{[L]} = \sigma'(z^{[L]}) \circ (a^{[L]} - y).$$

Proof of (3.7). Fix $l \in \{2, \dots, L-1\}$. Using the chain rule,

$$\delta_j^{[l]} = \frac{\partial C}{\partial z_j^{[l]}} = \sum_{m=1}^{n_{l+1}} \frac{\partial C}{\partial z_m^{[l+1]}} \frac{\partial z_m^{[l+1]}}{\partial z_j^{[l]}}.$$

Since $\dfrac{\partial C}{\partial z_m^{[l+1]}} = \delta_m^{[l+1]}$, it remains to compute

$$z_m^{[l+1]} = \sum_{k=1}^{n_l} w_{mk}^{[l+1]} a_k^{[l]} + b_m^{[l+1]}, \quad a_k^{[l]} = \sigma(z_k^{[l]}).$$

Therefore,

$$\frac{\partial z_m^{[l+1]}}{\partial z_j^{[l]}} = w_{mj}^{[l+1]} \sigma'(z_j^{[l]}).$$

Substituting back gives

$$\delta_j^{[l]} = \sigma'(z_j^{[l]}) \sum_{m=1}^{n_{l+1}} w_{mj}^{[l+1]} \, \delta_m^{[l+1]},$$

which is precisely

$$\delta^{[l]} = \sigma'(z^{[l]}) \circ (W^{[l+1]})^T \delta^{[l+1]}.$$

Proof of (3.8). Since

$$z_j^{[l]} = \sum_k w_{jk}^{[l]} a_k^{[l-1]} + b_j^{[l]},$$

we have $\dfrac{\partial z_j^{[l]}}{\partial b_j^{[l]}} = 1$ and zero otherwise. Hence,

$$\frac{\partial C}{\partial b_j^{[l]}} = \sum_{m=1}^{n_l} \frac{\partial C}{\partial z_m^{[l]}} \frac{\partial z_m^{[l]}}{\partial b_j^{[l]}} = \delta_j^{[l]}.$$

Proof of (3.9). Similarly,

$$\frac{\partial z_j^{[l]}}{\partial w_{jk}^{[l]}} = a_k^{[l-1]},$$

so

$$\frac{\partial C}{\partial w_{jk}^{[l]}} = \sum_{m=1}^{n_l} \frac{\partial C}{\partial z_m^{[l]}} \frac{\partial z_m^{[l]}}{\partial w_{jk}^{[l]}} = \delta_j^{[l]}\, a_k^{[l-1]}.$$

□

Algorithm 3.2 Gradient Descent on Neural Networks: Backpropagation

1: **Input**: Training data (x, y)
2: **Forward pass**: $x = a^{[1]} \to a^{[2]} \to \ldots \to a^{[L-1]} \to a^{[L]}$
3: Obtain $\delta^{[L]}$ from (3.6)
4: **Backward pass**: Form (3.7) obtain $\delta^{[L]} \to \delta^{[L-1]} \to \delta^{[L-2]} \to \ldots \to \delta^{[2]}$
5: **Calculation of the gradient of the loss** C: Use (3.8) and (3.9)

Training GD on Over–parametrized Neural Networks has been understood only recently. See [3, 8, 9, 30].

Computational Graph and General Backpropagation for taking derivatives: basic idea in tensorflow (see [25]).

Example. Consider

$$f(x) = \ln\big(\sin^2(x^2)\big),$$

defined for $\sin(x^2) \neq 0$. Introduce intermediate variables

$$u = x^2, \qquad v = \sin(u), \qquad w = v^2, \qquad f = \ln(w). \tag{3.10}$$

Computational graph:

$$\boxed{x} \longrightarrow \boxed{u = x^2} \longrightarrow \boxed{v = \sin(u)} \longrightarrow \boxed{w = v^2} \longrightarrow \boxed{f = \ln(w)}$$

$$\frac{\partial u}{\partial x} = 2x \qquad \frac{\partial v}{\partial u} = \cos(u) \qquad \frac{\partial w}{\partial v} = 2v \qquad \frac{\partial f}{\partial w} = \frac{1}{w}$$

Reverse-mode (backprop) computation:

Define adjoints $\bar{u} = \partial f/\partial u$, $\bar{v} = \partial f/\partial v$, $\bar{w} = \partial f/\partial w$, and $\bar{x} = \partial f/\partial x$. Start from the output:

$$\bar{f} = 1, \qquad \bar{w} = \bar{f}\,\frac{\partial f}{\partial w} = \frac{1}{w}.$$

Propagate backward along the edges using the chain rule:

$$\begin{aligned}
\bar{v} &= \bar{w}\,\frac{\partial w}{\partial v} = \frac{1}{w}(2v) = \frac{2}{v}, \\
\bar{u} &= \bar{v}\,\frac{\partial v}{\partial u} = \frac{2}{v}\cos(u) = 2\frac{\cos(u)}{\sin(u)} = 2\cot(u), \\
\bar{x} &= \bar{u}\,\frac{\partial u}{\partial x} = \big(2\cot(u)\big)(2x) = 4x\cot(u).
\end{aligned}$$

Substituting $u = x^2$ gives

$$f'(x) = \frac{\mathrm{d}f}{\mathrm{d}x} = 4x\cot(x^2), \qquad \sin(x^2) \neq 0. \tag{3.11}$$

This illustrates the key idea of automatic differentiation: evaluate the forward pass (3.10) once, then compute derivatives by a single backward pass using local Jacobians at each node.

In PyTorch, all computational flows are stored during calculation, so that any gradient can be calculated using back-propagation. This takes memory. So `torch.no_grad()` (inference mode) can be used to accelerate the inference process if no gradient computation is needed.

3.3 Online Principle Component Analysis (PCA)

Let $\boldsymbol{X}$ be a random vector in $\mathbb{R}^d$ with mean 0 and unknown covariance matrix

$$\Sigma = \mathbf{E}\left[\boldsymbol{X}\boldsymbol{X}^T\right] \in \mathbb{R}^{d\times d}\ .$$

Let the eigenvalues of Σ be ordered as $\lambda_1 > \lambda_2 \geq ... \geq \lambda_d \geq 0$. Principal Component Analysis (PCA) aims to find the principal eigenvector of Σ that corresponds to the largest eigenvalue λ_1, based on independent and identically distributed sample realizations $\boldsymbol{X}^{(1)}, ..., \boldsymbol{X}^{(n)}$. This can be casted into a *nonconvex stochastic optimization problem* given by

$$\begin{aligned}
&\text{maximize } \boldsymbol{u}^T\mathbf{E}\left[\boldsymbol{X}\boldsymbol{X}^T\right]\boldsymbol{u}\ , \\
&\text{subject to } \|\boldsymbol{u}\| = 1, \boldsymbol{u} \in \mathbb{R}^d\ .
\end{aligned}$$

Here $\| \bullet \|$ denotes the Euclidian norm. Assume the principle component $\boldsymbol{u}^*$ is unique.

Classical PCA:

$$\widehat{\boldsymbol{u}}^{(n)} = \arg\max_{\|\boldsymbol{u}\|=1} \boldsymbol{u}^T\widehat{\Sigma}^{(n)}\boldsymbol{u}\ .$$

Here $\widehat{\Sigma}^{(n)}$ is the empirical covariance matrix based on n samples:

$$\widehat{\Sigma}^{(n)} = \frac{1}{n}\sum_{i=1}^{n}\boldsymbol{X}^{(i)}(\boldsymbol{X}^{(i)})^T\ .$$

We can design an online version of the PCA algorithm as in Algorithm 3.3, where we must do at every iteration a normalization step using the retraction operation

$$\Pi \boldsymbol{u} = \frac{\boldsymbol{u}}{\|\boldsymbol{u}\|} \ .$$

Algorithm 3.3 Online PCA Algorithm

1: **Input**: Initialize $\boldsymbol{u}^{(0)}$ and choose the stepsize $\beta > 0$
2: **for** $n = 1, 2, ..., N$ **do**
3: Draw one sample $\boldsymbol{X}^{(n)}$ from the (streaming) data source
4: Update the iterate $\boldsymbol{u}^{(n)}$ by

$$\boldsymbol{u}^{(n)} = \Pi \left\{ \boldsymbol{u}^{(n-1)} + \beta \boldsymbol{X}^{(n)} (\boldsymbol{X}^{(n)})^T \boldsymbol{u}^{(n-1)} \right\} \ .$$

5: **end for**
6: **Output**: $\boldsymbol{u}^{(N)}$

The convergece of online PCA has been proved only recently in [16]. The difficulty here is that this online PCA is non-convex optimization.

Oja's algorithm: Step 4 in Algorithm 3.3 will be modified to

$$\boldsymbol{u}^{(n)} = \boldsymbol{u}^{(n-1)} + \beta \left(\boldsymbol{X}^{(n)} (\boldsymbol{X}^{(n)})^T \boldsymbol{u}^{(n-1)} - \left((\boldsymbol{u}^{(n-1)})^T X^{(n)} \right)^2 \boldsymbol{u}^{(n-1)} \right) \ ,$$

which is expected to control the norm growth. See [19].

3.4 Line Search Methods

Line search methods in general takes the following form

$$x^{k+1} = x^k + \alpha_k d^k \ , \ k = 0, 1, 2, ... \tag{3.12}$$

Here $\alpha_k > 0$ is the stepsize and $d^k \in \mathbb{R}^n$ is the descent direction.

Preliminary conditions: for $\bar{\varepsilon}, \gamma_1, \gamma_2 > 0$ we assume

$$\begin{aligned} &(1) \quad -(d^k)^T \nabla f(x^k) \geq \bar{\varepsilon} \|\nabla f(x^k)\| \cdot \|d^k\|; \\ &(2) \quad \gamma_1 \|\nabla f(x^k)\| \leq \|d^k\| \leq \gamma_2 \|\nabla f(x^k)\| \ . \end{aligned} \tag{3.13}$$

Notice that if $d^k = -\nabla f(x^k)$, then $\bar{\varepsilon} = \gamma_1 = \gamma_2 = 1$.

In fact one can expand

$$\begin{aligned} f(x^{k+1}) &= f(x^k + \alpha d^k) \\ &= f(x^k) + \alpha \nabla f(x^k)^T d^k + \alpha \int_0^1 [\nabla f(x^k + \gamma \alpha d^k) - \nabla f(x^k)]^T d^k d\gamma \\ &\leq f(x^k) - \alpha \left(\bar{\varepsilon} - \alpha \frac{L}{2} \gamma_2 \right) \|\nabla f(x^k)\| \cdot \|d^k\| \ . \end{aligned} \tag{3.14}$$

So if $\alpha \in \left(0, \frac{2\bar{\varepsilon}}{L\gamma_2}\right)$, then $f(x^{k+1}) < f(x^k)$, leading to a descent in objective value. Schemes of descent type produce iterates that satisfy a bound of the form

$$f(x^{k+1}) \leq f(x^k) - C\|\nabla f(x^k)\|^2 \text{ for some } C > 0 \ . \tag{3.15}$$

Various Line Search Methods:

(1) $d^k = -S^k \nabla f(x^k)$, S^k symmetric positively definite, $\lambda(S^k) \in [\gamma_1, \gamma_2]$;
In this case we have $-(d^k)^T \nabla f(x_k) = (\nabla f(x_k))^T S^k (\nabla f(x_k)) \geq \gamma_2 \|\nabla f(x^k)\|^2 \geq \gamma_2 \gamma_2^{-1} \|\nabla f(x^k)\| \cdot \|d^k\| = \|\nabla f(x^k)\| \cdot \|d^k\|$.

(2) Gauss-Southwell: $d^k = -[\nabla f(x^k)]_{i_k}$ where

$$i_k = \arg \max_{i=1,2,\ldots,n} \left| [\nabla f(x^k)]_i \right|$$

and

$$\nabla f(x) = ([\nabla f(x)]_1, \ldots, [\nabla f(x)]_n) \ ;$$

(3) Stochastic coordinate descent: $d^k = -[\nabla f(x^k)]_{i_k}$ where i_k is uniformly randomly drawn from $\{1, 2, \ldots, n\}$;

(4) Stochastic Gradient Methods: d^k is random, we will specify it later in Chapter 5;

(5) Exact line search $\min_{\alpha > 0} f(x^k + \alpha d^k)$;

(6) Approximate Line Search: choose a certain $\alpha > 0$, not necessarily the exact α in the above. This usually is accompanied with the "weak Wolfe conditions", which consists of (a) Armijo (Sufficient Decrease) Condition: $f(x^k + \alpha p^k) \leq f(x^k) + c_1 \alpha_k (p^k)^T \nabla f(x^k)$, where $c_1 \in (0, 1)$ and is often chosen to be of a small order of magnitude around 10^{-3}. This condition ensures the computed step length can sufficiently decrease the objective function $f(x^k)$. Only using this condition, however, it cannot be guaranteed that x^k will converge in a reasonable number of iterations, since Armijo condition is always satisfied with step length being small enough. Therefore, the second condition below needs to be paired with the sufficient decrease condition to keep α_k from being too short. (b) Curvature condition: $(\nabla f(x^k + \alpha p^k))^T p^k \geq c_2 (\nabla f(x^k))^T p^k$ where $c_2 \in (c_1, 1)$ is much greater than c_1 and is typically on the order of 0.1. The condition ensures a sufficient increase of the gradient;

(7) Backtracking Line Search: Starting from some $\bar{\alpha} > 0$ then look for $\alpha \in \left\{\bar{\alpha}, \beta\bar{\alpha}, \beta^2\bar{\alpha}, \ldots\right\}$ where $0 < \beta < 1$.

3.5 Convergence to Approximate Second Order Necessary Points

Second–order necessary point: $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive semidefinite.

Definition 3.5. *We call point $x \in \mathbb{R}^n$ an $(\varepsilon_g, \varepsilon_H)$–approximate second order stationary point if*

$$\|\nabla f(x)\| \leq \varepsilon_g \text{ and } \lambda_{min}(\nabla^2 f(x)) \geq -\varepsilon_H \tag{3.16}$$

for some choices of small constants $\varepsilon_g > 0$ and $\varepsilon_H > 0$.

How to search for such a second–order stationary point?

Set $M > 0$ to be the Lipschitz bound for the Hessian $\nabla^2 f(x)$, so that

$$\|\nabla^2 f(x) - \nabla^2 f(y)\| \leq M\|x - y\| \text{ for all } x, y \in \mathrm{dom}(f) \ . \tag{3.17}$$

Algorithm 3.4 Search for second–order approximate stationary point based on Gradient Descent

Input: Input initialization x^0, $k = 0, 1, 2, ...$
while $\|\nabla f(x^k)\| > \varepsilon_g$ **or** $\lambda_k < -\varepsilon_H$ **do**
  if $\|\nabla f(x^k)\| > \varepsilon_g$ **then**
    $x^{k+1} = x^k - \frac{1}{L}\nabla f(x^k)$
  else
    $\lambda^k := \lambda_{\min}(\nabla^2 f(x^k))$
    Choose p^k to be the eigenvector corresponding to the most negative eigenvalue of $\nabla^2 f(x^k)$
    Choose the size and sign of p^k such that $\|p^k\| = 1$ and $(p^k)^T \nabla f(x^k) \leq 0$
    $x^{k+1} = x^k + \alpha_k p^k$, $\alpha_k = \frac{2\lambda_k}{M}$
  end if
  $k \leftarrow k + 1$
end while
Output: An estimate of an $(\varepsilon_g, \varepsilon_H)$–approximate second order stationary point x^k

Cubic upper bound

$$f(x + p) \leq f(x) + \nabla f(x)^T p + \frac{1}{2} p^T \nabla^2 f(x) p + \frac{1}{6} M\|p\|^3 \ . \tag{3.18}$$

For steepest descent step we have from (3.4) that

$$f(x^{k+1}) \leq f(x^k) - \frac{1}{2L}\|\nabla f(x^k)\|^2 \leq f(x^k) - \frac{\varepsilon_g^2}{2L} \ .$$

Otherwise, the negative direction Hessian eigenvector search will output, by using third–order Taylor expansion and (3.17), that

$$\begin{aligned} f(x^{k+1}) &\leq f(x^k) + \alpha_k \nabla f(x^k)^T p^k + \frac{1}{2}\alpha_k^2 (p^k)^T \nabla^2 f(x^k) p^k + \frac{1}{6} M \alpha_k^3 \|p^k\|^3 \\ &\leq f(x^k) - \frac{1}{2}\left(\frac{2|\lambda_k|}{M}\right)^2 |\lambda_k| + \frac{1}{6} M \left(\frac{2|\lambda_k|}{M}\right)^3 \\ &= f(x^k) - \frac{2}{3}\frac{|\lambda_k|^3}{M^2} \\ &= f(x^k) - \frac{2}{3}\frac{\varepsilon_H^3}{M^2} \ . \end{aligned}$$

Number of iterations

$$K \leq \max\left(2L\varepsilon_g^{-2}, \frac{3}{2}M^2\varepsilon_H^{-3}\right)(f(x^0) - f(x^*)) \ .$$

3.6 Experiment: Learning a Neural Network via Gradient Descent and Backpropagation

Consider an L-hidden-layer fully connected neural network with hidden layer sizes $n_1, ..., n_L$, where the input layer has only one neuron $x \in \mathbb{R}$ and output layer has only one neuron $y \in \mathbb{R}$. Let the training data $(x_1, y_1), ..., (x_N, y_N)$ be an i.i.d sample of a bivariate unit normal distribution $(x, y) \in \mathcal{N}(0, I_2)$. We use the backpropagation algorithm to calculate the gradient vector of the empirical loss function and the population loss function of the neural network for a quadratic loss.

We experiment

(1) Different hidden layer sizes: $(n_1, ..., n_L)$;

(2) Different number of hidden layers: L;

(3) Different sizes of the training data set: n;

(4) Different activation functions: Sigmoid, ReLU, Tanh.

Following the above, we plot the 3-d graph of the loss function and the trajectory of the gradient descent algorithm by picking two network weight parameters as variables (from a certain layer to its next layer) and randomly choosing other network parameters as i.i.d. samples of normal distributions that follow the LeCun initialization: $w_{ij}^{(l)} \sim \mathcal{N}(0, \frac{1}{n_l})$ and $b_i^{(l)} \sim \mathcal{N}(0, 1)$, where n_l is the network width at level l.

Code can be found at https://github.com/huwenqing0606/Computational_Optimization/tree/main/src/backpropagation

Chapter 4

Accelerated Gradient Methods

4.1 Polyak's heavy ball method

Why acceleration in GD?

The GD method is "greedy" in the sense that it steps in the direction that is most productive at current iterate – no explicit use of knowledge of the objective function f at earlier iterations.

Adding "momentum" = search direction tends to be similar to that one used in the previous step.

Say the objective function $f(x) = \frac{1}{2}x^T Qx - b^T x + c$ is a quadratic function. It may well happen in deep neural network models, near equilibrium, the loss function is approximated by such a function f with a matrix Q that has a large condition number $\kappa = \frac{L}{m}$, see [6]. Slow convergence for κ large: for the strongly convex case we showed that in GD case the convergence rate is $\beta = 1 - \frac{1}{\kappa}$ which $\to 1$ when $\kappa \to \infty$. Fig 4.1 demonstrates oscillations of GD near local min when the Hessian has a large condition number.

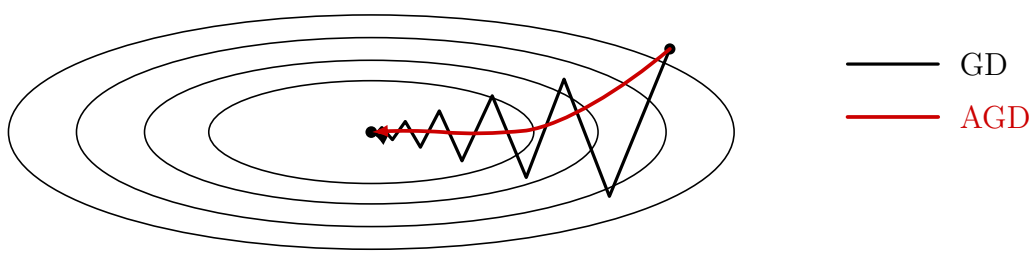

Figure 4.1: Schematic comparison of GD (zig-zag) and Accelerated GD (smoother) on an ill-conditioned valley.

Motivated from classical mechanics, GD is approximated by a first order ODE of the form

$$\text{GD:} \qquad \frac{dx}{dt} = -\nabla f(x) \ .$$

It is natural to modify the above equation to a second order Newton's equation that models a nonlinear oscillator:

$$\text{AGD:} \qquad \mu \frac{d^2x}{dt^2} = -\nabla f(x) - \mu b \frac{dx}{dt} \ .$$

"Small mass limit" $\mu \to 0$. Term $\mu \frac{d^2x}{dt^2}$ serves as "regularization".

Finite difference approximation:

$$\mu \frac{x(t+\Delta t) - 2x(t) + x(t-\Delta t)}{(\Delta t)^2} \approx -\nabla f(x(t)) - \frac{\mu b(x(t+\Delta t) - x(t))}{\Delta t} \ .$$

i.e.

$$x(t+\Delta t) = x(t) - \alpha \nabla f(x(t)) + \beta(x(t) - x(t-\Delta t)) \ .$$

This gives the "Polyak's heavy ball method" (see [20]).

$$x^{k+1} = x^k - \alpha \nabla f(x^k) + \beta(x^k - x^{k-1}) \ , \ x^{-1} = x^0 \ .$$

Algorithm 4.1 Polyak's heavy ball method (GD with momentum)

1: **Input**: Input initialization $x^{-1} = x^0$, $p^{-1} = 0$, stepsizes $\alpha, \beta > 0$, $k = 0, 1, 2, ...$
2: **for** $k = 0, 1, 2, ..., K-1$ **do**
3: $\quad p^k = -\alpha \nabla f(x^k) + \beta p^{k-1}$
4: $\quad x^{k+1} = x^k + p^k$
5: **end for**
6: **Output**: Output x^K

"Momentum": $p_k = x^{k+1} - x^k$. The momentum iteration

$$p^k = \beta p^{k-1} - \alpha \nabla f(x^k)$$

can be understood as doing an exponential moving average over the gradients. This is where the "memory" of "inertia" comes in, that inspires the adaptive methods.

4.2 Nesterov's Accelerated Gradient Descent (AGD)

"first order optimal method": Instead of evaluating $\nabla f(x^k)$, we evaluate $\nabla f(x^k + \beta(x^k - x^{k-1}))$.

Nesterov's accelerated gradient descent:

$$x^{k+1} = x^k - \alpha \nabla f(x^k + \beta(x^k - x^{k-1})) + \beta(x^k - x^{k-1}) \ , \ x^{-1} = x^0 \ . \tag{4.1}$$

See [17].

Algorithm 4.2 Nesterov's accelerated gradient descent

Input: Input initialization $x^{-1} = x^0$, stepsizes $\alpha_k, \beta_k > 0$, $k = 0, 1, 2, ...$
for $k = 0, 1, 2, ..., K-1$ **do**
  $y^k = x^k + \beta_k(x^k - x^{k-1})$
  $x^{k+1} = y^k - \alpha_k \nabla f(y^k)$
end for
Output: Output x^K

By introducing the momentum variable $p^k = x^{k+1} - x^k$ we can also rewrite the Nesterov's acclerated gradient descent in terms of momentum, this is the Netsterov's momentum algorithm:

Algorithm 4.3 Nesterov's momentum (equivalent to Nesterov's accelerated gradient descent)

Input: Input initialization $x^{-1} = x^0$, $p^{-1} = 0$, stepsizes $\alpha_k, \beta_k > 0$, $k = 0, 1, 2, ...$
for $k = 0, 1, 2, ..., K-1$ **do**
  $p^k = \beta_k p^{k-1} - \alpha_k \nabla f(x^k + \beta_k p^{k-1})$
  $x^{k+1} = x^k + p^k$
end for
Output: Output x^K

4.2.1 Convergence Proof of Nesterov's method: Quadratic Case.

Theorem 4.1 (Convergence of Nesterov's AGD in the quadratic case)**.** *Consider the quadratic function*

$$f(x) = \frac{1}{2}x^T Q x - b^T x + c,$$

where $Q \in \mathbb{R}^{n\times n}$ is symmetric positive definite and $0 < m = \lambda_n \leq \cdots \leq \lambda_1 = L$. Let the condition number $\kappa = \dfrac{L}{m}$ and the unique minimizer be x^*. Then the Nesterov's method with iteration scheme (4.1) and parameter choice $\alpha = \dfrac{1}{L}$, $\beta = \dfrac{\sqrt{L}-\sqrt{m}}{\sqrt{L}+\sqrt{m}} = \dfrac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ gives a convergence rate

$$\|x^k - x^*\| \leq \left(1 - \sqrt{\frac{m}{L}}\right)^k \|x^0 - x^*\|. \tag{4.2}$$

Consequently,

$$f(x^k) - f(x^*) = O\Big(\left(1 - \sqrt{\frac{m}{L}}\right)^k\Big). \tag{4.3}$$

Proof. Since $\nabla f(x) = Qx - b$, the unique minimizer satisfies $Qx^* = b$. Substituting $\nabla f(x) = Qx - b$ into (4.1) gives

$$x^{k+1} = x^k - \alpha\Big(Q(x^k + \beta(x^k - x^{k-1})) - b\Big) + \beta(x^k - x^{k-1}).$$

Define the error

$$e^k = x^k - x^*.$$

Since $Qx^* = b$, we obtain

$$Q(x^k + \beta(x^k - x^{k-1})) - b = Q\big(e^k + \beta(e^k - e^{k-1})\big).$$

Therefore

$$e^{k+1} = e^k - \alpha Q(e^k + \beta(e^k - e^{k-1})) + \beta(e^k - e^{k-1}).$$

Rearranging yields

$$e^{k+1} = (1+\beta)(I - \alpha Q)e^k - \beta(I - \alpha Q)e^{k-1}. \tag{4.4}$$

Define the extended state vector

$$w^k = \begin{pmatrix} e^k \\ e^{k-1} \end{pmatrix}.$$

Then the system (4.4) can be written as

$$w^{k+1} = Tw^k,$$

where

$$T = \begin{pmatrix} (1+\beta)(I - \alpha Q) & -\beta(I - \alpha Q) \\ I & 0 \end{pmatrix}.$$

Let $\rho(T)$ be the spectral radius of the matrix T, i.e., $\rho(T) \equiv \lim_{k\to\infty} \big(\|T^k\|\big)^{1/k}$. Since Q is symmetric positive definite,

$$Q = U \operatorname{diag}(\lambda_1, \ldots, \lambda_n) U^T.$$

Under the change of variables

$$\tilde{w}^k = \begin{pmatrix} U^T & 0 \\ 0 & U^T \end{pmatrix} w^k,$$

the matrix T becomes block diagonal with 2×2 blocks

$$T_i = \begin{pmatrix} (1+\beta)(1 - \alpha\lambda_i) & -\beta(1 - \alpha\lambda_i) \\ 1 & 0 \end{pmatrix}.$$

The characteristic polynomial of T_i is

$$\nu^2 - (1+\beta)(1 - \alpha\lambda_i)\nu + \beta(1 - \alpha\lambda_i) = 0.$$

Thus

$$\nu_{1,2} = \frac{(1+\beta)(1 - \alpha\lambda_i) \pm \sqrt{(1+\beta)^2(1 - \alpha\lambda_i)^2 - 4\beta(1 - \alpha\lambda_i)}}{2}.$$

Choose

$$\alpha = \frac{1}{L}, \qquad \beta = \frac{\sqrt{L} - \sqrt{m}}{\sqrt{L} + \sqrt{m}}.$$

Then the discriminant is non-positive and

$$|\nu_i| = \sqrt{\beta}\sqrt{1 - \frac{\lambda_i}{L}}.$$

Hence

$$\rho(T) = \max_i |\nu_i| = 1 - \sqrt{\frac{m}{L}}.$$

This gives the asserted convergence results in (4.2) and (4.3). □

Comparison with Gradient Descent.

Gradient Descent:

$$(1 - \frac{m}{L})^k \ ,$$

Accelerated Gradient:

$$(1 - \sqrt{\frac{m}{L}})^k \ .$$

GD needs $O\left(\frac{L}{m}|\ln\varepsilon|\right)$ to reach $f(x^k) - f^* \leq \varepsilon$ while Nesterov needs $O\left(\sqrt{\frac{L}{m}}|\ln\varepsilon|\right)$ to reach a factor ε in $\|\omega^k\|$. Key parameter is $\kappa = \frac{L}{m}$.

Example: $\kappa = 10000$, then GD converges with iteration $O(10000|\ln\varepsilon|)$ but Nesterov needs only $O(100|\ln\varepsilon|)$ iterations.

4.2.2 Convergence Proof of Nesterov's method: Strongly Convex Case.

Definition 4.1 (Lyapunov function)**.** *For a sequence $\{\omega^k\}$ we define its corresponding* Lyapunov function $V : \mathbb{R}^d \to \mathbb{R}$, $V(\omega) > 0$ *to be that for all $\omega \neq \omega^* \in \mathbb{R}^d$, we have*

$$V(\omega^*) = 0 \ , \ V(\omega^{k+1}) < \rho^2 V(\omega^k)$$

for some $0 < \rho < 1$.

Indeed, $V(x) = f(x) - f^*$ is a Lyapunov function for GD.

What is the Lyapunov function for Nesterov?

Theorem 4.2 (Convergence of Nesterov's AGD in the strongly convex case)**.** *Assume $f(x)$ is strongly m-convex with ∇f being L-Lipschitz, and assume x^* is the unique minimizer of $f(x)$. Let $\widetilde{x}^k = x^k - x^*$, where x^k is the k-th iteration of Nesterov's AGD. Set*

$$V_k = f(x^k) - f(x^*) + \frac{L}{2}\|\widetilde{x}^k - \rho^2\widetilde{x}^{k-1}\|^2 \ .$$

Set $\alpha_k = \frac{1}{L}$, $\beta_k = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$, $\rho^2 = 1 - \frac{1}{\sqrt{\kappa}}$. Then we have

$$V_{k+1} \leq \rho^2 V_k \ . \tag{4.5}$$

Proof. The Nesterov's iteration gives

$$x^{k+1} = x^k - \alpha_k \nabla f\left(x^k + \beta(x^k - x^{k-1})\right) + \beta_k(x^k - x^{k-1}) \ , \ x^0 = x^{-1} \ .$$

We rewrite it as the following iteration

$$\begin{cases} y^k = x^k + \beta_k(x^k - x^{k-1}) \ , \\ x^{k+1} = y^k - \alpha_k \nabla f(y^k) \ . \end{cases}$$

We recall that we have $m \le \lambda(\nabla^2 f) \le L$ and $\kappa = \dfrac{L}{m}$, $\alpha_k = \dfrac{1}{L}$, $\beta_k = \dfrac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$. Let $y^* = x^*$ and $\widetilde{y}^k = y^k - y^*$. Then $\widetilde{x}^k - \widetilde{y}^k = x^k - y^k$. We apply the argument used in Descent Lemma 3.2 with x^k replaced by y^k in (4.1) to get

$$f(x^{k+1}) = f\left(y^k - \frac{1}{L}\nabla f(y^k)\right) \le f(y^k) - \frac{1}{2L}\|\nabla f(y^k)\|^2 \ . \tag{4.6}$$

Set $u^k = \dfrac{1}{L}\nabla f(y^k)$ and $u^* = \dfrac{1}{L}\nabla f(y^*) = 0$. Define $\widetilde{u}^k = u^k - u^*$. Then

$$V_{k+1} = f(x^{k+1}) - f^* + \frac{L}{2}\left\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\right\|^2 \ge f(x^{k+1}) - f^* \ ,$$

and by (4.6) we have

$$\begin{aligned} V_{k+1} \ &\le f(y^k) - \frac{1}{2L}\|\nabla f(y^k)\|^2 - f^* + \frac{L}{2}\left\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\right\|^2 \\ &= f(y^k) - \frac{L}{2}\|u^k\|^2 - f^* + \frac{L}{2}\left\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\right\|^2 \\ &= \rho^2\Big[f(y^k) - f^* + L(u^k)^T(\tilde{x}^k - \tilde{y}^k)\Big] \\ &\quad -\rho^2 L(u^k)^T(\tilde{x}^k - \tilde{y}^k) \\ &\quad +(1-\rho^2)\Big[f(y^k) - f^* - L(u^k)^T\tilde{y}^k\Big] \\ &\quad +(1-\rho^2)L(u^k)^T\tilde{y}^k - \frac{L}{2}\|u^k\|^2 + \frac{L}{2}\left\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\right\|^2 \ . \end{aligned} \tag{4.7}$$

Since $f(x)$ is m-strongly convex, we have, by (2.13)

$$f(x^k) \ge f(y^k) + (\nabla f(y^k))^T(x^k - y^k) + \frac{m}{2}\|x^k - y^k\|^2 \ ,$$

which gives

$$f(y^k) \le f(x^k) - L(\widetilde{u}_k)^T(x^k - y^k) - \frac{m}{2}\|x^k - y^k\|^2 \ . \tag{4.8}$$

Similarly, we have

$$f(x^*) \ge f(y^k) + (\nabla f(y^k))^T(x^* - y^k) + \frac{m}{2}\|x^* - y^k\|^2 \ ,$$

which gives

$$f(x^*) \ge f(y^k) - L(\widetilde{u}^k)^T\widetilde{y}^k + \frac{m}{2}\|\widetilde{y}^k\|^2 \ . \tag{4.9}$$

Combining (4.7), (4.8) and (4.9) we have

$$\begin{aligned}
V_{k+1} &\le \rho^2\Big[f(x^k) - f^* - \frac{m}{2}\|\tilde{x}^k - \tilde{y}^k\|^2\Big] - \rho^2 L(\tilde{u}^k)^T(\tilde{x}^k - \tilde{y}^k) \\
&\quad +(1-\rho^2)\Big[f(y^k) - f^* - L(\tilde{u}^k)^T\tilde{y}^k\Big] + (1-\rho^2)L(\tilde{u}^k)^T\tilde{y}^k - \frac{L}{2}\|\tilde{u}^k\|^2 + \frac{L}{2}\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\|^2 \\
&= \rho^2\Big[f(x^k) - f^* - \frac{m}{2}\|\tilde{x}^k - \tilde{y}^k\|^2\Big] - \frac{m(1-\rho^2)}{2}\|\tilde{y}^k\|^2 \\
&\quad -\rho^2 L(\tilde{u}^k)^T(\tilde{x}^k - \tilde{y}^k) + (1-\rho^2)L(\tilde{u}^k)^T\tilde{y}^k - \frac{L}{2}\|\tilde{u}^k\|^2 + \frac{L}{2}\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\|^2 \\
&= \rho^2\Big[f(x^k) - f^* + \frac{L}{2}\|\tilde{x}^k - \rho^2\tilde{x}^{k-1}\|^2\Big] \\
&\quad -\frac{m\rho^2}{2}\|\tilde{x}^k - \tilde{y}^k\|^2 - \frac{m(1-\rho^2)}{2}\|\tilde{y}^k\|^2 - \frac{\rho^2 L}{2}\|\tilde{x}^k - \rho^2\tilde{x}^{k-1}\|^2 \\
&\quad +L(\tilde{u}^k)^T(\tilde{y}^k - \rho^2\tilde{x}^k) - \frac{L}{2}\|\tilde{u}^k\|^2 + \frac{L}{2}\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\|^2 \ .
\end{aligned} \tag{4.10}$$

Let

$$\begin{aligned}
R_k = \quad & -\frac{m\rho^2}{2}\|\tilde{x}^k - \tilde{y}^k\|^2 - \frac{m(1-\rho^2)}{2}\|\tilde{y}^k\|^2 - \frac{\rho^2 L}{2}\|\tilde{x}^k - \rho^2\tilde{x}^{k-1}\|^2 \\
& +L(\tilde{u}^k)^T(\tilde{y}^k - \rho^2\tilde{x}^k) - \frac{L}{2}\|\tilde{u}^k\|^2 + \frac{L}{2}\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\|^2 \ .
\end{aligned} \tag{4.11}$$

So we have

$$V_{k+1} \le \rho^2 V_k + R_k \ .$$

Using Lemma 4.3, the above gives (4.5). □

Lemma 4.3. *Under the assumptions in* Theorem 4.2*, we have*

$$R_k = -\frac{1}{2}L\rho^2\left(\frac{1}{\kappa} + \frac{1}{\sqrt{\kappa}}\right)\|\tilde{x}^k - \tilde{y}^k\|^2 \le 0 \ ,$$

where R_k is defined in (4.11)*.*

Proof. Let $\alpha = \alpha_k = \frac{1}{L}$, $\beta = \beta_k = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$, $\rho^2 = 1 - \frac{1}{\sqrt{\kappa}}$. We have

$$\begin{aligned}
R_k = & -\frac{m\rho^2}{2}\|\tilde{x}^k - \tilde{y}^k\|^2 - \frac{m(1-\rho^2)}{2}\|\tilde{y}^k\|^2 \\
& + L(\tilde{u}^k)^T(\tilde{y}^k - \rho^2\tilde{x}^k) - \frac{L}{2}\|\tilde{u}^k\|^2 + \frac{L}{2}\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\|^2 - \frac{\rho^2 L}{2}\|\tilde{x}^k - \rho^2\tilde{x}^{k-1}\|^2.
\end{aligned}$$

Now complete the square in the middle terms:

$$\begin{aligned}
& L(\tilde{u}^k)^T(\tilde{y}^k - \rho^2\tilde{x}^k) - \frac{L}{2}\|\tilde{u}^k\|^2 \\
& \quad = -\frac{L}{2}\Big[\|\tilde{y}^k - \rho^2\tilde{x}^k - \tilde{u}^k\|^2 - \|\tilde{y}^k - \rho^2\tilde{x}^k\|^2\Big].
\end{aligned}$$

Since

$$\tilde{y}^k - \tilde{u}^k = y^k - u^k - y^* = x^{k+1} - x^* = \tilde{x}^{k+1},$$

we obtain

$$\begin{aligned}
R_k &= -\frac{m\rho^2}{2}\|\tilde{x}^k - \tilde{y}^k\|^2 - \frac{m(1-\rho^2)}{2}\|\tilde{y}^k\|^2 \\
&\quad - \frac{L}{2}\Big[\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\|^2 - \|\tilde{y}^k - \rho^2\tilde{x}^k\|^2\Big] \\
&\quad + \frac{L}{2}\|\tilde{x}^{k+1} - \rho^2\tilde{x}^k\|^2 - \frac{\rho^2 L}{2}\|\tilde{x}^k - \rho^2\tilde{x}^{k-1}\|^2 \\
&= -\frac{m\rho^2}{2}\|\tilde{x}^k - \tilde{y}^k\|^2 - \frac{m(1-\rho^2)}{2}\|\tilde{y}^k\|^2 \\
&\quad + \frac{L}{2}\|\tilde{y}^k - \rho^2\tilde{x}^k\|^2 - \frac{\rho^2 L}{2}\|\tilde{x}^k - \rho^2\tilde{x}^{k-1}\|^2.
\end{aligned}$$

Divide by L and use $\kappa = \dfrac{L}{m}$, we get

$$\frac{R_k}{L} = -\frac{\rho^2}{2\kappa}\|\tilde{x}^k - \tilde{y}^k\|^2 - \frac{1-\rho^2}{2\kappa}\|\tilde{y}^k\|^2 + \frac{1}{2}\|\tilde{y}^k - \rho^2\tilde{x}^k\|^2 - \frac{\rho^2}{2}\|\tilde{x}^k - \rho^2\tilde{x}^{k-1}\|^2.$$

Next, from

$$\tilde{y}^k = \tilde{x}^k + \beta(\tilde{x}^k - \tilde{x}^{k-1}),$$

we have

$$\tilde{x}^k - \tilde{y}^k = -\beta(\tilde{x}^k - \tilde{x}^{k-1}), \qquad \tilde{x}^{k-1} = \tilde{x}^k - \frac{1}{\beta}(\tilde{y}^k - \tilde{x}^k).$$

Hence

$$\begin{aligned}
\tilde{x}^k - \rho^2\tilde{x}^{k-1} &= \tilde{x}^k - \rho^2\left(\tilde{x}^k - \frac{1}{\beta}(\tilde{y}^k - \tilde{x}^k)\right) \\
&= (1-\rho^2)\tilde{x}^k + \frac{\rho^2}{\beta}(\tilde{y}^k - \tilde{x}^k).
\end{aligned}$$

Now since

$$\rho^2 = 1 - \frac{1}{\sqrt{\kappa}}, \qquad \beta = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}.$$

Then

$$1-\rho^2 = \frac{1}{\sqrt{\kappa}}, \qquad \frac{\rho^2}{\beta} = \frac{1-\frac{1}{\sqrt{\kappa}}}{\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}} = 1 + \frac{1}{\sqrt{\kappa}}.$$

Therefore

$$\tilde{x}^k - \rho^2\tilde{x}^{k-1} = \frac{1}{\sqrt{\kappa}}\tilde{x}^k + \left(1 + \frac{1}{\sqrt{\kappa}}\right)(\tilde{y}^k - \tilde{x}^k).$$

Also,

$$\tilde{y}^k - \rho^2\tilde{x}^k = (\tilde{y}^k - \tilde{x}^k) + (1-\rho^2)\tilde{x}^k = (\tilde{y}^k - \tilde{x}^k) + \frac{1}{\sqrt{\kappa}}\tilde{x}^k.$$

Substituting these into $\frac{R_k}{L}$ gives

$$\begin{aligned}\frac{R_k}{L} &= -\frac{\rho^2}{2\kappa}\|\tilde{x}^k - \tilde{y}^k\|^2 - \frac{1}{2\kappa\sqrt{\kappa}}\|\tilde{y}^k\|^2 \\ &\quad + \frac{1}{2}\left\|(\tilde{y}^k - \tilde{x}^k) + \frac{1}{\sqrt{\kappa}}\tilde{x}^k\right\|^2 \\ &\quad - \frac{\rho^2}{2}\left\|\frac{1}{\sqrt{\kappa}}\tilde{x}^k + \left(1 + \frac{1}{\sqrt{\kappa}}\right)(\tilde{y}^k - \tilde{x}^k)\right\|^2 .\end{aligned}$$

Write

$$d^k := \tilde{y}^k - \tilde{x}^k.$$

Then $\tilde{y}^k = \tilde{x}^k + d^k$, and so

$$\begin{aligned}\frac{R_k}{L} &= -\frac{\rho^2}{2\kappa}\|d^k\|^2 - \frac{1}{2\kappa\sqrt{\kappa}}\|\tilde{x}^k + d^k\|^2 \\ &\quad + \frac{1}{2}\left\|d^k + \frac{1}{\sqrt{\kappa}}\tilde{x}^k\right\|^2 - \frac{\rho^2}{2}\left\|\frac{1}{\sqrt{\kappa}}\tilde{x}^k + \left(1 + \frac{1}{\sqrt{\kappa}}\right)d^k\right\|^2 .\end{aligned}$$

Expanding the squares and collecting the coefficients of

$$\|\tilde{x}^k\|^2, \qquad (\tilde{x}^k)^T d^k, \qquad \|d^k\|^2,$$

one finds that the coefficients of $\|\tilde{x}^k\|^2$ and $(\tilde{x}^k)^T d^k$ cancel exactly, while the coefficient of $\|d^k\|^2$ is

$$-\frac{\rho^2}{2\kappa} - \frac{1}{2\kappa\sqrt{\kappa}} + \frac{1}{2} - \frac{\rho^2}{2}\left(1 + \frac{1}{\sqrt{\kappa}}\right)^2 = -\frac{1}{2}\rho^2\left(\frac{1}{\kappa} + \frac{1}{\sqrt{\kappa}}\right).$$

Hence

$$\begin{aligned}\frac{R_k}{L} &= -\frac{1}{2}\rho^2\left(\frac{1}{\kappa} + \frac{1}{\sqrt{\kappa}}\right)\|d^k\|^2 \\ &= -\frac{1}{2}\rho^2\left(\frac{1}{\kappa} + \frac{1}{\sqrt{\kappa}}\right)\|\tilde{x}^k - \tilde{y}^k\|^2.\end{aligned}$$

Therefore

$$\begin{aligned}R_k &= -\frac{1}{2}L\rho^2\left(\frac{1}{\kappa} + \frac{1}{\sqrt{\kappa}}\right)\|\tilde{x}^k - \tilde{y}^k\|^2 \\ &\le 0\end{aligned}$$

as desired. □

Corollary 4.4. *We have*

$$f(x^k) - f^* \le \left(1 - \frac{1}{\sqrt{\kappa}}\right)^k \left\{f(x_0) - f^* + \frac{m}{2}\|x_0 - x^*\|^2\right\} .$$

Proof. By Theorem 4.2 we have

$$V_{k+1} \le \rho^2 V_k \le \cdots \le (\rho^2)^k V_0 \ .$$

Since $x^0 = x^{-1}$ we have

$$\begin{aligned} f(x^k) - f^* &\le \left(1 - \frac{1}{\sqrt{\kappa}}\right)^k \left[f(x^0) - f^* + \frac{L}{2}\|\tilde{x}^0 - \rho^2 \tilde{x}^{-1}\|^2\right] \\ &= \left(1 - \frac{1}{\sqrt{\kappa}}\right)^k \left[f(x^0) - f^* + \frac{L}{2}(1-\rho^2)^2\|x^0 - x^*\|^2\right] \\ &= \left(1 - \frac{1}{\sqrt{\kappa}}\right)^k \left[f(x^0) - f^* + \frac{L}{2}\left(\sqrt{\frac{m}{L}}\right)^2 \|x^0 - x^*\|^2\right] \\ &= \left(1 - \frac{1}{\sqrt{\kappa}}\right)^k \left[f(x^0) - f^* + \frac{m}{2}\|x^0 - x^*\|^2\right] \ . \end{aligned}$$

where we have used

$$1 - \rho^2 = \frac{1}{\sqrt{\kappa}} = \sqrt{\frac{m}{L}} \ .$$

This is to say we have

$$f(x^k) - f^* \le \left(1 - \frac{1}{\sqrt{\kappa}}\right)^k \left\{f(x_0) - f^* + \frac{m}{2}\|x_0 - x^*\|^2\right\} \ ,$$

where one can verify that $V_0 = f(x_0) - f^* + \frac{m}{2}\|x_0 - x^*\|^2$. □

In summary, for Nesterov's AGD in strongly convex case, the convergence rate is $\sim O\left(1 - \frac{1}{\sqrt{\kappa}}\right)^k$, while for GD it is $\sim O\left(1 - \frac{1}{\kappa}\right)^k$. This is the same as in the quadratic case.

4.2.3 Convergence Proof of Nesterov's method: Convex Case.

Now we consider the case when the objective function $f(x)$ is only (weakly) convex, i.e., $m = 0$. We employ the previous section's Lyapunov's method again.

Set $\alpha_k = \frac{1}{L}$ but in this case our β_k is variable, and thus ρ_k is variable. The Lyapunov function is again

$$V_k = f(x^k) - f^* + \frac{L}{2}\|\widetilde{x}^k - \rho_{k-1}^2 \widetilde{x}^{k-1}\|^2 \ .$$

Lemma 4.5. *We have, for any $k \ge 0$, that*

$$V_{k+1} \le \rho_k^2 V_k + R_k^{(\text{weak})} \ , \tag{4.12}$$

where

$$R_k^{(\text{weak})} = \frac{L}{2}\|\widetilde{y}^k - \rho_k^2 \widetilde{x}^k\|^2 - \frac{\rho_k^2 L}{2}\|\widetilde{x}^k - \rho_{k-1}^2 \widetilde{x}^{k-1}\|^2 \ . \tag{4.13}$$

Proof. One can go through the calculations of Theorem 4.2, with the modification that β_k, ρ_k depend on k, and in particular letting $m = 0$ in (4.10), to get

$$V_{k+1} \leq \rho_k^2 V_k + R_k^{(\text{weak})}$$

where

$$R_k^{(\text{weak})} = -\frac{\rho_k^2 L}{2}\|\tilde{x}^k - \rho_{k-1}^2 \tilde{x}^{k-1}\|^2 + L(\tilde{u}^k)^T(\tilde{y}^k - \rho_k^2 \tilde{x}^k) - \frac{L}{2}\|\tilde{u}^k\|^2 + \frac{L}{2}\|\tilde{x}^{k+1} - \rho_k^2 \tilde{x}^k\|^2 \ ,$$

where $\tilde{u}^k = u^k - u^* = \frac{1}{L}\nabla f(y^k)$, $\tilde{y}^k = \tilde{x}^{k+1} + \tilde{u}^k$. This gives

$$\begin{aligned} L(\tilde{u}^k)^T(\tilde{y}^k - \rho_k^2 \tilde{x}^k) &= L(\tilde{u}^k)^T(\tilde{x}^{k+1} + \tilde{u}^k - \rho_k^2 \tilde{x}^k) \\ &= L\|\tilde{u}^k\|^2 + L(\tilde{u}^k)^T(\tilde{x}^{k+1} - \rho_k^2 \tilde{x}^k) \ . \end{aligned}$$

Thus

$$\begin{aligned} R_k^{(\text{weak})} &= -\frac{\rho_k^2 L}{2}\|\tilde{x}^k - \rho_{k-1}^2 \tilde{x}^{k-1}\|^2 + \frac{L}{2}\|\tilde{u}^k\|^2 + L(\tilde{u}^k)^T(\tilde{x}^{k+1} - \rho_k^2 \tilde{x}^k) + \frac{L}{2}\|\tilde{x}^{k+1} - \rho_k^2 \tilde{x}^k\|^2 \\ &= -\frac{\rho_k^2 L}{2}\|\tilde{x}^k - \rho_{k-1}^2 \tilde{x}^{k-1}\|^2 + \frac{L}{2}\|\tilde{u}^k + \tilde{x}^{k+1} - \rho_k^2 \tilde{x}^k\|^2 \\ &= -\frac{\rho_k^2 L}{2}\|\tilde{x}^k - \rho_{k-1}^2 \tilde{x}^{k-1}\|^2 + \frac{L}{2}\|\tilde{y}^k - \rho_k^2 \tilde{x}^k\|^2 \ , \end{aligned}$$

where in the last step we have used the fact that $\tilde{y}^k = \tilde{x}^{k+1} + \tilde{u}^k$. This gives the Lemma. □

We would like to set

$$R_k^{(\text{weak})} = 0 \ .$$

By (4.13) this is saying

$$\|\tilde{y}^k - \rho_k^2 \tilde{x}^k\|^2 = \rho_k^2 \|\tilde{x}^k - \rho_{k-1}^2 \tilde{x}^{k-1}\|^2 \ ,$$

which is valid as long as we have

$$\tilde{y}^k - \rho_k^2 \tilde{x}^k = \rho_k(\tilde{x}^k - \rho_{k-1}^2 \tilde{x}^{k-1}) \ . \tag{4.14}$$

Since by Nesterov's scheme $y^k = x^k + \beta_k(x^k - x^{k-1})$, and thus $\tilde{y}^k = \tilde{x}^k + \beta_k(\tilde{x}^k - \tilde{x}^{k-1}) = (1+\beta_k)\tilde{x}^k - \beta_k \tilde{x}^{k-1}$. So for (4.14) we just need

$$(1 + \beta_k - \rho_k^2)\tilde{x}^k - \beta_k \tilde{x}^{k-1} = \rho_k \tilde{x}^k - \rho_k \rho_{k-1}^2 \tilde{x}^{k-1} \ . \tag{4.15}$$

Compare the coefficients in (4.15), we set

$$\begin{cases} 1 + \beta_k - \rho_k^2 = \rho_k, \\ \beta_k = \rho_k \rho_{k-1}^2 \ , \end{cases}$$

which means

$$1 + \rho_k \rho_{k-1}^2 - \rho_k^2 = \rho_k \ .$$

In conclusion in order to have $R_k^{\text{weak}} = 0$, we need ρ_k to be a solution to the quadratic recursive equation

$$\rho_k^2 + (1 - \rho_{k-1}^2)\rho_k - 1 = 0 \ , \ \rho_0 = 0 \ , \tag{4.16}$$

which gives us Algorithm 4.4.

Algorithm 4.4 Nesterov's optimal algorithm for general convex function f

1: **Input**: Input initialization $x^{-1} = x^0$, $\beta_0 = \rho_0 = 0$ and Lipschitz constant L for f
2: **for** $k = 0, 1, 2, ..., K-1$ **do**
3: $\quad y^k = x^k + \beta_k(x^k - x^{k-1})$
4: $\quad x^{k+1} = y^k - \frac{1}{L}\nabla f(y^k)$
5: $\quad$ Define $\rho_{k+1} \in [0, 1]$ to be the root of the quadratic equation

$$1 + \rho_{k+1}(\rho_k^2 - 1) - \rho_{k+1}^2 = 0$$

6: $\quad$ Set $\beta_{k+1} = \rho_{k+1}\rho_k^2$
7: **end for**
8: **Output**: Output x^K

Lemma 4.6. *For the ρ_k defined in (4.16) we have, if $\rho_k > 0$, then*

$$1 - \rho_k^2 \leq \frac{2}{k+2} \ .$$

Theorem 4.7 (Convergence of Nesterov's AGD in the weakly convex case [17])**.** *For the weakly convext function $f(x)$, Algorithm 4.4 gives the convergence rate*

$$f(x^k) - f^* \leq \frac{2L}{(k+1)^2}\|x^0 - x^*\|^2 \ .$$

Proof. Equation (4.16) gives us a recursive relation

$$\rho_k^2 = \frac{(1-\rho_k^2)^2}{(1-\rho_{k-1}^2)^2} \ , k = 1, 2, ... \ ,$$

under which $R_k^{(\text{weak})} = 0$. So

$$V_k \leq \rho_{k-1}^2 V_{k-1}$$

and thus

$$V_k \leq \rho_{k-1}^2\rho_{k-2}^2 ... \rho_1^2 V_1 = \frac{(1-\rho_{k-1}^2)^2}{(1-\rho_0^2)^2}V_1 \ .$$

Since $(1-\rho_{k+1}^2)^2 \leq \left(\frac{2}{k+1}\right)^2$ by Lemma 4.6, $\rho_0 = 0$, and apply (4.12) for $k = 0$:

$$V_1 \leq R_0^{(\text{weak})} = \frac{L}{2}\|\tilde{y}^0 - \rho_0^2\tilde{x}^0\|^2 = \frac{L}{2}\|x^0 - x^*\|^2 \ ,$$

we get

$$f(x^k) - f^* + \frac{L}{2}\|\tilde{x}^k - \rho_{k-1}^2 \tilde{x}^{k-1}\|^2 \leq \frac{4}{(k+1)^2}\left(\frac{L}{2}\|x^0 - x^*\|^2\right) ,$$

which implies the statement of this Theorem. □

In summary we have the Convergence rate $\sim O\left(\frac{1}{k^2}\right)$ for Nesterov's method in the (weakly) convex case.

How to understand Nesterov's convergence? It has been discussed till recently and connection with differential equations are revealed in [23]. Suggest choosing $\beta_k = \frac{k-1}{k+2}$. Su-Boyd-Candés differential equation:

$$\ddot{X}_t + \frac{3}{t}\dot{X}_t + \nabla f(X_t) = 0 .$$

4.2.4 Nesterov's method reaches first-order optimality

Nesterov's method achieves first order optimal convergence rate [18]. Indeed, one can construct a carefully designed function for which *no* method that makes use of all gradient observed up to and including iteration k (namely $\nabla f(x^i)$, $i = 0, 1, 2, ..., k$) can produce a sequence $\{x^k\}$ that achieves a rate better than the order $O\left(\frac{1}{k^2}\right)$.

Set $f(x) = \frac{1}{2}x^T Ax - e_1^T x$, where

$$A = \begin{pmatrix} 2 & -1 & 0 & 0 & ... & ... & 0 \\ -1 & 2 & -1 & 0 & ... & ... & 0 \\ ... & ... & ... & ... & ... & ... & ... \\ 0 & 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 2 \end{pmatrix} , \ e_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ ... \\ 0 \end{pmatrix} .$$

Optimization $\min_{x \in \mathbb{R}^n} f(x)$ has a solution x^* such that

$$Ax^* = e_1 . \tag{4.17}$$

Lemma 4.8. *The equation (4.17) can be solved and its components are*

$$x_i^* = 1 - \frac{i}{n+1}$$

for $i = 1, 2, ..., n$.

Proof. We have

$$\begin{cases} 2x_1^* - x_2^* & = & 1 \\ -x_1^* + 2x_2^* - x_3^* & = & 0 \\ ... \\ -x_{j-1}^* + 2x_j^* - x_{j+1}^* & = & 0 \\ ... \\ -x_{n-2}^* + 2x_{n-1}^* - x_n^* & = & 0 \\ -x_{n-1}^* + 2x_n^* & = & 0 \end{cases}$$

The equation $-x^*_{n-1} + 2x^*_n = 0$ gives $x^*_{n-1} = 2x^*_n$. Then for any $j = n-2, ..., 2$ we have $-x^*_{j-1} + 2x^*_j - x^*_{j+1} = 0$. This gives $x^*_{j-1} = 2x^*_j - x^*_{j+1}$. Starting from $j = n-2$, this inductively gives $x^*_{n-2} = 3x^*_n$, ..., $x^*_j = (n-j+1)x^*_n$, ..., $x^*_2 = (n-1)x^*_n$. Then $2x^*_1 - x^*_2 = 1$ which gives

$$2x^*_1 - (n-1)x^*_n = 1 \tag{4.18}$$

Let $S = x^*_1 + ... + x^*_n$ and summing all equations in the system gives $-(S - x^*_n) + 2S - (S - x^*_1) = 1$, i.e.,

$$x^*_n + x^*_1 = 1 \ . \tag{4.19}$$

Equation (4.18) and (4.19) together give $x^*_n = \dfrac{1}{n+1} = 1 - \dfrac{n}{n+1}$ so that $x^*_j = \dfrac{n-j+1}{n+1} = 1 - \dfrac{j}{n+1}$ for $j = 2, ..., n-1$, and $x^*_1 = 1 - \dfrac{1}{n+1}$ as desired. □

Let the starting point $x^0 = (0, ..., 0)^T$, and the iterate

$$x^{k+1} = x^k + \sum_{j=0}^{k} \gamma_j \nabla f(x^j)$$

for some coefficients γ_j, $j = 0, 1, ..., k$. Then

$$x^{k+1} = x^k + \sum_{j=0}^{k} \gamma_j (Ax^j - e_1)$$

so that each iterate x^k can have nonzero entries only in its first k components. Thus

$$\|x^k - x^*\|^2 \geq \sum_{j=k+1}^{n} (x^*_j)^2 = \sum_{j=k+1}^{n} \left(1 - \frac{j}{n+1}\right)^2 \ .$$

Lemma 4.9. *For $k = 1, 2, ..., \dfrac{n}{2} - 1$ we have*

$$\|x^k - x^*\|^2 \geq \frac{1}{8}\|x^0 - x^*\|^2 \ . \tag{4.20}$$

Lemma 4.10. *We have*

$$f(x^k) - f(x^*) \geq \frac{3L}{32(k+1)^2}\|x^0 - x^*\|^2 \ , \ k = 1, 2, ..., \frac{n}{2} - 1 \tag{4.21}$$

where $L = \|A\|_2$.

Proof. We have

$$f(x) - f(x^*) = \frac{1}{2}(x - x^*)^T A(x - x^*) \geq \frac{\lambda_{\min}(A)}{2}\|x - x^*\|^2 \ .$$

Applying this to $x = x^k$, and using the estimate (4.20) we obtain

$$f(x^k) - f(x^*) \geq \frac{\lambda_{\min}(A)}{16}\|x^0 - x^*\|^2.$$

Now the eigenvalues of A are

$$\lambda_j = 2 - 2\cos\frac{j\pi}{n+1}, \qquad j = 1, 2, \ldots, n,$$

so that

$$\lambda_{\min}(A) = 2 - 2\cos\frac{\pi}{n+1}.$$

Using the inequality

$$1 - \cos t \geq \frac{3}{8}t^2, \qquad 0 \leq t \leq \frac{\pi}{2},$$

we get

$$\lambda_{\min}(A) = 2\left(1 - \cos\frac{\pi}{n+1}\right) \geq \frac{3}{4}\frac{\pi^2}{(n+1)^2}.$$

Since $L = \|A\|_2 \leq 4$, it follows that

$$\lambda_{\min}(A) \geq \frac{3L}{4(n+1)^2}.$$

Moreover, for $k \leq \frac{n}{2} - 1$, we have

$$2(k+1) \leq n+1,$$

and hence

$$\frac{1}{(n+1)^2} \geq \frac{1}{4(k+1)^2}.$$

Therefore,

$$\lambda_{\min}(A) \geq \frac{3L}{16(k+1)^2}.$$

Substituting this into the previous estimate yields (4.21). □

4.3 Conjugate gradient method

Conjugate gradient method is a more classical method in computational mathematics. Unlike the Nesterov's accelerated gradient method, the conjugate gradient method does not require knowledge of the convexity parameters L and m to compute the appropriate step–sizes. Unfortunately, it does not admit particularly rigorous analysis when the objective function f is not quadratic.

Originally this method comes from numerical linear algebra: goal is to solve $Qx = p$. But this can be formulated as optimization problem $\min_x f(x)$ with $f(x) = \frac{1}{2}x^T Qx - p^T x + c$.

Algorithm 4.5 Conjugate Gradient Method

Input: Input initialization x^0, $y^0 = -\nabla f(x^0)$, $f(x) = \frac{1}{2}x^T Q x - p^T x + c$, $k = 0, 1, 2, ...$
for $k = 0, 1, 2, ..., K-1$ **do**
  $r^k = Qx^k - p$
  $\alpha_k = \dfrac{\langle y^k, r^k \rangle}{\langle y^k, Qy^k \rangle}$
  $x^{k+1} = x^k - \alpha_k y^k$
  $y^{k+1} = -\nabla f(x^{k+1}) + \beta_k y^k$
  $\beta_{k+1} = \dfrac{\langle r^k, Qy^k \rangle}{\langle y^k, Qy^k \rangle}$
end for
Output: Output x^K

Exact line search in the direction y^k: $\alpha_k = \arg\min_{\alpha > 0} f(x^k + \alpha y^k)$.

Conjugate gradient direction: β_{k+1} is so chosen that $\langle y^{k+1}, Qy^k \rangle = 0$.

4.4 Experiment: GD vs Accelerated GD

In this experiment, we compare the behavior of Gradient Descent v.s. Accelarated Gradient Descent on quadratic and perturbed quadratic functions.

Consider a two-dimensional quadratic function

$$f(x_1, x_2) = \frac{1}{2}Ax_1^2 + \frac{1}{2}Bx_2^2$$

for $A, B > 0$ and a small perturbation of this function

$$g(x_1, x_2) = \frac{1}{2}Ax_1^2 + \frac{1}{2}Bx_2^2 + \varepsilon(x_1^2 + x_2^2)^{3/2}$$

for $\varepsilon > 0$. Use Gradient Descent, Polyak's heavy ball method as well as Nesterov's Acclerated Gradient Descent to optimize f and g.

We experiment the following

(1) different/random initializations;

(2) plot the trajectory as well as show its dynamics via an animation;

(3) plot the evolution of the error function as a function of number of iterations;

(4) compare on different parmeters $A, B, \varepsilon > 0$;

We perform the same experiments for the non-convex function

$$h(x_1, x_2) = \frac{1}{2}Ax_1^2 - \frac{1}{2}Bx_2^2 \ .$$

As an example, optimizing f using $A = 1, B = 100$ has convergence shown as follows:

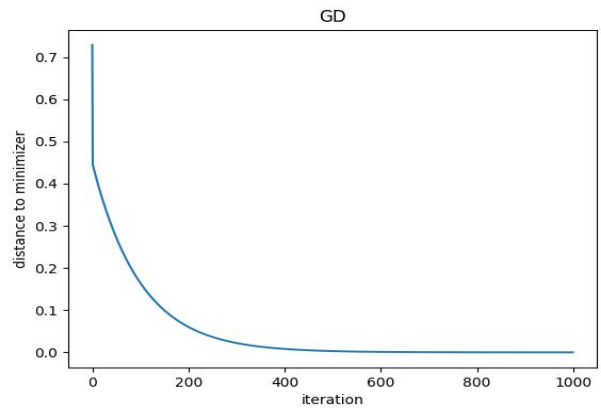

Figure 4.2: Convergence of GD: $A = 1, B = 100, \alpha = 0.01, \beta = 1, \varepsilon = 0.1$

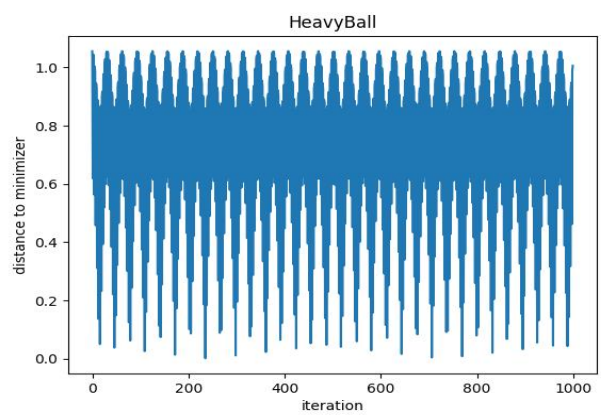

Figure 4.3: Convergence of HeavyBall: $A = 1, B = 100, \alpha = 0.01, \beta = 1, \varepsilon = 0.1$

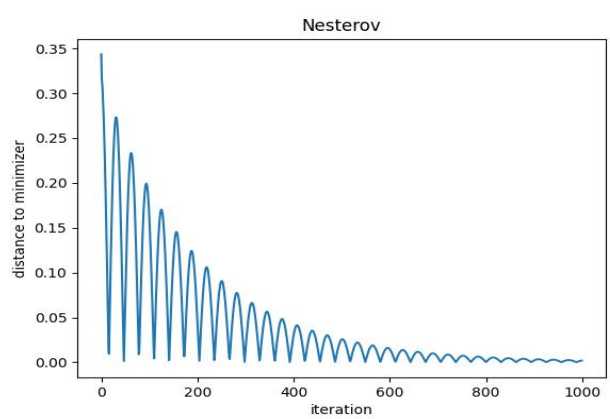

Figure 4.4: Convergence of Nesterov: $A = 1, B = 100, \alpha = 0.01, \beta = 1, \epsilon = 0.1$

Chapter 5

Stochastic Gradient Methods

5.1 SGD

Stochastic Gradient Descent (SGD) is a stochastic analogue of the Gradient Descent algorithm, aiming at finding the local or global minimizers of the function expectation parameterized by some random variable. Schematically, the algorithm can be interpreted as targeting at finding a local minimum point of the expectation of function

$$f(x) = \mathbf{E} f(x;\zeta) \ , \tag{5.1}$$

where the index random variable ζ follows some prescribed distribution $\mathcal{D}$, and the weight vector $x \in \mathbb{R}^d$. SGD updates via the iteration

$$x^{k+1} = x^k - \alpha_k g(x^k;\zeta_k) \ , \tag{5.2}$$

where the noisy gradient $g(x;\zeta) = \nabla f(x;\zeta)$, $\alpha_k > 0$ is the learning rate, and ζ_k are i.i.d. random variables that have the same distribution as $\zeta \sim \mathcal{D}$.

In particular, in the case of supervised learning, assume the input training data is given by $\{(a_i, b_i)\}_{i=1}^n$ and the fitting function is $g(a) = b$. Then the loss function is $\ell_{(a,b)}(x) = L(g(a), b; x)$ where x is the model weight. Let $f_i(x) = \ell_{(a_i,b_i)}(x)$. Then the optimization problem has the objective function

$$f(x) = \frac{1}{n}\sum_{i=1}^n f_i(x) \ .$$

In this case, if we run GD or AGD for f directly, we have

$$\begin{aligned} &\text{GD:} && x^{k+1} = x^k - \eta \nabla f\big(x^k\big) \ , \\ &\text{Nesterov AGD:} && x^{k+1} = x^k - \alpha \nabla f\big(x^k + \beta\big(x^k - x^{k-1}\big)\big) + \beta\big(x^k - x^{k-1}\big) \ . \end{aligned}$$

This means at each iteration step we have to compute $\nabla f(x) = \dfrac{1}{n}\displaystyle\sum_{i=1}^n \nabla f_i(x)$. Usually n is large therefore gradient queries of ∇f_i is large for each iteration step: if we run T the iterations, the total computational cost is $O(T \times n)$. Assume $T = 1000$, $n = 1000$, then $T \times n = 10^6 \gg 1$.

To mitigate this, we use SGD where the random variable ζ samples size m ($m \leq n$) mini–batches $\mathcal{B}$ uniformly from an index set $\{1, 2, ..., n\}$: $\mathcal{B} \subset \{1, 2, ..., n\}$ and $|\mathcal{B}| = m$ (batchsize). In this case, given loss functions $f_1(x), ..., f_n(x)$ on training data, we have

$$f(x;\zeta) = \frac{1}{m}\sum_{i\in\mathcal{B}} f_i(x) \text{ and } f(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x) \ .$$

Algorithm 5.1 Stochastic Gradient Descent

1: **Input**: Input initialization x^0, index random variable $\zeta \sim \mathcal{D}$, noisy gradient $g(x;\zeta) = \nabla f(x;\zeta)$ such that $\mathbf{E}g(x;\zeta) = \nabla f(x)$, learning rate $\alpha_k > 0$, $k = 0, 1, 2, ...$
2: **for** $k = 0, 1, 2, ..., K-1$ **do**
3: Sample the i.i.d. random variable $\zeta_k \sim \mathcal{D}$
4: Calculate the noisy gradient $g(x^k;\zeta_k) = \nabla f(x^k;\zeta_k)$
5: $x^{k+1} = x^k - \alpha_k g(x^k;\zeta_k)$
6: **end for**
7: **Output**: Output x^K

Example 5.1. Assume $m = 1$, then $\mathcal{B} = \{i_k\}$ where i_k is drawn uniformly randomly from $\{1, ..., n\}$ independent of k. Then we compare

$$\begin{aligned} \text{GD}: &\quad x^{k+1} = x^k - \frac{\eta}{n}\sum_{i=1}^{n}\nabla f_i(x^k) \ , \quad \text{complexity } O(T\times n) \ , \\ \text{SGD}: &\quad x^{k+1} = x^k - \eta\nabla f_{i_k}(x^k) \ , \quad \text{complexity } O(T) \ . \end{aligned}$$

Only one query of ∇f_i at each iteration.

Lemma 5.1. *We have, for any $f_i(x)$, $i = 1, 2, ..., n$, that*

$$\mathbf{E}\left(\frac{1}{m}\sum_{i\in\mathcal{B}} f_i(x)\right) = \frac{1}{n}\sum_{i=1}^{n} f_i(x) \ .$$

Proof. B_k are a sequence of i.i.d. mini-batches: $B_k \subset \{1, \ldots, n\}$, $|B_k| = m$. Since B_k is taken uniformly randomly among all size m-subsets of $\{1, \ldots, n\}$, the total number of size-m subsets is

$$\binom{n}{m} = \frac{n!}{m!(n-m)!} \ .$$

When we enumerate all $\binom{n}{m}$ mini-batches $\mathcal{B}$, each index $i \in \{1, ..., n\}$ will be repeated $\binom{n-1}{m-1}$ times. This gives

$$\begin{aligned} \mathbf{E}\left(\frac{1}{m}\sum_{i\in B_k} f_i(x)\right) &= \frac{\frac{1}{m}\sum_{i=1}^{n}\binom{n-1}{m-1} f_i(x)}{\binom{n}{m}} \\ &= \frac{1}{m}\cdot\frac{(n-1)!}{(m-1)!(n-m)!}\cdot\frac{1}{\binom{n}{m}}\sum_{i=1}^{n} f_i(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x) \ . \end{aligned}$$

□

Let us analyze the convergence properties of SGD. To measure the rate of convergence, our metrics are set as follows:

- f is strongly convex: the metric is $\mathbf{E}\left[\|x^k - x^*\|^2\right]$.
- f is convex but not necessarily strongly convex: the metric is $f(x^k) - f(x^*)$.

We make the following assumption regarding the random gradient surrogate $g(x;\zeta)$ used in (5.2):

Assumption 5.1. *We assume that for some $L_g > 0$ and $B > 0$ we have*

$$\mathbf{E}_\zeta\left[\|g(x;\zeta)\|_2^2\right] \leq L_g^2\|x - x^*\|^2 + B^2 \tag{5.3}$$

for all x.

Remark 5.2. *In the mini-batch stochastic gradient case, $g(x;\zeta) = \frac{1}{m}\sum_{i\in\mathcal{B}} \nabla f_i(x)$. Then we have*

$$\begin{aligned}
& \mathbf{E}_\zeta\|g(x;\zeta)\|_2^2 \\
= \;& \mathbf{E}\left\|\frac{1}{m}\sum_{i\in\mathcal{B}} \nabla f_i(x)\right\|^2 \\
= \;& \mathbf{E}\left\|\frac{1}{m}\sum_{i\in\mathcal{B}} (\nabla f_i(x) - \nabla f_i(x^*)) + \frac{1}{m}\sum_{i\in\mathcal{B}} \nabla f_i(x^*)\right\|^2 \\
\leq \;& 2\mathbf{E}\left\|\frac{1}{m}\sum_{i\in\mathcal{B}} (\nabla f_i(x) - \nabla f_i(x^*))\right\|^2 + 2\mathbf{E}\left\|\frac{1}{m}\sum_{i\in\mathcal{B}} \nabla f_i(x^*)\right\|^2 \\
\leq \;& 2\mathbf{E}\sum_{i\in\mathcal{B}} \frac{1}{m^2}\sum_{i\in\mathcal{B}} \|\nabla f_i(x) - \nabla f_i(x^*)\|^2 + B^2 \\
\leq \;& 2\mathbf{E}\frac{1}{m}\sum_{i\in\mathcal{B}} \|\nabla f_i(x) - \nabla f_i(x^*)\|^2 + B^2 \\
\leq \;& 2\mathbf{E}\|\nabla f_i(x) - \nabla f_i(x^*)\|^2 + B^2 \;.
\end{aligned}$$

So we can pick $L_g = \sqrt{2}L_{\nabla f}$. If $L_g = 0$ in (5.3), then f cannot be m-strongly convex over an unbounded domain, since by Jensen's inequality

$$\|\nabla f(x)\|^2 = \|\mathbf{E}g(x;\zeta)\|^2 \leq \mathbf{E}\left[\|g(x;\zeta)\|^2\right] \overset{\text{since } L_g=0}{\leq} B^2$$

but

$$\begin{aligned}
& \|x - x^*\| \cdot \|\nabla f(x)\| \\
\geq \;& \langle x - x^*, \nabla f(x)\rangle \\
= \;& \langle x - x^*, \nabla f(x) - \nabla f(x^*)\rangle \\
= \;& \left\langle x - x^*, \int_0^1 \nabla^2 f(x^* + \gamma(x - x^*))(x - x^*)d\gamma \right\rangle \geq m\|x - x^*\|^2
\end{aligned}$$

for all x. This means $\|\nabla f(x)\|^2 \geq m^2\|x - x^*\|^2$, which further implies $B \geq m\|x - x^*\|$, a contradiction.

Theorem 5.3. *Assume $B > 0$ and $L_g > 0$, and f is m-strongly convex with unique minimizer x^*. We choose $\alpha \in \left(0, \frac{2m}{L_g^2}\right)$ such that $1 - 2m\alpha + \alpha^2 L_g^2 > 0$. For the iteration (5.2) we set $\alpha_k = \alpha$. Then*

$$\mathbf{E}\|x^k - x^*\|^2 \le \left(1 - 2m\alpha + \alpha^2 L_g^2\right)^k \|x^0 - x^*\|^2 + \frac{\alpha B^2}{2m - \alpha L_g^2} \ . \tag{5.4}$$

Proof. We perform a square expansion for the stochastic iteration (5.2) to get

$$\begin{aligned} \|x^{k+1} - x^*\|^2 &= \|x^k - \alpha_k g(x^k;\zeta_k) - x^*\|^2 \\ &= \|x^k - x^*\|^2 - 2\alpha_k \langle g(x^k;\zeta_k), x^k - x^*\rangle + \alpha_k^2 \|g(x^k;\zeta_k)\|^2 \ . \end{aligned} \tag{5.5}$$

So

$$\mathbf{E}\,\|x^{k+1} - x^*\|^2 = \mathbf{E}\,\|x^k - x^*\|^2 - 2\alpha_k\,\mathbf{E}\left\langle g(x^k;\zeta_k),\, x^k - x^*\right\rangle + \alpha_k^2\,\mathbf{E}\,\|g(x^k;\zeta_k)\|^2.$$

We have

$$\begin{aligned} & \mathbf{E}\left[\langle g(x^k;\zeta_k),\, x^k - x^*\rangle\right] \\ = \ & \mathbf{E}\Big[\mathbf{E}_{\zeta_k}\Big[\left\langle g(x^k;\zeta_k),\, x^k - x^*\right\rangle \Big| \, \zeta_0, \ldots, \zeta_{k-1}\Big]\Big] \qquad (\text{“telescoping”}) \\ = \ & \mathbf{E}\Big[\Big\langle \mathbf{E}_{\zeta_k}\Big[g(x^k;\zeta_k) \,|\, \zeta_0, \ldots, \zeta_{k-1}\Big],\, x^k - x^*\Big\rangle\Big] \\ = \ & \mathbf{E}\left[\langle \nabla f(x^k),\, x^k - x^*\rangle\right] \ . \end{aligned}$$

Here in the last step we have used the fact that $\mathbf{E}_\zeta\, g(x;\zeta) = \nabla f(x)$. Similarly

$$\mathbf{E}\,\|g(x^k;\zeta_k)\|^2 = \mathbf{E}\Big[\mathbf{E}_{\zeta_k}\Big[\|g(x^k;\zeta_k)\|_2^2 \Big| \, \zeta_0, \ldots, \zeta_{k-1}\Big]\Big] \le \mathbf{E}\Big[L_g^2\|x^k - x^*\|^2 + B^2\Big] \, .$$

Set $a_k = \mathbf{E}\left[\|x^k - x^*\|^2\right]$, then from the above and (5.5) we get

$$a_{k+1} \le \left(1 + \alpha_k^2 L_g^2\right) a_k - 2\alpha_k \mathbf{E}\left[\langle \nabla f(x^k), x^k - x^*\rangle\right] + \alpha_k^2 B^2 \ . \tag{5.6}$$

If $B > 0$, $L_g > 0$ and f has strong-convexity constant $m > 0$, then $\nabla^2 f(x) \succeq mI$ and the unique minimizer x^* satisfies $\nabla f(x^*) = 0$. Since

$$\langle \nabla f(x) - \nabla f(x^*), x - x^*\rangle = \int_0^1 \langle \nabla^2 f\big(x^* + \gamma(x - x^*)\big)(x - x^*), x - x^*\rangle \, d\gamma \ge m\|x - x^*\|^2 \ ,$$

we have $\left\langle \nabla f(x),\, x - x^*\right\rangle \ge m\|x - x^*\|^2$. Putting $x = x^k$ and taking $\mathbf{E}$-expectation we get $\mathbf{E}\big[\langle \nabla f(x^k),\, x^k - x^*\rangle\big] \ge m\,\mathbf{E}\|x^k - x^*\|^2$. This together with (5.6) gives

$$a_{k+1} \le (1 - 2m\alpha_k + \alpha_k^2 L_g^2) a_k + \alpha_k^2 B^2 \ .$$

Set $\alpha_k = \alpha$ and $\mu = 1 - 2m\alpha + \alpha^2 L_g^2$, then the above gives

$$a_{k+1} \le \mu^{k+1} a_0 + \alpha^2 B^2 (1 + \mu + \ldots + \mu^k) = \mu^{k+1} a_0 + \frac{\alpha^2 B^2 (1 - \mu^k)}{1 - \mu} \ .$$

When $\mu \in (0, 1)$ this gives (5.4). □

We see from (5.4) that there is a "threshold value" $\dfrac{\alpha B^2}{2m - \alpha L_g^2}$ which prevents SGD to converge (i.e. there is always a fluctuation). To over comes this, "epoch doubling" and "hyper-parameter tuning" have been used.

Remark 5.4. *Some other cases can also be discussed similarly:*

- $L_g = 0$. *Let the average output* $\bar{x}^K = \dfrac{\sum_{j=0}^{K} \alpha_j x^j}{\sum_{j=0}^{K} \alpha_j}$. *Set* $D_0 = \|x^0 - x^*\|^2$, $\alpha_k = \alpha > 0$ *and* $\alpha_{opt} = \dfrac{D_0}{B\sqrt{T+1}}$, $\theta = \dfrac{\alpha}{\alpha_{opt}}$. *If* $L_g = 0$, *then*
$$\mathbf{E}\left[f(\bar{x}^T) - f(x^*)\right] \leq \left(\frac{1}{2}\theta + \frac{1}{2}\theta^{-1}\right)\frac{BD_0}{\sqrt{T+1}} \ .$$
- $B = 0$. *Assume that* $\langle \nabla f(x), x - x^* \rangle \geq m\|x - x^*\|^2$ *for some* $m > 0$. *Then if* $\alpha_k = \alpha \in \left(0, \dfrac{2m}{L_g^2}\right)$, *then we obtain a linear rate of convergence. Optimal* $\alpha = \dfrac{2}{L_g^2}$ *and*
$$a_k \leq \left(1 - \frac{m^2}{L_g^2}\right)^k D_0^2 < \varepsilon \Longleftrightarrow T \geq \left\lceil \frac{L_g^2}{m^2} \ln\left(\frac{D_0^2}{\varepsilon}\right)\right\rceil \ .$$

5.2 Stochastic Variance-Reduced Gradient Descent (SVRG) and its variants

5.2.1 SVRG and its Convergence Proof

Variance Reduction Methods: Goal is to reduce the variance produced by the stochastic terms in the iteration scheme.

SVRG: Johnson, R., Zhang, T., Accelerating Stochastic Gradient using Predictive Variance Reduction, *NIPS*, 2013.

Goal is to minimize $\min P(\omega)$ where $P(\omega) = \dfrac{1}{n}\sum_{i=1}^{n} \psi_i(\omega)$.

SGD: draw i_t randomly from $\{1, ..., n\}$ and set
$$\omega^{(t)} = \omega^{(t-1)} - \eta_t \nabla \psi_{i_t}(\omega^{(t-1)}) \ .$$

How to reduce variance? We can keep a snapshot $\widetilde{\omega}$ after every m SGD iterations, and we maintain the average gradient
$$\widetilde{\mu} = \nabla P(\widetilde{\omega}) = \frac{1}{n}\sum_{i=1}^{n} \nabla \psi_i(\widetilde{\omega}) \ .$$

Variance–reduction:
$$\omega^{(t)} = \omega^{(t-1)} - \eta_t \left(\nabla \psi_{i_t}(\omega^{(t-1)}) - \nabla \psi_{i_t}(\widetilde{\omega}) + \widetilde{\mu}\right) \ .$$

Algorithm 5.2 Stochastic Variance Reduced Gradient

Input: Input initialization $\widetilde{\omega}_0$, update frequency m and learning rate η
for $s = 1, 2, ...$ **do**
  $\widetilde{\omega} = \widetilde{\omega}_{s-1}$
  $\widetilde{\mu} = \frac{1}{n}\sum_{i=1}^{n} \nabla\psi_i(\widetilde{\omega})$
  $\omega_0 = \widetilde{\omega}$
  for $t = 1, 2, ..., m$ **do**
    Sample i_t independently and uniformly from $\{1, ..., n\}$
    Update the weight $\omega_t = \omega_{t-1} - \eta\left(\nabla\psi_{i_t}(\omega_{t-1}) - \nabla\psi_{i_t}(\widetilde{\omega}) + \widetilde{\mu}\right)$
  end for
  $\widetilde{\omega}_s \leftarrow \omega_t$ for randomly chosen $t \in \{0, ..., m-1\}$
end for
Output: $\widetilde{\omega}_s$

Convergence analysis of SVRG. Follow the original SVRG paper.

Theorem 5.5. *Consider the SVRG algorithm 5.2 and assume that each $\nabla\psi_i$ is L–Lipschitz and convex, and the function $P(\omega) = \frac{1}{n}\sum_{i=1}^{n}\psi_i(\omega)$ is γ–strongly convex. Set $\omega_* = \arg\min_{\omega} P(\omega)$ and assume that the epoch length m is so large that*

$$\alpha = \frac{1}{\gamma\eta(1-2L\eta)m} + \frac{2L\eta}{1-2L\eta} < 1 \ ,$$

then we have the linear convergence in expectation of SVRG algorithm 5.2, i.e.

$$\mathbf{E}[P(\widetilde{\omega}_s) - P(\omega_*)] \leq \alpha^s[P(\widetilde{\omega}_0) - P(\omega_*)] \ .$$

5.2.2 SAG, SAGA, SARAH, SPIDER, STORM

SAG: Roux, Nicolas L., Mark Schmidt, Francis R. Bach, A stochastic gradient method with an exponential convergence rate for finite training sets. *NIPS*, 2012.

Guaranteed linear convergence rate for strongly convex objective $P(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w)$.

Algorithm 5.3 Stochastic Average Gradient: minimizing $P(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(x)$

Input: Input initialization x_0, stepsize $\alpha > 0$, $d = 0$, $y_i = 0$, $i = 1, 2, ..., n$
for $k = 0, 1, ..., K-1$ **do**
  Sample i uniformly randomly from $\{1, 2, ..., n\}$
  $d \leftarrow d - y_i + \nabla f_i(x)$
  $y_i = \nabla f_i(x)$
  $x \leftarrow x - \frac{\alpha}{n} d$
end for
Output: x_K

SAGA: Defazio, A., Bach, F., Lacoste-Julien, S., SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives, *NIPS*, 2014.

Guaranteed linear convergence rate for strongly convex objective $P(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w)$.

Algorithm 5.4 SAGA: minimizing $P(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(x)$

Input: Input initial vector $x^0 \in \mathbb{R}^d$ and known gradients $\nabla f_i(\phi_i^0) \in \mathbb{R}^d$ with $\phi_i^0 = x^0$ for each i. These derivatives are stored in an $n \times d$ matrix.

for $k = 0, 1, ..., K-1$ **do**

  Sample j uniformly randomly from $\{1, 2, ..., n\}$

  Take $\phi_j^{k+1} = x^k$ and store $\nabla f_j(\phi_j^{k+1})$ in the table. All other entries in the table remain unchanged. The quantity ϕ_j^{k+1} is not explicitly stored.

  Update x using $\nabla f_j(\phi_j^{k+1})$, $\nabla f_j(\phi_j^k)$ and the table average:

$$w^{k+1} = x^k - \gamma\left[\nabla f_j(\phi_j^{k+1}) - \nabla f_j(\phi_j^k) + \frac{1}{n}\sum_{i=1}^{n}\nabla f_j(\phi_i^k)\right] ,$$

$$x^{k+1} = \mathrm{prox}_\gamma^h(w^{k+1}) = \arg\min_{x\in\mathbb{R}^d}\left\{h(x) + \frac{1}{2\gamma}\|x - w^{k+1}\|^2\right\} .$$

end for

Output: x^K

SARAH: Lam M. Nguyen, Jie Liu, Katya Scheinberg, Martin Takáč, SARAH: A Novel Method for Machine Learning Problems Using Stochastic Recursive Gradient, *Proceedings of the 34th International Conference on Machine Learning, PMLR* **70**:2613-2621, 2017.

Guaranteed linear convergence rate for strongly convex objective $P(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w)$.

Algorithm 5.5 StochAstic Recursive grAdient algoritHm: minimizing $P(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w)$

Input: Initialization $\widetilde{w}_0$, the learning rate $\eta > 0$ and the inner loop size m
for $s = 1, 2, ...$ **do**
  $w_0 = \widetilde{w}_{s-1}$
  $v_0 = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(w_0)$
  $w_1 = w_0 - \eta v_0$
  for $t = 1, ..., m-1$ **do**
    Sample i_t uniformly randomly in $\{1, 2, ..., n\}$
    $v_t = \nabla f_{i_t}(w_t) - \nabla f_{i_t}(w_{t-1}) + v_{t-1}$
    $w_{t+1} = w_t - \eta v_t$
  end for
  Set $\widetilde{w}_s = w_t$ with t chosen uniformly randomly from $\{0, 1, ..., m-1\}$
end for
Output: $\widetilde{w}_s$

SPIDER: Cong Fang, Chris Junchi Li, Zhouchen Lin, Tong Zhang, SPIDER: Near-Optimal Non-Convex Optimization via Stochastic Path-Integrated Differential Estimator, *NIPS*, 2018.

No assumption on convexity. Optimal Rate $O\left(\min(n^{1/2}\varepsilon^{-2}, \varepsilon^{-3})\right)$ for finding an ε–approximate first order stationary point.

Algorithm 5.6 Stochastic Path–Integrated Differential EstiamtoR: Searching First Order stationary point

Input: Input $\boldsymbol{x}^0$, q, S_1, S_2, n_0, ε and $\widetilde{\varepsilon}$
for $k = 0, 1, ..., K-1$ **do**
  if $\mathrm{mod}(k, q) = 0$ **then**
    Draw S_1 samples (or compute the full gradient in the finite sum case), let $\boldsymbol{v}^k = \nabla f_{\mathcal{S}_1}(\boldsymbol{x}^k)$
  else
    Draw S_2 samples, and let $\boldsymbol{v}^k = \nabla f_{\mathcal{S}_2}(\boldsymbol{x}^k) - \nabla f_{\mathcal{S}_2}(\boldsymbol{x}^{k-1}) + \boldsymbol{v}^{k-1}$
  end if
  —**Option I**—
  if $\|\boldsymbol{v}^k\| \leq 2\widetilde{\varepsilon}$ **then**
    Return $\boldsymbol{x}^k$
  else
    $\eta = \dfrac{\varepsilon}{Ln_0}$
    $\boldsymbol{x}^{k+1} = \boldsymbol{x}^k - \eta \dfrac{\boldsymbol{v}^k}{\|\boldsymbol{v}^k\|}$
  end if
  —**Option II**—
  $\eta^k = \min\left(\dfrac{\varepsilon}{Ln_0\|\boldsymbol{v}^k\|}, \dfrac{1}{2Ln_0}\right)$
  $\boldsymbol{x}^{k+1} = \boldsymbol{x}^k - \eta^k \boldsymbol{v}^k$
end for
Output: Output $\widetilde{\boldsymbol{x}}$ uniformly at random from $\{\boldsymbol{x}^k\}_{k=0}^{K-1}$

Remark 5.6. *In the classical SGD algorithm, the minibatches $\mathcal{B}$ are sampled uniformly randomly from the index set $\{1, 2, ..., n\}$ with fixed minibatch size $m \leq n$. This kind of minibatches can be regarded as* "sample without replacement" *minibatches. This is the commonly understood minibatches in practice.*

In the SPIDER algorithm, the samples $\mathcal{S}_1$ and $\mathcal{S}_2$ are drawn with replacement, so that they form minibatches $\mathcal{S}$ that are sampled uniformly randomly from the index set $\{1, 2, ..., n\}$ but with replacement, and the minibatch has a given size, say m but counting multiplicities. This kind of minibacthes can be regarded as "sample with replacement" *minibacthes. This is less seen in the literature.*

The reason why "sample with replacement" minibatches are used in the SPIDER algorithm is because of the following fact: consider estimating the variance of the stochastic estimator $\frac{1}{m}\sum_{i\in\mathcal{S}} \nabla f_i(x)$ where $\mathcal{S}$ is the "sample with replacement" minibatch with size m, then

$$\mathbf{E}\left\|\frac{1}{m}\sum_{i\in\mathcal{S}} \nabla f_i(x) - \nabla f(x)\right\|^2 = \mathbf{E}\left\|\frac{1}{m}\sum_{i\in\mathcal{S}} (\nabla f_i(x) - \nabla f(x))\right\|^2 = \frac{1}{m}\mathbf{E}\|\nabla f_i(x) - \nabla f(x)\|^2 \ ,$$

where in the last expectation the variable i is taken uniformly randomly from $\{1, 2, ..., n\}$.

On the contrary, if $\mathcal{B}$ is the "sample without replacement" minibatch with the same size m, then instead we have

$$\begin{aligned}\mathbf{E}\left\|\frac{1}{m}\sum_{i\in\mathcal{B}}\nabla f_i(x)-\nabla f(x)\right\|^2 &= \mathbf{E}\left\|\frac{1}{m}\sum_{i\in\mathcal{B}}(\nabla f_i(x)-\nabla f(x))\right\|^2 \\ &\le \mathbf{E}\sum_{i\in\mathcal{B}}\frac{1}{m^2}\sum_{i\in\mathcal{B}}\|\nabla f_i(x)-\nabla f(x)\|^2 = \mathbf{E}\|\nabla f_i(x)-\nabla f(x)\|^2 \ .\end{aligned}$$

So we see a factor of $1/m$ is lost.

Previous mentioned methods typically use *non-adaptive learning rates* and *reliance on giant batch sizes* to construct variance reduced gradients through the use of low-noise gradients calculated at a "checkpoint".

STOchastic Recursive Momentum (STORM, Cutkosky, A., Orabona, F., Momentum-Based Variance Reduction in Non-Convex SGD, arXiv:1905.10018v2, 2020.) is a recently proposed novel variance-reduction method that achieves variance reduction through the use of a variant of the momentum term.

This method never needs to compute a checkpoint gradient and achieves the optimal convergence rate of $O(1/T^{1/3})$. In fact, the key ideas in momentum-based methods is to replace the gradient estimator with an exponential moving avearge of the gradient estimators in all previous iteration steps, i.e.

$$\begin{aligned}\boldsymbol{d}_t &= (1-a)\boldsymbol{d}_{t-1} + a\nabla f(\boldsymbol{x}_t,\xi_t) \ , \\ \boldsymbol{x}_t &= \boldsymbol{x}_{t-1} - \eta\boldsymbol{d}_t \ ,\end{aligned}$$

where $\nabla f(x,\xi)$ is the usual stochastic gradient estimator. The STORM estimator incorporates ideas in SARAH estimator to modify the momentum updates as

$$\begin{aligned}\boldsymbol{d}_t &= (1-a)\boldsymbol{d}_{t-1} + a\nabla f(\boldsymbol{x}_t,\xi_t) + (1-a)(\nabla f(\boldsymbol{x}_t,\xi_t) - \nabla f(\boldsymbol{x}_{t-1},\xi_t)) \ , \\ \boldsymbol{x}_t &= \boldsymbol{x}_{t-1} - \eta\boldsymbol{d}_t \ .\end{aligned}$$

We can think of the momentum iteration as an "exponential moving average" version of SARAH, that is

$$\boldsymbol{d}_t = (1-a)\left[\boldsymbol{d}_{t-1} + (\nabla f(\boldsymbol{x}_t,\xi_t) - \nabla f(\boldsymbol{x}_{t-1},\xi_t))\right] + a\nabla f(\boldsymbol{x}_t,\xi_t) \ .$$

Here the first part is like SARAH update, but it is weighted with the stochastic gradient update. Moreover, different from the SARAH update, the STORM update does not have to compute checkpoint gradients. Rather, variance-reduction is achieved by tuning the weight parameter a appropriately. The exact algorithm goes as follows:

Algorithm 5.7 STORM: STOchastic Recursive Momentum

Input: Parameters k, w, c, initial point $\boldsymbol{x}_1$
Sample ξ_1
$G_1 \leftarrow \|\nabla f(\boldsymbol{x}_1, \xi_1)\|$
$\boldsymbol{d}_1 \leftarrow \nabla f(\boldsymbol{x}_1, \xi_1)$
$\eta_0 \leftarrow k/w^{1/3}$
for $t = 1, 2, ..., T$ **do**
  $\eta_t \leftarrow \dfrac{k}{(w + \sum_{i=1}^{t} G_i^2)^{1/3}}$
  $\boldsymbol{x}_{t+1} \leftarrow \boldsymbol{x}_t - \eta_t \boldsymbol{d}_t$
  $a_{t+1} \leftarrow c\eta_t^2$
  Sample ξ_{t+1}
  $G_{t+1} \leftarrow \|\nabla f(\boldsymbol{x}_{t+1}, \xi_{t+1})\|$
  $\boldsymbol{d}_{t+1} \leftarrow (1 - a_{t+1})\boldsymbol{d}_t + a_{t+1}\nabla f(\boldsymbol{x}_{t+1}, \xi_{t+1}) + (1 - a_{t+1})[\nabla f(\boldsymbol{x}_{t+1}, \xi_{t+1}) - \nabla f(\boldsymbol{x}_t, \xi_{t+1})]$
end for
Output: $\widehat{\boldsymbol{x}}$ sampled uniformly randomly from $\boldsymbol{x}_1, ..., \boldsymbol{x}_T$ (In practice, set $\widehat{\boldsymbol{x}} = \boldsymbol{x}_T$)

5.3 Escape from saddle points and nonconvex problems, SGLD, Perturbed GD

Continuous point of view:

Hu, W., Li, C.J., Li, L., Liu, J., On the diffusion approximation of nonconvex stochastic gradient descent. *Annals of Mathematical Science and Applications*, Vol. **4**, No. 1 (2019), pp. 3-32.

SGLD: Maxim Raginsky, Alexander Rakhlin, Matus Telgarsky, Non-convex learning via Stochastic Gradient Langevin Dynamics: a nonasymptotic analysis, *Proceedings of the 2017 Conference on Learning Theory, PMLR* **65**:1674-1703, 2017.

SGLD is a continuous version of SGD with additional moderate and isotropic noise.

Convergence Guarantee: Xu, P., Chen, J., Zou, D., Gu, Q., Global convergence of Langevin Dynamics Based Algorithms for Nonconvex Optimization, *NIPS*, 2018.

Algorithm 5.8 Stochastic Gradient Langevin Dynamics: minimizing $P(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w)$

Input: Input initialization $w_0 \in \mathbb{R}^d$, stepsize $\alpha > 0$, inverse temperature $\beta > 0$
for $k = 0, 1, ..., K-1$ **do**
  Sample a random minibatch $B \subset \{1, 2, ..., n\}$
  Calculate $\nabla f_i(w_k)$ for $i \in B$
  Sample i.i.d. $\varepsilon_k \sim \mathcal{N}(0, I_d)$
  $w_{k+1} = w_k - \frac{\eta}{|B|}\sum_{i\in B} \nabla f_i(w_k) + \sqrt{2\eta\beta^{-1}}\varepsilon_k$
end for
Output: w_K

Perturbed GD: Chi Jin, Rong Ge, Praneeth Netrapalli, Sham M. Kakade, Michael I. Jordan, How to Escape Saddle Points Efficiently, *ICML* 2017.

Perturbed GD has guaranteed escape rate from saddle points.

Algorithm 5.9 Perturbed Gradient Descent: PGD$(\boldsymbol{x}_0, \ell, \rho, \varepsilon, c, \delta, \Delta_f)$

Input: Input Initialization $\boldsymbol{x}_0$.
Parameters $\chi \leftarrow 3\max\left\{\ln\left(\frac{d\ell\Delta_f}{c\varepsilon^2\delta}\right), 4\right\}$, $\eta \leftarrow \frac{c}{\ell}$, $r \leftarrow \frac{\sqrt{c}}{\chi^2}\cdot\frac{\varepsilon}{\ell}$, $g_{\text{thres}} \leftarrow \frac{\sqrt{c}}{\chi^2}\cdot\varepsilon$, $f_{\text{thres}} \leftarrow \frac{c}{\chi^3}\cdot\sqrt{\frac{c^3}{\rho}}$, $t_{\text{thres}} \leftarrow \frac{\chi}{c^2}\cdot\frac{\ell}{\sqrt{\rho\varepsilon}}$, $t_{\text{noise}} \leftarrow -t_{\text{thres}} - 1$.
for $t = 0, 1, ..., K-1$ **do**
  if $\|\nabla f(\boldsymbol{x}_t)\| \le g_{\text{thres}}$ and $t - t_{\text{noise}} > t_{\text{thres}}$ **then**
    $\widetilde{\boldsymbol{x}}_t \leftarrow \boldsymbol{x}_t$, $t_{\text{noise}} \leftarrow t$
    $\boldsymbol{x}_t \leftarrow \widetilde{\boldsymbol{x}}_t + \xi_t$, ξ_t uniformly $\sim \mathbb{B}_0(r)$
  end if
  if $t - t_{\text{noise}} = t_{\text{thres}}$ and $f(\boldsymbol{x}_t) - f(\widetilde{\boldsymbol{x}}_{t_{\text{noise}}}) > -f_{\text{thres}}$ **then**
    RETURN $\widetilde{\boldsymbol{x}}_{t_{\text{noise}}}$
  end if
  $\boldsymbol{x}_{t+1} \leftarrow \boldsymbol{x}_t - \eta\nabla f(\boldsymbol{x}_t)$
end for
Output: $\boldsymbol{x}_K$

5.4 Coordinate Descent and Stochastic Coordinate Descent

References:

Wright, S.J., Coordinate Descent Algorithms, *Mathematical Programming*, 2015, **151**(1): 3–34.

Nesterov, Y., Efficiency of Coordinate Descent Methods on Huge–Scale Optimization Problems, *SIAM Journal on Optimization*, 2012, **22**(2): 341.

The k–th iteration of Coordinate Descent applied to a function $f : \mathbb{R}^n \to \mathbb{R}$ chooses some

index $i_k \in \{1, 2, ..., n\}$ and takes iteration

$$x^{k+1} = x^k + \gamma_k e_{i_k} \ ,$$

where e_{i_k} is the i_k–th unit vector and γ_k is the step.

Example 1. Gauss–Seidel method:

$$\gamma_k = \arg\min_{\gamma} f(x^k + \gamma e_{i_k}) \ .$$

Example 2. Coordinate Gradient Descent method:

$$\gamma_k = -\alpha_k \frac{\partial f}{\partial x_{i_k}}(x^k) e_{i_k}$$

for some $\alpha_k > 0$.

Example 3. If i_k is chosen randomly the method is Stochastic Coordinate Descent.

Descent methods need $f(x^{k+1}) < f(x^k)$ for all k.

Chapter 6

Second Order Methods

6.1 Newton's method

Second order methods: Produce an optimization sequence $\{x^k\}$ using second–order Taylor expansions (quadratic surrogate)

$$\min_{x \in \mathcal{U}(x^k)} f(x) \approx \min_{x \in \mathcal{U}(x^k)} \left[f(x^k) + \nabla f(x^k)^T (x - x^k) + \frac{1}{2}(x - x^k)^T \nabla^2 f(x^k)(x - x^k) \right] ,$$

where $\mathcal{U}(x^k)$ is a convex open neighborhood of x^k.

If the Hessian $\nabla^2 f(x^k)$ is invertible, the minimum is taken at $x^k - [\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$.

Newton's iteration:

$$x^{k+1} = x^k - [\nabla^2 f(x^k)]^{-1} \nabla f(x^k) \ . \tag{6.1}$$

Algorithm 6.1 Newton's method

1: **Input**: Input Initialization x^0.
2: **for** $k = 0, 1, ..., K - 1$ **do**
3: Calculate the gradient $\nabla f(x^k)$
4: Calculate the Hessian $\nabla^2 f(x^k)$ and the inverse $[\nabla^2 f(x^k)]^{-1}$
5: Update $x^{k+1} = x^k - [\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$
6: **end for**
7: **Output**: x^K

Convergence of Newton's method.

Theorem 6.1 (Local superlinear convergence of the Newton's method)**.** *Suppose that the function f is twice differentiable and the Hessian $\nabla^2 f(x)$ is Lipschitz–continuous with Lipschitz constant $M > 0$, i.e.*

$$\|\nabla^2 f(x) - \nabla^2 f(y)\| \leq M \|x - y\| \ .$$

Let x^* be a second–order stationary point in the sense of Theorem 2.7 (b), then if the starting point x_0 is sufficiently close to x^*, then the sequence of iterates from Newton's method (6.1) converges to x^* at a superlinear rate (in particular, this rate is quadratic).

Proof. See the proof of Theorem 3.5 of the textbook: Nocedal, J., Wright, S.J., *Numerical Optimization*, Second Edition, Springer, 2006. □

Drawbacks of Newton's method: (1) Inverse of Hessian $[\nabla^2 f(x)]^{-1}$ hard to calculate for high–dimensional problems; (2) No positive definite guarantee of Hessian $\nabla^2 f(x)$.

6.2 Quasi–Newton: BFGS

Reference: J. E. Dennis, Jr., Jorge J. Moré, Quasi–Newton methods, motivation and theory, *SIAM Review*, Vol. **19**, No. 1 (Jan. 1977), pp. 46–89.

Quasi–Newton method: If the Hessian $\nabla^2 f(x^k)$ is not positive definite or it is hard to calculate, we need to replace it with a positive definite matrix B_k that approximates $\nabla^2 f(x^k)$: $B_k \approx \nabla^2 f(x^k)$. Or we update the inverse of the Hessian matrix $H_k \approx [\nabla^2 f(x^k)]^{-1}$ iteratively, so that we do not have to recalculate the approximation of the inverse of the Hessian matrix $[\nabla^2 f(x^k)]^{-1}$ at each iteration.

Set $B_k = \nabla^2 f(x^k)$. Quasi–Newton condition uses quadratic surrogate around each two consecutive iterations x^k and x^{k+1}, which produces an iteration that calculates B_{k+1} from current iteration. In fact we have

$$f(x^k) \approx f(x^{k+1}) + \nabla f(x^{k+1})^T (x^k - x^{k+1}) + \frac{1}{2}(x^k - x^{k+1})^T \nabla^2 f(x^{k+1})(x^k - x^{k+1})$$

gives

$$\nabla f(x^k) \approx \nabla f(x^{k+1}) + B_{k+1}(x^k - x^{k+1}) \ .$$

Set $y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$ and $s_k = x^{k+1} - x^k$. Then

$$B_{k+1} s_k = y_k \ . \tag{6.2}$$

Equation (6.2) is called the *secant equation*. The secant equation will produce a next iteration B_{k+1} from current iteration, and the update rule from B_k to B_{k+1} should always satisfy the following criteria: (1) it satisfies the secant equation; (2) it keeps symmetry and positive definiteness of B_{k+1}; (3) it is a low rank modification from B_k to B_{k+1}.

The positive definiteness can be directly obtained from the secant equation since we have from (6.2) that $s_k^T B_{k+1} s_k = s_k^T y_k > 0$. This give the *curvature condition*: for any $x, y \in \mathrm{Dom}(f)$ we have

$$(x - y)^T (\nabla f(x) - \nabla f(y)) > 0 \ . \tag{6.3}$$

It can be shown that when f is strongly convex then (6.3) is satisfied. This is left as one problem in Homework 4.

Secant equation is solved by doing minimization such as

$$\min_B \|B - B_k\| \text{ subject to } B = B^T \ , \ Bs_k = y_k \ .$$

Choosing appropriate norm, solution is
DFP (Davidson–Fletcher–Powell) update

$$B_{k+1} = (I - \rho_k y_k s_k^T) B_k (I - \rho_k s_k y_k^T) + \rho_k y_k y_k^T \ , \ \rho_k = \frac{1}{y_k^T s_k} \ . \tag{6.4}$$

Sherman–Morrison–Woodbury formula

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1} u v^T A^{-1}}{1 + v^T A^{-1} u} \ , \text{ where } u, v \text{ are column vectors} \ .$$

Thus we get (try it!) the DFP iteration for $H_k = B_k^{-1}$ as

$$H_{k+1} = H_k - \frac{H_k y_k y_k^T H_k}{y_k^T H_k y_k} + \frac{s_k s_k^T}{y_k^T s_k} \ . \tag{6.5}$$

In a same way but conversely, we have the secant equation for H:

$$H_{k+1} y_k = s_k \ . \tag{6.6}$$

Secant equation is solved by doing minimization such as

$$\min_H \|H - H_k\| \text{ subject to } H = H^T \ , \ Hy_k = s_k \ .$$

BFGS (Broyden–Fletcher–Goldfarb–Shanno) update

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T \ , \ \rho_k = \frac{1}{s_k^T y_k} \ . \tag{6.7}$$

Invert it we get

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k} \ . \tag{6.8}$$

Algorithm 6.2 Quasi–Newton: BFGS method

1: **Input**: Input Initialization x^0, $\nabla f(x^0)$, $H_0 = [\nabla^2 f(x^0)]^{-1}$, $B_0 = I$, learning rate $\eta > 0$
2: **for** $k = 0, 1, ..., K-1$ **do**
3: Calculate $x^{k+1} = x^k - \eta H_k \nabla f(x^k)$
4: Calculate $y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$ and $s_k = x^{k+1} - x^k$
5: Update $H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$
6: Update $B_{k+1} = B_k - \dfrac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \dfrac{y_k y_k^T}{y_k^T s_k}$
7: **end for**
8: **Output**: x^K

Superlinear convergence of BFGS method (see Theorem 6.6 in Nocedal, J., Wright, S.J., *Numerical Optimization*, Second Edition, Springer 2006.).

6.3 Stochastic Quasi–Newton

Reference: Byrd, R.H., Hansen, S.L., Nocedal, J. et al, A stochastic quasi–Newton method for large–scale optimization, *SIAM Journal on Optimization*, 2016, 26(2): 1008–1031.

Large–scale optimization

$$F(w) = \frac{1}{N} \sum_{i=1}^{N} f_i(w) \ .$$

Stochastic gradient estimate: $b = |\mathcal{S}| \ll N$, randomly $\mathcal{S} \subset \{1, 2, ..., N\}$, take

$$\hat{\nabla} F(w) = \frac{1}{b} \sum_{i \in \mathcal{S}} \nabla f_i(w) \ .$$

It is necessary to employ a limited memory variant that is scalable in the number of variables, but enjoys only a linear rate of convergence.

Quasi–Newton updating is inherently an overwriting process rather than an averaging process, and therefore the vector $y = \nabla F(w^{k+1}) - \nabla F(w^k)$ must reflect the action of the Hessian of the entire objective F, which is not achieved by differencing stochastic gradients based on small samples.

To achieve stable Hessian approximation one needs to *decouple* the stochastic gradient and curvature estimation calculations.

Algorithm 6.3 Stochastic Quasi–Newton Method

Input: Input Initialization w^1, positive integers M, L, and step–length sequence $\alpha^k > 0$
Set $t = -1$ ▷ Records number of correction pairs currently computed
$\bar{w}_t = 0$
for $k = 1, 2, ..., K-1$ **do**
  Choose a sample $\mathcal{S} \subset \{1, 2, ..., N\}$
  Calculate the stochastic gradient $\hat{\nabla} F(w^k) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \nabla f_i(w)$
  $\bar{w}_t = \bar{w}_t + w^k$
  if $k \leq 2L$ **then**
    $w^{k+1} = w^k - \alpha^k \hat{\nabla} F(w^k)$ ▷ Stochastic gradient iteration
  else
    $w^{k+1} = w^k - \alpha^k H_t \hat{\nabla} F(w^k)$, where H_t is defined by Algorithm 6.4
  end if
  if $\mod (k, L) = 0$ **then**
    $t = t + 1$
    $\bar{w}_t = \bar{w}_t / L$
    if $t > 0$ **then**
      Choose a sample $\mathcal{S}_H \subset \{1, 2, ..., N\}$ to define $\hat{\nabla}^2 F(\bar{w}_t) \equiv \frac{1}{|\mathcal{S}_H|} \sum_{i \in \mathcal{S}_H} \nabla^2 f(w)$
      Compute $s_t = \bar{w}_t - \bar{w}_{t-1}$, $y_t = \hat{\nabla}^2 F(\bar{w}_t)(\bar{w}_t - \bar{w}_{t-1})$
      ▷ Compute correction pairs every L iterations
    end if
    $\bar{w}_t = 0$
  end if
end for
Output: w^K

Algorithm 6.4 Stochastic Quasi–Newton Method: Hessian Updating

Input: Updating counter t, memory parameter M, correction pairs (s_j, y_j), $j = t - \widetilde{m} + 1, ..., t$ where $\widetilde{m} = \min\{t, M\}$
Set $H = \frac{s_t^T y_t}{y_t^T y_t} I$
for $j = t - \widetilde{m} + 1, ..., t$ **do**
  $\rho_j = \frac{1}{y_j^T s_j}$
  Apply BFGS formula

$$H \leftarrow (I - \rho_j s_j y_j^T) H (I - \rho_j y_j s_j^T) + \rho_j s_j s_j^T$$

end for
Output: new matrix $H_t = H$

Chapter 7

Optimization in Modern Machine Learning

7.1 Adaptive Methods

Adapted algorithms are those training algorithms that update the parameters under training using not only the current gradient information but also all past gradient information, as well as adaptively change the learning rate. These algorithms are sometimes also called to be belonging to the Ada–Series.

7.1.1 SGD with momentum

Reference: [24]

Algorithm 7.1 SGD with momentum (= stochastic heavy ball method)

Input: Input Initialization $w_0 = w_{-1}$ and $v_{-1} = 0$
for $k = 0, 1, ..., K-1$ **do**
  Randomly sample minibatch $\mathcal{S}_i \subset \{1, 2, ..., n\}$ such that $|\mathcal{S}_i| = m$
  Update $v_{k+1} = \mu v_k - \eta \frac{1}{m} \sum_{i \in \mathcal{S}} \nabla f_i(w_k)$
  Update $w_{k+1} = w_k + v_{k+1}$
end for
Output: w_K

7.1.2 AdaGrad, RMSProp, AdaDelta

AdaGrad (see [10]) not just accumulate historical information (such as square gradient information) but also adjust the learning rate (stepsize) according to the magnitude of historical gradient information at different dimensions, so that those dimensions with relatively small magnitudes of the gradient have larger learning rates. In this way, the algorithm favors some dimensions that have currently small magnitude of the gradient but contributes importantly to the optimization search

path. This is more suitable to optimize neural network models that possess many local minimizers and saddle points. Empirical evidence shows that AdaGrad can stabilize SGD. The disadvantage of AdaGrad comes from the term $\sqrt{g_{k+1}}$ in the denominator, since as it accumulates during the training process the learning rates will gradually tend to 0 and this leads to early stopping.

Algorithm 7.2 AdaGrad

Input: Input Initialization w_0 and $g_0 = 0$
for $k = 0, 1, ..., K-1$ **do**
  Update $g_{k+1} = g_k + (\nabla f(w_k))^2$
  Update $w_{k+1} = w_k - \dfrac{\eta \nabla f(w_k)}{\sqrt{g_{k+1}} + \varepsilon_0}$, $\varepsilon_0 > 0$
end for
Output: w_K

RMSProp (see [26]) is similar to AdaGrad but also makes use of the idea of SGD with momentum, this gives an exponential weight–decay average of historical information. RMSProp does a weighted average of both g_k and $(\nabla f(w_k))^2$, so that the weights in historical information will be small and thus it favors more on current gradient information. This avoids the monotonic decay of the learning rate (stepsize) along with the training process.

Algorithm 7.3 RMSProp

Input: Input Initialization w_0 and $g_0 = 0$
for $k = 0, 1, ..., K-1$ **do**
  Update $g_{k+1} = \gamma g_k + (1-\gamma)(\nabla f(w_k))^2$
  Update $w_{k+1} = w_k - \dfrac{\eta \nabla f(w_k)}{\sqrt{g_{k+1}} + \varepsilon_0}$, $\varepsilon_0 > 0$
end for
Output: w_K

AdaDelta (see [29]), based on RMSProp, adjust the accumulated historical information according to the magnitude of gradients.

Algorithm 7.4 AdaDelta

Input: Input Initialization w_0 and $u_0 = 0$, $v_0 = 0$ and $g_0 = 0$
for $k = 0, 1, ..., K-1$ **do**
  Update $g_{k+1} = \gamma g_k + (1-\gamma)(\nabla f(w_k))^2$
  Update $v_{k+1} = \gamma v_k + (1-\gamma) u_k^2$
  Update $u_{k+1} = -\dfrac{\sqrt{v_k + \varepsilon_0} \nabla f(w_k)}{\sqrt{g_{k+1}} + \varepsilon_0}$
  Update $w_{k+1} = w_k + u_{k+1}$
end for
Output: w_K

7.1.3 Adam

Adam (see [15], [21]) combines all ingredients in SGD–momentum, AdaGrad, RMSProp, AdaDelta: (1) The update direction is determined by accumulated historical gradient information; (2) Correct the stepsize using accumulated square gradient information; (3) Exponential decay of accumulated information. Adam has been widely used in training neural networks.

Algorithm 7.5 Adam

Input: α: stepsize; $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates; $f(\theta)$: stochastic objective function with parameters θ; θ_0: initial parameter vector; $m_0 \leftarrow 0$ (initialize first moment vector); $v_0 \leftarrow 0$ (initialize second moment vector); $t \leftarrow 0$ (initialize timestep)
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
  $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
  $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
  $\hat{m}_t \leftarrow m_t/(1 - \beta_1^t)$ (Compute bias–corrected first moment estimate)
  $\hat{v}_t \leftarrow v_t/(1 - \beta_2^t)$ (Compute bias–corrected second raw moment estimate)
  $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$ (Update parameters)
end while
Output: θ_t (Resulting parameters)

7.2 Training a Transformer: AdamW and Muon

7.2.1 Transformer Neural Network

See [28] and [14]. GPT uses transformer network so we take this example (nanochat).

Computational Modules.

• RMS norm: normalization of token's embedding vector $\boldsymbol{x} \in \mathbb{R}^d$. There are no learnable params in RMSNorm. This is different from transformer paper's LayerNorm.

$$\mathrm{RMSNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x}}{\mathrm{RMS}(\boldsymbol{x}) + \varepsilon} \ , \ \mathrm{RMS}(\boldsymbol{x}) = \sqrt{\sum_{i=1}^{d} x_i^2} \ , \ \boldsymbol{x} = (x_1, ..., x_d) \ , \ \varepsilon > 0 \ .$$

• Rotary Positional Embedding (RoPE): positional embedding using sin, cos positional tensors. This is different from transformer paper's Positional Embedding.

Input x (which is the Q, K in causal attention module) with shape (B, T, H, D) where B =batch size, T =sequence length, H =number of heads in multi-head attention, D =dimension of each head (must be even). Let $\sin = (\sin^{t,i})_{0 \le t \le T-1, 0 \le i \le D/2}$, $\cos = (\cos^{t,i})_{0 \le t \le T-1, 0 \le i \le D/2}$ be tensors each with shape $(1, T, 1, D/2)$. Then for each batch b, each index $t \in \{0, ..., T-1\}$, each head h, RoPE

does the following:

$$(x^{b,t,h,0}, ..., x^{b,t,h,D-1}) \to \begin{cases} \cos^{t,i} x^{b,t,h,i} + \sin^{t,i} x^{b,t,h,i+D/2} & \text{for } 0 \le i \le D/2-1 \ , \\ -\sin^{t,i-D/2} x^{b,t,h,i-D/2} + \cos^{t,i-D/2} x^{b,t,h,i} & \text{for } D/2 \le i \le D-1 \ . \end{cases}$$

Position information, originally in t, is encoded in such rotation via $\cos, \sin$. Usually we take $\cos^{t,i} = \cos\left(\frac{t}{10000^{2i/D}}\right)$ and $\sin^{t,i} = \sin\left(\frac{t}{10000^{2i/D}}\right)$.

• Causal Self-Attention. The self-attention module with specific causal mask construction.

– number of heads `n_head` % number of kv heads `n_kv_head` $= 0$.

– self-attention. Input x has shape $(B, T, _)$, which stands for the current forward pass. Then Q, K, V are from input x, i.e., $Q = W_Q^T X$ etc. They are re-shaped into shape $(B, T, \texttt{n_head}, \texttt{head_dim})$ for Q and $(B, T, \texttt{n_kv_head}, \texttt{head_dim})$ for K, V, respectively. Then apply RoPE to Q, K, apply RMSNorm to Q, K, finally apply $(1, 2)$ transpose $(B, T, H, D) \to (B, H, T, D)$ to Q, K, V, i.e., to process each attention head to the full batch (head-wise parallelization);

– kv-cache and multi-query-attention. T in (B, H, T, D) stands for the number of queries. For Q, use number of queries in this forward pass $= T_q$; for K, V, after kv-cache, use number of queries in the cache + current forward pass $= T_q + (T_k)_{\text{prev}} = T_k$.

PyTorch 2.0's `scaled_product_attention`:

```
def scaled_dot_product_attention(
    query: Tensor,
    key: Tensor,
    value: Tensor,
    attn_mask: Tensor | None = None,
    dropout_p: float = 0.0,
    is_causal: bool = False,
    scale: float | None = None,
    enable_gqa: bool = False,
) -> Tensor: ...
```

Multi-Query-Attention (Group-Query-Attention, gqa):

`enable_gqa = self.n_head != self.n_kv_head`: multiple query heads use the same key/value head, or in other words duplicately use key/value head for a chunk of query heads. This saves computation. Example: `n_head`= 16, `n_kv_head`= 4, then each group of 4 query heads share the same kv head, or the same kv head repeatedly used 4 times for each chunk of 4 query heads.

Mask construction in multi-query-attention: Discuss the following cases

(1) Inference with no increment, i.e. infer the whole sequence at once ($T_q = T_k$) or training (kv cache is empty): Auto regressive case. Each query only attends keys/values before it. Use standard causal mask. See Figure 7.1 left.

(2) $T_q = 1$, inference with single query in this forward pass. Each query attends all the keys/values in the cache. No mask needed. See Figure 7.1 middle.

(3) Not (1) or (2): $1 < T_q < T_k$. Inference with a chunk of T_q queries in this forward pass. Each query attends all the keys/values in the old kv cache $(T_k)_{\text{prev}} = T_k - T_q$, but for the newly added T_q items of kv, i.e. within the chunk we use causal attention so that each query only attends keys/values before it. See Figure 7.1 right.

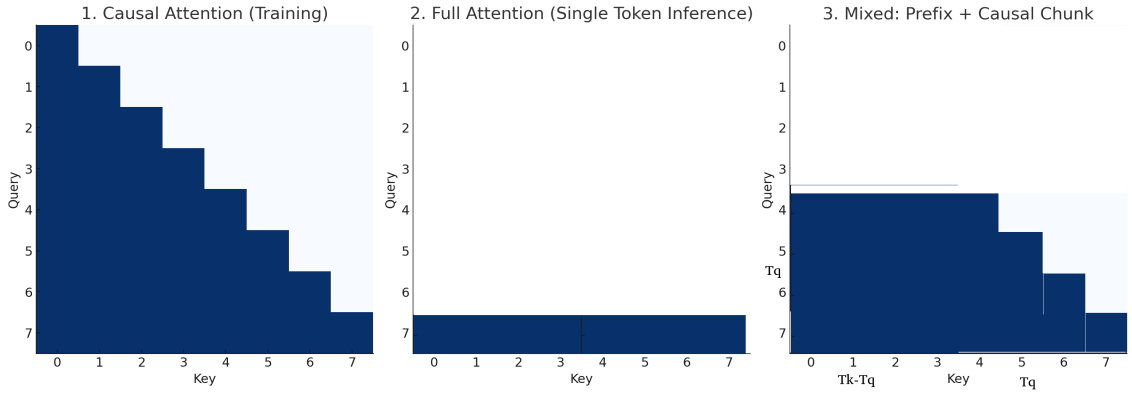

Figure 7.1: MQA masks.

– final projection. Concatenate heads

$$(B, \texttt{n_head}, T, \texttt{head_dim}) \to (B, T, \texttt{n_head}, \texttt{head_dim}) \to (B, T, \texttt{n_head} \times \texttt{head_dim}) \ .$$

Followed by a linear layer $\texttt{n_head} \times \texttt{head_dim} \to \texttt{n_embd}$.

• MLP. Layer structure: $\texttt{n_embd} \to 4\times\ \texttt{n_embd} \xrightarrow{\text{ReLU}^2} \texttt{n_embd}$. Activation function use

$$\text{ReLU}^2(x) = \begin{cases} x^2 & \text{if } x \geq 0 \ , \\ 0 & \text{if } x < 0 \ . \end{cases}$$

• Block. Use residual connections

$$x \mapsto x' = x + \text{Causal Self-Attention} \mapsto x'' = x' + \text{MLP} \ .$$

The whole GPT workflow.

• Network architecture:

– `wte`: word-to-embedding layer, `vocab_size`= 50304 is the amount of tokens, `n_embd`= 768 is the embedding dimension.

– `h`: transformer block layer, `n_layer`= 12 layers of Block.

– `lm_head`: language modeling head, head layer, embedding → token logits (`vocab_size` tokens).

```
Input ids
   ↓
Embedding (wte)
   ↓
Norm
   ↓
n_layer × Transformer Blocks (Block) (n_layer=12)
   ↓
Norm
   ↓
Linear Output Head (lm_head)
   ↓
Logits / Loss
```

- Initialize weights:
  - linear layers: $\mathcal{N}(0, \sigma)$ where $\sigma = \frac{1}{\sqrt{fan_in}} \cdot \left(1 \wedge \sqrt{\frac{fan_out}{fan_in}}\right)$
  - embedding layers: $\mathcal{N}(0, 1)$
  - `lm_head` weights: 0
  - `c_proj` weights: 0
  - initialize the rotary embeddings: `_precompute_rotary_embeddings()`, use $\cos^{t,i} = \cos\left(\frac{t}{10000^{2i/D}}\right)$ and $\sin^{t,i} = \sin\left(\frac{t}{10000^{2i/D}}\right)$ as we indicated before, with `rotary_seq_len`, i.e. running range of t, being 10 times of the `seq_len`= 1024.
- Optimizers: AdamW for embedding layers and Muon for linear layers.
- Computing the logits: `lm_head` layer, softcap the logits using $15*\texttt{tanh}\left(\frac{\text{logit}}{15}\right)$ (`softcap`= 15). Training mode use cross-entropy loss towards the `target`.
- Naive autoregressive streaming inference: generate the next token through output logits. Only first top k logits are considered before softmax. Then softmax logits with control of randomness using `tempreature`: when `tempreature`> 0, then $\frac{\text{logit}}{\text{tempreature}}$ as the new `logit`, then softmax and random sampling; but when `tempreature`$= 0$, deterministically sample argmax of `logit`.
- Use of `bfloat16` for GPU.

7.2.2 AdamW

Motivation and setup. Momentum methods maintain a first-moment (mean) estimate of gradients. Adam (Adaptive Moment Estimation) extends this idea by also maintaining a second-moment (uncentered variance) estimate, enabling per-parameter adaptive step sizes. Let $\hat{\mathbf{g}}_t$ denote the stochastic gradient at step t (mini-batch gradient of the loss w.r.t. θ_t).

Review of Adam: first and second moment estimates. Adam maintains exponential moving averages of the gradient and its elementwise square:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\hat{\mathbf{g}}_t, \tag{7.1}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\hat{\mathbf{g}}_t \odot \hat{\mathbf{g}}_t, \tag{7.2}$$

with hyperparameters $\beta_1, \beta_2 \in [0,1)$ and initialization $\mathbf{m}_0 = \mathbf{0}$, $\mathbf{v}_0 = \mathbf{0}$. Because these moving averages start at zero, they are biased toward zero at early iterations. Adam therefore uses *bias-corrected* estimates

$$\hat{\mathbf{m}}_t \;=\; \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{v}}_t \;=\; \frac{\mathbf{v}_t}{1-\beta_2^t}. \tag{7.3}$$

The parameter update is then

$$\theta_{t+1} \;=\; \theta_t - \eta_t \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}, \tag{7.4}$$

where $\sqrt{\cdot}$ and division are elementwise, and $\varepsilon > 0$ ensures numerical stability. Intuitively, coordinates with consistently large gradient magnitudes obtain larger $\hat{\mathbf{v}}_t$ and thus smaller effective step sizes, while coordinates with smaller or sparser gradients obtain larger effective step sizes.

Weight decay and the "L^2 regularization" distinction. A common regularization strategy is *weight decay*, which shrinks parameters toward zero during training. In classic (non-adaptive) SGD, adding an L^2 penalty $\frac{\lambda}{2}\|\theta\|_2^2$ to the loss leads to the update

$$\theta_{t+1} \;=\; \theta_t - \eta_t \left(\hat{\mathbf{g}}_t + \lambda \theta_t\right),$$

which is equivalent to multiplicative decay $\theta_{t+1} = (1-\eta_t\lambda)\theta_t - \eta_t \hat{\mathbf{g}}_t$.

However, *for Adam*, naively adding the L^2 gradient term $\lambda\theta_t$ to $\hat{\mathbf{g}}_t$ is **not** equivalent to true weight decay, because the adaptive preconditioning in (7.4) scales each coordinate differently. Concretely, with L^2 added inside Adam one obtains (schematically)

$$\theta_{t+1} \approx \theta_t - \eta_t \frac{\hat{\mathbf{m}}_t(\hat{\mathbf{g}}_t + \lambda\theta_t)}{\sqrt{\hat{\mathbf{v}}_t(\hat{\mathbf{g}}_t + \lambda\theta_t)} + \varepsilon},$$

so the effective shrinkage applied to θ_t is entangled with the moment estimates and varies across parameters and training time. This coupling can lead to suboptimal regularization behavior in practice.

AdamW: decoupled weight decay. AdamW resolves this issue by *decoupling* the weight-decay shrinkage from the gradient-based update. In its standard form, AdamW performs Adam updates using the loss gradient *without* L^2 added, and then applies weight decay directly to parameters:

$$\theta_{t+1} \;=\; \theta_t - \eta_t \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon} \;+\; \lambda\, \theta_t \right), \tag{7.5}$$

where $\lambda \geq 0$ is the weight-decay coefficient. Equivalently, (7.5) can be written as a multiplicative shrinkage followed by an Adam step:

$$\theta_{t+1} \;=\; (1-\eta_t\lambda)\theta_t - \eta_t \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}.$$

This separation restores the intended interpretation of weight decay as parameter shrinkage that is independent of the adaptive gradient preconditioner.

Implementation note (parameter groups). In deep networks and LLMs, weight decay is commonly applied to weight matrices (e.g., linear/MLP/attention projections) but *not* to certain parameters such as biases and normalization parameters (e.g., LayerNorm scale and shift), because decaying these can degrade optimization and representation capacity. This is typically implemented by placing parameters into distinct optimizer groups with different λ values.

Default hyperparameters (for reference). Unless otherwise stated, typical Adam/AdamW defaults are $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$, with the learning rate η_t and weight decay λ tuned to the model scale, batch size, and schedule.

7.2.3 Muon

Motivation Empirically, the parameter updates produced by SGD-momentum and Adam for two-dimensional parameter tensors[1] in transformer-based neural networks often show high condition numbers. This observation leads to the assumption that the updates are of low rank and are dominated by only a few directions. To encounter this phenomenon, the Muon (*MomentUm Orthogonalized by Newton–Schulz*) optimizer applies an iterative approximation method to the momentum-based update matrix so that its singular values are driven towards unity. Consequently, the conditioning of the update matrix improves, and the scale of "rare directions" effectively increases [13].

Mathematical foundation for the Muon optimizer Assume w.l.o.g. $m \geq n$ (otherwise consider the matrix transpose). Let $\mathbf{M} \in \mathbb{R}^{m\times n}$ be a real-valued matrix with reduced Singular Value Decomposition (SVD) $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, where $\mathbf{U} \in \mathbb{R}^{m\times n}$ and $\mathbf{V}^\top \in \mathbb{R}^{n\times n}$ are *orthogonal* matrices and $\mathbf{\Sigma} = \operatorname{diag}[(\sigma_1, \ldots, \sigma_n)] \in \mathbb{R}^{n\times n}$ is a *diagonal* matrix with monotonically decreasing singular values $\{\sigma_i\}_{i=1}^n$ of $\mathbf{M}$.
Define the best *semi-orthogonal* approximation to $\mathbf{M}$ as

$$\operatorname{Ortho}(\mathbf{M}) := \arg\min_{\mathbf{O}}\{\|\mathbf{O} - \mathbf{M}\|_F \ : \text{ either } \mathbf{O}\mathbf{O}^\top = \mathbf{I} \text{ or } \mathbf{O}^\top\mathbf{O} = \mathbf{I}\},$$

where $\|\cdot\|_F$ denotes the Frobenius norm. The following part shows that $\mathbf{U}\mathbf{V}^\top = \operatorname{Ortho}(\mathbf{M})$.
First, due to the assumption that $m \geq n$ and the orthogonality of $\mathbf{U}$ and $\mathbf{V}$,

$$(\mathbf{U}\mathbf{V}^\top)^\top(\mathbf{U}\mathbf{V}^\top) = \mathbf{V}\mathbf{U}^\top\mathbf{U}\mathbf{V}^\top = \mathbf{I}_n$$

is true.
Second, take any matrix $\mathbf{O} \in \mathbb{R}^{m\times n}$ with orthonormal columns, then

$$\|\mathbf{O} - \mathbf{M}\|_F^2 = \|\mathbf{O}\|_F^2 - 2\operatorname{tr}(\mathbf{O}^\top\mathbf{M}) + \|\mathbf{M}\|_F^2. \tag{7.6}$$

[1] Two-dimensional parameter tensors occur from linear transformations, such as in MLP and attention projections.

Since $\|\mathbf{O}\|_F^2 = \mathrm{tr}(\mathbf{O}^\top\mathbf{O}) = \mathrm{n}$, minimizing (7.6) is equivalent to maximizing $\mathrm{tr}(\mathbf{O}^\top\mathbf{M})$.
Use the SVD of $\mathbf{M}$ and the cyclicity of the trace operator to obtain

$$\mathrm{tr}(\mathbf{O}^\top\mathbf{M}) = \mathrm{tr}(\mathbf{\Sigma}\mathbf{V}^\top\mathbf{O}^\top\mathbf{U}).$$

Since $\mathbf{V}^\top\mathbf{O}^\top\mathbf{U}$ has singular values ≤ 1, von Neumann's trace inequality gives

$$\mathrm{tr}(\mathbf{O}^\top\mathbf{M}) \leq \mathrm{tr}(\mathbf{\Sigma}),$$

with equality at $\mathbf{O} = \mathbf{U}\mathbf{V}^\top$, which proves the statement.
Muon replaces the momentum-based update matrix $\mathbf{M}$ by an approximation of $\mathbf{U}\mathbf{V}^\top$, which is achieved by odd polynomial iteration, also known as Newton–Schulz iteration [13].
Define the odd matrix polynomial of degree $2n+1$ as

$$p(\mathbf{X}) \;=\; \sum_{i=1}^{n} \alpha_i (\mathbf{X}\mathbf{X}^\top)^i\,\mathbf{X} \;=\; \alpha_1\mathbf{X} + \alpha_2(\mathbf{X}\mathbf{X}^\top)\mathbf{X} + \alpha_3(\mathbf{X}\mathbf{X}^\top)^2\,\mathbf{X} + \ldots + \alpha_n(\mathbf{X}\mathbf{X}^\top)^n\,\mathbf{X}$$

Then

$$\begin{aligned} p(\mathbf{M}) &= \sum_{i=1}^{n} \alpha_i (\mathbf{M}\mathbf{M}^\top)^i\,\mathbf{M} \\ &= \sum_{i=1}^{n} \alpha_i (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top)^\top)^i\,\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \\ &= \mathbf{U}\left(\sum_{i=1}^{n} \alpha_i (\mathbf{\Sigma}\mathbf{\Sigma}^\top)^i\,\mathbf{\Sigma}\right)\mathbf{V}^\top \\ &= \mathbf{U}p(\mathbf{\Sigma})\mathbf{V}^\top \end{aligned}$$

shows that applying an odd matrix polynomial to $\mathbf{M}$ is equivalent to applying it to $\mathbf{\Sigma}$ from its SVD.
Another key observation is that normalizing the matrix $\mathbf{M}$ by $\frac{\mathbf{M}}{\|\mathbf{M}\|_F}$ does not affect its orthogonal approximation, because $\mathrm{Ortho}(c\mathbf{M}) = \mathrm{Ortho}(\mathbf{M})$. This normalization step will ensure $\sigma_i \in [0,1]$ for $i = 1,\ldots,n$ [13].
Since $\mathbf{\Sigma}$ is a diagonal matrix, the objective after normalization reduces to finding the coefficients of an odd real polynomial $\hat{p}$ such that

$$\hat{p}^N(x) = (\hat{p}\circ\ldots\circ\hat{p})(x) \to 1 \text{ as } N\to\infty \text{ for all } x\in(0,1].$$

Orthogonalization via Newton–Schulz iterations The original implementation of the Muon optimizer chooses the quintic polynomial

$$\hat{p}(x) = 3.4445x - 4.7750x^3 + 2.0315x^5,$$

which makes a sacrifice of exact convergence towards 1 for $x\in(0,1]$, but maximizes the convergence rate of small singular values [13]. The Newton–Schulz iteration defined in Algorithm 7.6 produces an approximate orthogonalized update with singular values close to 1.

Algorithm 7.6 Newton–Schulz

Require: Momentum update matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$, steps $N \in \mathbb{N}$

```
ε ← 1e − 7
(a, b, c) ← (3.4445, 4.7750, 2.0315)
Normalize X ← M/(‖M‖_F + ε)
if m > n then
    Transpose X ← X^⊤
end if
for t = 1, ..., N do
    A ← XX^⊤
    B ← bA + cAA
    M ← aX + BX
end for
if m > n then
    Transpose back X ← X^⊤
end if
return X
```

The Muon training loop Denote by θ_t the two-dimensional parameters of the model, by $\mathbf{M}_t$ the momentum matrix, and by $\mathbf{G}_t$ the gradient matrix at step t. Muon begins from a momentum-style update direction:

$$\mathbf{M}_t \;=\; \mu\,\mathbf{M}_{t-1} + \mathbf{G}_t, \tag{7.7}$$

where $\mu \in [0, 1)$ and $\mathbf{M}_0 = \mathbf{0}$. Some implementations optionally use a Nesterov-style lookahead.
Muon then post-processes the momentum direction $\mathbf{M}_t$ by approximately projecting it onto a spectrally normalized matrix. It does so by applying a small fixed number (e.g. 5 in the original implementation) of Newton–Schulz iterations (see Algorithm 7.6), which are a GPU-friendly sequence of matrix multiplications.
Finally, Muon applies the orthogonalized update to the matrix parameters by

$$\theta_t = \theta_{t-1} - \eta \mathbf{O}_t, \tag{7.8}$$

where η is the learning rate. Practical Muon variants for large-scale training emphasize that decoupled weight decay and careful update scaling are important for stability at scale.
The training loop is summarized in Algorithm 7.7 [13].

Algorithm 7.7 Muon

Require: Initial parameters θ_0, learning rate $\eta > 0$, momentum $\mu \in [0, 1)$, steps N

```
Initialize M_0 ← 0
for t = 1, ..., N do
    Compute gradient G_t ← ∇_θ L_t(θ_{t-1})
    M_t ← μ M_{t-1} + G_t
    O_t ← Newton-Schulz(M_t, 5)
    Update parameters θ_t ← θ_{t-1} − η O_t
end for
return θ_N
```

Hybrid pattern Let θ be partitioned into (i) matrix parameters θ_{mat} (all 2D weights in transformer blocks) and (ii) remaining parameters θ_{vec} (biases, normalization parameters, embeddings, unembeddings, and other 1D/3D tensors). A common and effective configuration is:

$$\theta_{\text{mat}} \text{ updated by Muon ((7.8))}, \qquad \theta_{\text{vec}} \text{ updated by AdamW (§\textbf{??})}.$$

This split reflects Muon's matrix-specific design: the orthogonalization step is meaningful for linear transformations (2D), whereas for vectors and scalars it is either undefined or not beneficial. In implementation, the hybrid approach is realized through optimizer parameter groups and typically entails maintaining separate optimizer states and optionally separate learning-rate schedules for θ_{mat} and θ_{vec}.

7.3 Dual Methods

7.3.1 Dual Coordinate Descent/Ascent

Reference: Chapter 13 & 14 of Nocedal, J., Wright, S.J., *Numerical Optimization*, Second Edition, Springer, 2006.

Constraint optimization problem P_0:

$$\min_w f(w)$$
$$\text{s.t. } g_i(w) \le 0 \ , \ i = 1, 2, ..., m_1 \ , \ h_j(w) = 0 \ , \ j = 1, 2, ..., m_2 \ .$$

Let the optimal solution be p^*.

Unconstrained Version:

$$\min_w \left(f(w) + \infty \sum_{i=1}^{m_1} \mathbf{1}_{[g_i(w)>0]} + \infty \sum_{j=1}^{m_2} \mathbf{1}_{[h_j(w)\neq 0]} \right) \ .$$

Approximate the indicators by linear functions, obtain the Lagrangian function

$$L(w, \lambda, \nu) \equiv f(w) + \sum_{i=1}^{m_1} \lambda_i g_i(w) + \sum_{j=1}^{m_2} \nu_j h_j(w) \ ,$$

in which $\lambda_i \geq 0$ and $\nu_j \in \mathbb{R}$ are the *Lagrange multipliers*. The variable w is called the *primal* variable.

Set the *Lagrangian dual function*

$$h(\lambda, \nu) \equiv \inf_{\text{acceptable } w} L(w, \lambda, \nu) \ , \ \lambda_i \geq 0 \text{ and } \nu_j \in \mathbb{R} \ .$$

Since for any acceptable w we have $g_i(w) \leq 0$ and $h_i(w) = 0$, so we have

$$h(\lambda, \nu) \leq \inf_{\text{acceptable } w} f(w) = p^* \ .$$

The variables (λ, ν) are called *dual* variables.

Observation: $h(\lambda, \nu)$ is always concave (even when the original problem P_0 is not convex) in the dual variables λ and ν.

Dual Problem D_0 ("primal-dual"):

$$\begin{aligned} &\max_{\lambda, \nu} h(\lambda, \nu) \\ &\text{s.t. } \lambda_i \geq 0 \ , \ i = 1, 2, ..., m \ . \end{aligned}$$

Let the optimal solution be d^*. Then $d^* \leq p^*$.

If $d^* = p^*$ this is called *strong duality*. A sufficient condition for strong duality is Slater condition and a necessary condition for strong duality is KKT condition.

Set $d^* = h(\lambda^*, \nu^*)$. Then solve the unconstrained optimization problem

$$\min_w L(w, \lambda^*, \nu^*) \ .$$

If the solution is acceptable then it is the optimal solution for the original problem.

Algorithm 7.8 Dual Coordinate Ascent

1: **Input**: Input Initialization v_0, $\lambda_0 \geq 0$
2: **for** $k = 0, 1, ..., K-1$ **do**
3: Calculate w_k such that $w_k = \arg\min_w L(w, \lambda_{k-1}, v_{k-1})$
4: Update $\lambda_{k,i} = \max\{\lambda_{k-1,i} + \eta_k g_i(w_k), 0\}$, $i = 1, 2, ..., m_1$
5: Update $v_{k,j} = v_{k-1,j} + \eta_k h_j(w_k)$, $j = 1, 2, ..., m_2$
6: **end for**
7: **Output**: w_K

7.3.2 SDCA

Shalev–Schwarz, S., Zhang, T., Stochastic Dual Coordinate Ascent Methods for Regularized Loss Minimization, *Journal of Machine Learning Research*, **14**(2013), pp. 567–599.

Let $x_1, ..., x_n$ be vectors in $\mathbb{R}^d$ and $\phi_1, ..., \phi_n$ be a sequence of scalar convex functions, and let $\lambda > 0$ be a regularization parameter. Our goal is to solve the optimization problem $P(w^*) = \min_{w \in \mathbb{R}^d} P(w)$ where

$$P(w) = \frac{1}{n} \sum_{i=1}^{n} \phi_i(w^T x_i) + \frac{\lambda}{2} \|w\|^2 \ .$$

Here $\phi_i(w^T x_i)$ is the loss function for linear models. For example, given labels $y_1, ..., y_n$ in $\{\pm 1\}$, the SVM problem (with linear kernels and no bias term) is obtained by setting $\phi_i(a) = \max\{0, 1 - y_i a\}$; regularized logistic regression is obtained by setting $\phi_i(a) = \log(1 + \exp(-y_i a))$; ridge regression is obtained by setting $\phi_i(a) = (a - y_i)^2$; regression with the absolute–value is obtained by setting $\phi_i(a) = |a - y_i|$; support vector regression is obtained by setting $\phi_i(a) = \max\{0, |a - y_i| - \nu\}$ for some predefined insensitivity parameter $\nu > 0$.

Dual formulation: $D(\alpha^*) = \max_{\alpha \in \mathbb{R}^d} D(\alpha)$ where

$$D(\alpha) = \frac{1}{n} \sum_{i=1}^{n} -\phi_i^*(-\alpha_i) - \frac{\lambda}{2} \left\| \frac{1}{\lambda n} \sum_{i=1}^{n} \alpha_i x_i \right\|^2$$

where $\phi_i^*(u) = \max_z (zu - \phi_i(z))$.

In fact we have

$$\begin{aligned}
\min_w P(w) &= \min_w \left[\frac{1}{n} \sum_{i=1}^{n} (\phi_i(w^T x_i) + \alpha_i (w^T x_i)) + \frac{\lambda}{2} \left(\|w\|^2 - \frac{2}{\lambda n} \sum_{i=1}^{n} \alpha_i w^T x_i \right) \right] \\
&= \min_w \left[\frac{1}{n} \sum_{i=1}^{n} (\phi_i(w^T x_i) + \alpha_i (w^T x_i)) + \frac{\lambda}{2} \left\| w - \frac{1}{\lambda n} \sum_{i=1}^{n} \alpha_i x_i \right\|^2 - \frac{\lambda}{2} \left\| \frac{1}{\lambda n} \sum_{i=1}^{n} \alpha_i x_i \right\|^2 \right] \\
&\geq \min_w \left[\frac{1}{n} \sum_{i=1}^{n} (\phi_i(w^T x_i) + \alpha_i (w^T x_i)) \right] - \frac{\lambda}{2} \left\| \frac{1}{\lambda n} \sum_{i=1}^{n} \alpha_i x_i \right\|^2 \\
&\geq \frac{1}{n} \sum_{i=1}^{n} \min_\omega (\phi_i(w^T x_i) + \alpha_i (w^T x_i)) - \frac{\lambda}{2} \left\| \frac{1}{\lambda n} \sum_{i=1}^{n} \alpha_i x_i \right\|^2 \\
&\geq \frac{1}{n} \sum_{i=1}^{n} -\max_z (-\phi_i(z) - \alpha_i z) - \frac{\lambda}{2} \left\| \frac{1}{\lambda n} \sum_{i=1}^{n} \alpha_i x_i \right\|^2 \\
&= \frac{1}{n} \sum_{i=1}^{n} -\phi_i^*(-\alpha_i) - \frac{\lambda}{2} \left\| \frac{1}{\lambda n} \sum_{i=1}^{n} \alpha_i x_i \right\|^2 = D(\alpha)
\end{aligned}$$

for any $\alpha \in \mathbb{R}^d$.

So $\min_w P(w) \geq \max_\alpha D(\alpha)$. Set $w(\alpha) = \frac{1}{\lambda n} \sum_{i=1}^{n} \alpha_i x_i$, then strong duality holds and

$$w(\alpha^*) = w^* \ , \ P(w^*) = D(\alpha^*) \ .$$

Algorithm 7.9 Stochastic Dual Coordinate Ascent

Input: Input Initialization α_0, $w_0 = w(\alpha_0)$
for $k = 0, 1, ..., K-1$ **do**
  Randomly sample $i_k \in \{1, 2, ..., n\}$
  Find $\Delta\alpha_{i_k} = \arg\max_z \left\{ -\phi^*_{i_k}(-\alpha_{k,i} + z) - \frac{\lambda n}{2} \left\| w_k + \frac{1}{\lambda n} z x_{i_k} \right\|^2 \right\}$
  Update $\alpha_{k+1} = \alpha_k + \Delta\alpha_{i_k} e_{i_k}$, $w_{k+1} = w_k + \frac{1}{\lambda n} \Delta\alpha_{i_k} x_{i_k}$
end for
Output (Average Option): $\bar{\alpha} = \frac{1}{K - K_0} \sum_{i=K_0+1}^{K} \alpha_i$ and $\bar{w} = w(\bar{\alpha}) = \frac{1}{K - K_0} \sum_{i=K_0+1}^{K} w_{k-1}$, return $\bar{w}$
Output (Random Option): Let $\bar{\alpha} = \alpha_k$ and $\bar{w} = w_k$ for some random $k \in K_0 + 1, ..., K$, return $\bar{w}$

7.4 Optimization with Constraints and Discontinuous Objective Function

7.4.1 Subgradient Projection Method, Frank–Wolfe Method

References:

Levitin, E.S., Polyak, B.T., Constrained Minimization Methods, *USSR Computational Mathematics and Mathematical Physics*, 1966, **6**(5), pp. 1–50.

Frank, M., Wolfe, P., An algorithm for quadratic programming, *Naval Research Logistics (NRL)*, 1956, **3**(1–2), pp. 95–110.

The subgradient $\partial f(x_0)$ for a convex function at point x_0 is defined as

$$\partial f(x_0) = \{c \in \mathbb{R} : f(x) - f(x_0) \geq c(x - x_0)\} \ .$$

Algorithm 7.10 Subgradient Projection Method

Input: Input Initialization x_0; Convex Set $\mathcal{X}$; learning rate $\eta > 0$
for $k = 0, 1, ..., K-1$ **do**
  Randomly sample a subgradient $g_k \in \partial f(x_k)$
  Update $y_{k+1} = x_k - \eta g_k$
  Project $x_{k+1} = \Pi_{\mathcal{X}}(y_{k+1})$, $\Pi_{\mathcal{X}}(y) = \arg\min_{x \in \mathcal{X}} \|y - x\|$
end for
Output: x_K

The projection $\Pi_{\mathcal{X}}(y)$ can be hard to calculate in practical cases. However, subgradient projection = first gradient descent then projection; Frank–Wolfe = combine gradient descent with projection to constraint.

How to do that? GD is an optimal choice of search direction via first–order Taylor expansion:

$$f(x+td) = f(x) + t\nabla f(x+\gamma td)^T d \text{ for some } \gamma \in (0,1) \ .$$

If $d^T \nabla f(x) < 0$, then d is a descent direction.
"Steepest Descent": $d_{\text{optimal}} = \arg \inf_{\|d\|=1} d^T \nabla f(x) = -\dfrac{\nabla f(x)}{\|\nabla f(x)\|}$.
Want to incorporate the constraint information: how to do that?

$$d_{\text{optimal}} = \arg \inf_{d \in \mathcal{X}} d^T \nabla f(x) \ .$$

Algorithm 7.11 Frank–Wolfe Method

1: **Input**: Input Initialization $x_0 \in \mathcal{X}$; Convex Set $\mathcal{X}$; weights $\eta_k > 0$
2: **for** $k = 0, 1, ..., K-1$ **do**
3: Calculate $\nabla f(x_k)$
4: Update $d_k = \arg \inf_{d \in \mathcal{X}} d^T \nabla f(x_k)$
5: Update $x_{k+1} = (1-\gamma_k)x_k + \gamma_k d_k$
6: **end for**
7: **Output**: x_K

7.4.2 ADMM

Reference: Boyd, S., Parikh, N., Chu, E., Peleato, B., Eckstein, J., Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers, *Foundations and Trends in Machine Learning*, Vol. 3, No. 1 (2010), pp. 1–122.

Equality–constrained convex optimization problem:

$$\text{minimize } f(x) \text{ subject to } Ax = b \ .$$

Augmented Lagrangian.

$$L_\rho(x,y) = f(x) + y^T(Ax-b) + \frac{\rho}{2}\|Ax-b\|_2^2 \ .$$

The augmented Lagrangian can be viewed as the (unaugmented) Lagrangian associated with the problem

$$\text{minimize } f(x) + \frac{\rho}{2}\|Ax-b\|_2^2 \text{ subject to } Ax = b \ .$$

This problem can be solved by dual gradient ascent:

$$\begin{aligned} x^{k+1} &\equiv \arg\min_x L(x, y^k) \ , \\ y^{k+1} &\equiv y^k + \rho(Ax^{k+1} - b) \ . \end{aligned}$$

Alternating Direction Method of Multipliers (ADMM).

Optimization problem has the form

$$\text{minimize } f(x) + g(z) \text{ subject to } Ax + Bz = c \ .$$

Optimal value

$$p^* = \inf\{f(x) + g(z); Ax + Bz = c\} \ .$$

Augmented Lagrangian can be formed as

$$L_\rho(x, z, y) = f(x) + g(z) + y^T(Ax + Bz - c) + \frac{\rho}{2}\|Ax + Bz - c\|_2^2 \ .$$

This problem can be solved by dual gradient ascent:

$$\begin{aligned} (x^{k+1}, z^{k+1}) &\equiv \arg\min_{x,z} L_\rho(x, z, y^k) \ , \\ y^{k+1} &\equiv y^k + \rho(Ax^{k+1} + Bz^{k+1} - c) \ . \end{aligned}$$

Unlike the classical dual gradient ascent which minimizes the augmented Lagrangian jointly with respect to the two primal variables x and z, ADMM updates x and z in an alternating or sequential fashion, which accounts for the term *alternating direction*.

Algorithm 7.12 Alternating Direction Method of Multipliers

1: **Input**: Input Initialization (x^0, z^0), dual variable y^0, parameter $\rho > 0$
2: **for** $k = 0, 1, ..., K-1$ **do**
3: Calculate $x^{k+1} \equiv \arg\min_x L_\rho(x, z^k, y^k)$
4: Calculate $z^{k+1} \equiv \arg\min_z L_\rho(x^{k+1}, z, y^k)$
5: Calculate $y^{k+1} = y^k + \rho(Ax^{k+1} + Bz^{k+1} - c)$
6: **end for**
7: **Output**: (x^K, z^K)

7.4.3 Proximal operators and proximal algorithms

Reference: Parikh, N., Boyd, S., Proximal Algorithms, *Foundations and Trends in Optimization*, Vol. 1, No. 3 (2013), pp. 123–231.

Also see http://stanford.edu/~boyd/papers/pdf/prox_slides.pdf

Let $R = R(v)$ be a convex function. With respect to the convex function R the proximal operator $\text{prox}_R(w) : \mathbb{R}^d \to \mathbb{R}^d$ is defined as

$$\text{prox}_R(w) = \arg\min_v \left(R(v) + \frac{1}{2}\|w - v\|^2 \right) \ .$$

If $R(w) = 0$, then $\text{prox}_0(w) = w$; if $R(w) = aI_C(w)$ (where $C \subseteq \mathbb{R}^d$ is usually a convex set, and the indicator function is defined as in (2.5)), then $\text{prox}_{aI_C}(w) = \arg\min_{v \in C} \|w - v\|^2$ degenerates to a projection operator.

We have

$$p = \text{prox}_R(w) \Longleftrightarrow w - p \in \partial R(p) \ .$$

Also

$$\text{prox}_{aR}(w) = \arg\min_v \left(R(v) + \frac{1}{2a}\|w - v\|^2 \right) \ .$$

Convex optimization problem

$$\min_{w \in \mathcal{W}} f(w) = l(w) + R(w) \ ,$$

where $l(w)$ is a convex differentiable function and $R(w)$ is non–differentiable convex function (such as L_1–regularization term).

Algorithm 7.13 Proximal Gradient Descent

1: **Input**: Input Initialization w_0, convex set $\mathcal{W}$, learning rates $\eta_k > 0$
2: **for** $k = 0, 1, ..., K-1$ **do**
3: Calculate $\nabla l(w_k)$
4: Update parameter $v_{k+1} = w_k - \eta_k \nabla l(w_k)$
5: Apply proximal operator $w_{k+1} = \text{prox}_{\eta_k R}(v_{k+1})$
6: **end for**
7: **Output**: w_K

7.5 Optimization and Generalization

7.5.1 Generalization Bounds in Statistical Machine Learning

Consider the supervised learning framework. Let the training sample (x_1, y_1), ..., (x_N, y_N) follow the same distribution $(x, y) \sim F(x, y)$ under probability measure $\mathbf{P}$ (or $\mathbf{P}_{(x,y)}$, and the corresponding expectation is $\mathbf{E}$ or $\mathbf{E}_{(x,y)}$). Let the loss functions be $\ell_i(\theta) = \ell((x_i, y_i), \theta)$, $\theta \in \mathbb{R}^d$, $i = 1, 2, ..., N$. Define the empirical loss function

$$L_N(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell_i(\theta) \ . \tag{7.9}$$

Let the local minimizer of $L_N(\theta)$ be θ_N^*, which may not be unique (in this case we further denote them as $\theta_N^{*,1}, ..., \theta_N^{*,k}$). Then the empirical loss function evaluated at θ_N^* is $L_N(\theta_N^*)$. The *generalization gap* is defined to be

$$\text{Gap}_N \equiv \left| L_N(\theta_N^*) - \mathbf{E}_{(x,y)} \ell((x, y), \theta_N^*) \right| = \left| \frac{1}{N} \sum_{i=1}^{N} \ell((x_i, y_i), \theta_N^*) - \mathbf{E}_{(x,y)} \ell((x, y), \theta_N^*) \right| \ . \tag{7.10}$$

We further define the *two-sided generalization gaps* as

$$\mathrm{Gap}_N^+ \equiv L_N(\theta_N^*) - \mathbf{E}_{(x,y)}\ell((x,y),\theta_N^*) \ , \tag{7.11}$$

$$\mathrm{Gap}_N^- \equiv \mathbf{E}_{(x,y)}\ell((x,y),\theta_N^*) - L_N(\theta_N^*) \ . \tag{7.12}$$

In classical statistical learning theory, the basic estimates of generalization gap bounds [2] are bounding Gap_N in (7.10) in terms of features of the family of functions that consists of the loss functions $\ell_i(\theta)$ in which we vary the parameters θ and the training data points (x_i, y_i), $i = 1, 2, ..., N$. Typical such features are the *Vapnik-Chervonenkis dimension* (VC dimension) and the *Rademacher complexity.*

(a) VC dimension. Let the loss function be $\ell((x,y),\theta) = \mathbf{1}(h(x,\theta) \neq y)$ for a family of functions $h(x,\theta)$, $\theta \in \Theta$. Then we define the *growth function* as

$$\Pi_\Theta(N) = \max_{x_1,...,x_N,\theta\in\Theta} |\{(h(x_1,\theta),...,h(x_N,\theta)),\theta \in \Theta\}| \ . \tag{7.13}$$

Then we have the following bound for the gap Gap_N in (7.10):

$$\mathbf{P}\left(\mathrm{Gap}_N > \varepsilon\right) \leq 4\Pi_\Theta(2N)\exp\left(-\frac{N\varepsilon^2}{8}\right) \ . \tag{7.14}$$

The *VC dimension* is then defined using the growth function

$$VC(\Theta) = \max\{N : \Pi_\Theta(N) = 2^N\} \ . \tag{7.15}$$

Suppose $VC(\Theta) = d$, then for any $N > d$, $0 < \delta < 1$ we obtain the generalization gap estimate in terms of VC-dimension as

$$\mathbf{P}\left(\mathrm{Gap}_N^- \leq \sqrt{\frac{8d\ln\frac{2eN}{d} + 8\ln\frac{4}{\delta}}{N}}\right) \geq 1-\delta \ . \tag{7.16}$$

The above bound is distribution-free and data-independent. The notion of VC dimension fails to be able to explain the effectiveness of over-parameterized deep neural network models, since in that case the VC dimension blows up yet the generalization gap remains reasonably small.

(b) Rademacher complexity. Let the loss function be $\ell((x,y),\theta) = \mathbf{1}(h(x,\theta) \neq y)$ for a family of functions $h(x,\theta)$, $\theta \in \Theta$. Given the training data points $(x_1,y_1),...,(x_N,y_N)$, the empirical Rademacher complexity is defined as

$$\widehat{R}_{x_1,...,x_N}(\Theta) = \mathbf{E}_{\sigma_1,...,\sigma_N}\left[\sup_{\theta\in\Theta}\frac{1}{N}\sum_{i=1}^N \sigma_i h(x_i,\theta)\right] \ , \tag{7.17}$$

where the Rademacher variables $\sigma_1,...,\sigma_N$ independently take values 1 or -1 with probabilities 1/2. Then we have the gap estimate

[2] see Vapnik, V., *Statistical Learning Theory*, Wiley-Interscience, first edition, 1988.

$$\mathbf{P}\left(\mathrm{Gap}_N^- \leq 2\widehat{R}_{x_1,\ldots,x_N}(\Theta) + 3\sqrt{\frac{\ln\frac{2}{\delta}}{2N}}\right) \geq 1-\delta\ . \tag{7.18}$$

Again, the above generalization error bound only makes use of the whole parameter space $\theta \in \Theta$ instead of specifying the information provided by the training algorithm, such as SGD. Indeed, if we run the optimization process like SGD, which can be regarded as a search algorithm of θ_N^*, we may end up in finding among many optimizers $\theta_N^{*,1}, \ldots \theta_N^{*,k}$ a specific one that achieves good generalization properties.

7.5.2 Training Matters: Connecting Optimization with Generalization

When applying the classical gap estimates such as (7.16) and (7.18) to deep learning models, the immense size of the number of parameters will make the complexity measuraments such as VC-dimensional and Rademacher complexity very large, and thus estimates (7.16) and (7.18) become very rough and useless in practice. This phenomenon is called the "curse of dimensionality", indicating that deep models may overfit the training data and cannot generalize well on test data. However, in practical experiments people found that deep learning models almost never have this bad generalization, and instead, models that are trained by such optimization methods as SGD generalizes very well on test data, with test accuracies often reaching 80-85%+. This seemingly mysterious phenomenon has been one of the major challenges of the theory of deep learning. Currently (at the writing of these lecture notes, in the middle of 2020) we have made only shallow understanding of this phenomenon. It is commonly believed that such strange behavior may be due to the interaction of two effects: the *implicit regularization* of the optimization dynamics and the *network architecture*.

A few papers/preprints suggested:

Cao, Y. and Gu, Q., Generalization Bounds of Stochastic Gradient Descent for Wide and Deep Neural Networks, *NeurIPS*, 2019.

Mei, S. and Montanari, A., The generalization error of random features regression: Precise asymptotics and double descent curve, arXiv:1908.05355 [math.ST]

Jacot, A., Gabriel, F., and Hongler, C., Neural Tangent Kernel: Convergence and generalization in neural networks, NIPS, 2018.

Wu, J., Hu, W., Xiong, H., Huan, J., Braverman, V. and Zhu, Z., On the Noisy Gradient Descent that Generalizes as SGD, arXiv:1906.07405 [cs.LG]

7.5.3 Experiments: Real Data Sets

• CIFAR-10 is a common benchmark in machine learning for image recognition. The CIFAR-10 dataset consists of 60000 32×32 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images.

The dataset is divided into five training batches and one test batch, each with 10000 images. The test batch contains exactly 1000 randomly-selected images from each class. The training

batches contain the remaining images in random order, but some training batches may contain more images from one class than another. Between them, the training batches contain exactly 5000 images from each class.

The CIFAR-10 and CIFAR-100 datasets are introduced at this webpage

http://www.cs.toronto.edu/~kriz/cifar.html

• Convolutional Neural Networks: see

http://cs231n.github.io/convolutional-networks/

• Use TensorFlow to train and evaluate a convolutional neural network (CNN) on both CPU and GPU. We also demonstrate how to train a CNN over multiple GPUs. Availabe at

https://github.com/tensorflow/models/tree/master/tutorials/image/cifar10

https://github.com/keras-team/keras/blob/master/examples/cifar10_cnn.py

Detailed instructions on how to get started available at:

https://www.tensorflow.org/tutorials/images/deep_cnn

• Make use of Google-Colab for GPU/TPU support. See

https://medium.com/deep-learning-turkey/google-colab-free-gpu-tutorial-e113627b9f5d

• The MNIST database of handwritten digits, has a training set of 60,000 examples, and a test set of 10,000 examples. It is a subset of a larger set available from NIST. The digits have been size-normalized and centered in a fixed-size image. It is a good database for people who want to try learning techniques and pattern recognition methods on real-world data while spending minimal efforts on preprocessing and formatting. See

http://yann.lecun.com/exdb/mnist/

https://github.com/keras-team/keras/blob/master/examples/mnist_cnn.py

• More Datasets and Error/Accuracy results in recent Papers. See

http://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html.

Bibliography

[1] https://en.wikipedia.org/wiki/Matrix_factorization_(recommender_systems).

[2] https://github.com/huwenqing0606/Computational_Optimization.

[3] Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. *arXiv preprint arXiv:1811.03962*, 2018.

[4] Jacot Arthur, Gabriel Franck, and Hongler Clement. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in Neural Information Processing Systems, arXiv:1806.07572*, 31:8571–8580, 2018.

[5] Alon Brutzkus, Amir Globerson, Eran Malach, and Shai Shalev-Shwartz. Sgd learns over–parameterized networks that provably generalize on linearly separable data. *ICLR, arXiv:1710.10174*, 12:1–12, 2018.

[6] P. Chaudhari and S. Soatto. Stochastic gradient descent performes variational inference, converges to limit cycles for deep networks. *ICLR*, 2018.

[7] Y. Chi, Y.M. Lu, and Y. Chen. Nonconvex optimization meets low-rank matrix factorization: An overview. *IEEE Transactions on Signal Processing*, 67(20), 2019.

[8] Simon S. Du, Jason D. Lee, Haochuan Li, Liwei Wang, and Xiyu Zhai. Gradient descent finds global minima of deep neural networks. *arXiv preprint arXiv:1811.03804*, 2018.

[9] Simon S. Du, Xiyu Zhai, Barnabás Póczos, and Aarti Singh. Gradient descent provably optimizes over-parameterized neural networks. *arXiv preprint arXiv:1810.02054*, 2018.

[10] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.

[11] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* Springer, 2009.

[12] Catherine F. Higham and Desmond J. Higham. *Deep Learning: An Introduction for Applied Mathematicians.* arXiv:1801.05894[math.HO], 2018.

[13] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. 2024.

[14] A. Karpathy. https://github.com/karpathy/nanochat.

[15] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.

[16] C.J Li, M. Wang, H. Liu, and T. Zhang. Near–optimal stochastic approximation for online principal component estimation. *Mathematical Programming*, 167(1):75–97, 2018.

[17] Y. Nesterov. A method of solving a convex programming problem with convergence rate $o(1/k^2)$. *Soviet Math Dokl*, 27(2), 1983.

[18] Y. Nesterov. *Introductory Lectures on Convex Optimization*. Springer, 2004.

[19] E. Oja. Simplified neuron model as a principal component analyzer. *Journal of mathematical biology*, 15(3):267–273, 1982.

[20] B. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[21] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the Convergence of Adam and Beyond. *ICLR*, 2018.

[22] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[23] W. Su, S. Boyd, and E.J. Candés. A differential equation for modeling nesterov's accelerated gradient method: theory and insights. *Journal of Machine Learning Research*, 17:1–43, 2016.

[24] I. Sutskever, J. Martens, G. Dahl, and G. Hinton. On the importance and momentum in deep learning. *ICML*, 2013.

[25] TensorFlow. Automatic differentiation and gradient tape. https://www.tensorflow.org/tutorials/customization/autodiff. Accessed: 2026-02-17.

[26] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—rmsprop: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012. Lecture notes.

[27] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, 2000.

[28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. arXiv:1706.03762.

[29] Matthew D. Zeiler. Adadelta: An adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.

[30] Xiaojin Zhang, Yuchen Yu, Lin Wang, and Quanquan Gu. Learning one-hidden-layer relu networks via gradient descent. *arXiv preprint arXiv:1806.07808*, 2018.